Mt. San Antonio College
Electronics 74 Instruction Notes and Lab Manual

Contributors

John Tsai
Jacob Christ
Brian Dobrovodsky
Derrick Ramos
Angel Ramos

Introduction

<Picture of everyone>

Welcome to the exciting world of microcontroller development. Microcontrollers are the work horse of the electronics and computer industries today and surround us almost invisibly yet are ubiquitous in our daily lives. On my desk as I type this and look around I estimate that there are between ten to thirty devices in my reach that have microcontrollers in them.

Wireless mouse and Keyboard (Three microcontrollers, one for the keyboard, one for the mouse and one in the receiver).
Laptop Computer (At least one, probably three or four)
4 hard drives (At least one per drive)
2 USB Hubs (May actually be an ASIC)
USB Headset (May actually be an ASIC)
KVM Switch (At least one)
Programmable Power Supply
Uninterruptable Power Supply
Cell Phone
Desk Phone
Digital Wrist Watch
And many of my little products that I am in the process of developing

When I think about my home, in my Kitchen I probably have at a minimum ten devices that have microcontrollers in them.

Coffee Pot
Refrigerator Ice Maker
Dish Washer
Microwave Oven
Stove Top
Cooking Timer
Smoke Detector
Wine Cooler
Bread Maker
2 Trash Cans (sensor to detect presents of refuge to open can)

Microcontrollers are the heart of small inexpensive electronic devices that give them the seemingly intelligent life that makes our modern world convenient and exciting to live in.

This class is based on the PIC18F4620 microcontroller manufactured by Microchip Technologies, Inc. (http://www.microchip.com), the largest manufacturer of microcontrollers in the world. To put in perspective how prolific microcontrollers are, in 2007 Microchip Technologies sold 1 billion PIC microcontrollers worldwide. That is one microcontroller for every six people on the planet in 2007 alone. And Microchip is not alone; there are at least ten different companies that manufacture microcontroller products.

This material for this class has been developed for second year electronic students and assumes that the students have a cursory knowledge of digital electronics and have never been exposed to microcontrollers or any programming languages. As exciting as microcontrollers are, there is a price that comes with massive usage of the devices. There are thousands of people that are working in and on microcontroller and programming technologies, genereting hundreds of new ideas daily which makes for a lot to learn for the uninitiated.

At minimum, you have to understand how the internals of the microcontrollers work. This includes built-in functions of microcontrollers, knowledge of electronic components and C Programming Language(or Assembly language, if you prefer it). This book will provide enough information for you to get start in Embedded programming. We will show concepts through examples and step by step learning. Make sure every reader will have a solid foundation for the art of embedded programming after finishing all the laboratories.

Jacob Christ & John Tsai
January 6th, 2009

What is a microcontroller?

<picture of Microcontroller>

A microcontroller is typically a monolithic silicone device that is packaged in a single chip that provides the developer with all the tools needed to build a complete computer system with the addition of very few additional components.

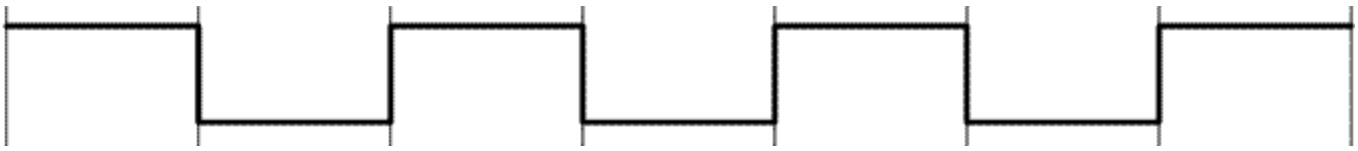
Microprocessor

<Picture of CPU, microcontroller>

Information processing unit of the chip often referred to as the CPU. The CPU reads the stored program in program memory, interprets the instructions that are primarily used to direct the flow of information into and out of data memory and into and out of peripherals. Additionally program flow control is handled by the

microprocessor.
In desktop and laptop computers the microprocessor is typically a standalone chip and all the additional features the make up a microcontroller are relegated to other chips that typically sit on the mother board. The ability to put different technologies onto a single chip is blurring what traditional computer architecture and an embedded architecture look like and some computers on a chip have existed for quite some time that allow for incredible size reduction of computers allowing the creation of palm top computers.

Clock



The clock for the microcontroller is a stable oscillator that provides the clockwork timing needed for the digital circuitry in the entire chip. Not all microcontrollers have a built in oscillator and an external crystal is needed for these devices to operate properly.

Program Memory

<Picture of memory>

Program memory is a non-volatile memory used to hold the application or operating system program for the embedded device. Non-volatile refers to ability for the memory to retain its value without out power applied to the chip. Examples of types of non-volatile memory are:
ROM (Read Only Memory, programmed at the factory and not erasable)
PROM or OTP (Programmable Read Only Memory or One Time Programmable memory that cannot be erased once programmed)
EPROM (Erasable Programmable Read Only Memory)
EEPROM (Electronically (byte) Erasable Programmable Read Only Memory)
Flash Memory (Electronically (bank) Erasable Programmable Read Only Memory)
The program stored in program memory is often referred to as firmware as opposed to software that would be loaded off of a hard drive on a PC.

Data Memory

Data memory is volatile memory used to hold program data while the program is running. Volatile memory will not retain its values without power applied. Volatile memory is often called RAM (Random Access Memory) or RWM (Read Write Memory). The term RAM is quite popular but doesn't quite tell the whole story since ROM type memory is also random accessible. The better term is RWM since the memory can be altered on the fly while a program is running without special considerations that are necessary to erase a ROM chip.
There are at least two common types of RWM, Static and Dynamic. Static RWM is built out of transistors and is very fast and priced accordingly, dynamic ram is built out of capacitors and is not as fast at static RWM but can be built much denser and at a lower cost. Also the nature of storing values as charges in a capacitor has the added complexity of requiring a dynamic ram controller to keep the correct values refreshed while the memory is not in use.
Microcontrollers internal data memory is typical static RWM. Dynamic RWM is most commonly used as the main memory of a PC. Side note, the cache of a PC microprocessor is made of static RWM and is on the microprocessor, but this was not always the case. Cache memory of older PC's was external from the microprocessor.

Other Non-Volatile Storage

In addition to program and data memory, some microcontrollers have additional non-volatile storage that can be used to store program data that survives power outages.

Peripherals

The peripherals of the microcontroller are what really differentiate a microprocessor from a microcontroller. The peripherals are used for the control and sensing of many things on a single chip. Listed below are some common peripherals that can be found on a microcontroller, however the list is by no means complete.

Parallel Ports

Serial Ports

UART

SPI

I2C

USB

Timers / Counters

Analog Circuitry

ADC's

Comparators

What can a microcontroller do?

What are the limits of your imagination?

Table of Contents

Lab 1 - Installing and Using Development Tools
Lab 2 - Digital I/O, aka Knight Rider + Buttons
Lab 3 - Knight Rider with Interrupt/ Sub Routine
Lab 4 - Introduction to Asynchronous Serial - PIC Hardware
Lab 5 - Introduction to Asynchronous Serial - Serial ISR's
Lab 6 - Introduction to Asynchronous Serial
Introduction to Parser
Lab 7 - Interfacing with analog electronic devices
Lab 8 - Introduction to Serial Peripheral Interface/Digital Analog Converter
Lab 9 - Introduction to Serial Peripheral Interface/ LCD Display and Internal EEPROM

Lab 10 - Timer and Real Time Clock
Lab 11 - Introduce to PWM / Buzzer
Lab 12 - External EEPROM and other Devices

Applications

Appendix

Lab 1 - Installing and Using Development Tools (Get From Derrick)

<Mt.SAC DEV board PIC>

Let get started! In our first lab, we are going to get familiar with Mt. SAC DEV board and install and setup PIC development environment in your computer. We will also demonstrates the way to compile, and upload the firmware to microcontrollers. Make sure

you understanding everything before move on to the next lab. We will use these skills frequently in this book and whenever you are doing PIC developments. After this lab, you will be ready to learn PIC and Boost C!

Materials:

1. MSAC Dev Board

2. PC With CD-ROM Drive and at least one USB port

References:

PIC18F2525/2620/4525/4620 data sheet

BoostC C Compiler for PICmicro Reference Manual

MT.SAC DEV Board schematic diagram

Goals:

Ability to install and verify functionality of development tools required to develop embedded applications on a PC.

Procedure:

1. Installing BoostC

2. Installing MSAC Control

3. Loading project in BoostC

4. Compiling / Linking

5. Uploading Firmware to Dev Board

Study Notes:

Understanding PIC18F4620

<PIC of PIC18F4620>

PIC18F4620 is an **8-bit** PIC microcontroller. The speed of CPU is **10 MIPS**. Using C compiler optimized **RISC architecture**(**Harvard** architecture). 64 Kilobytes flash memory. Internal EEPROM: 1024KB. It comes with internal oscillator, **31kHz to 8MHz**, up to 32 MHz with 4X PLL. Operating voltage range from **2.0V to 5.5V**.

PIC18F4620 has many built-in functions, here is the peripherals of PIC18F4620:

3 Programmable external Interrupts
4 Input Change Interrupts
1 Capture/Compare/PWM (CCP) modules
1 Enhanced Capture/Compare/PWM (ECCP) modules

Master Synchronous Serial Port (MSSP) module ((SPI and I2C)
Enhanced Addressable USART module
10-bit Analog Digital Converter 13 channels.
Dual Analog Comparators with Input Multiplexing
Programmable 16-Level High/Low-Voltage Dectection (HLVD)module
Fail-Safe Clock Monitor
Watchdog Timer
Programmable Brown-out Reset

(For detail, please check out Datasheet)

(PIC18F4620 data sheet from www.Microchip.com)

These are technical terms and definitions might help you to read microcontroller and microprocessor datasheet :

MIPS - Million instructions per Second

CISC - A **complex instruction set computer** (**CISC**, pronounced like "*sisk*") is a microprocessor instruction set architecture (ISA) in which each instruction can execute several low-level operations, such as a load from memory, an arithmetic operation, and a memory store, all in a single instruction. The term was retroactively coined in contrast to reduced instruction set computer (RISC).

RISC - The acronym **RISC** (pronounced *risk*), for **reduced instruction set computing**, represents a CPU design strategy emphasizing the insight that simplified instructions which "do less" may still provide for higher performance *if* this simplicity can be utilized to make instructions execute very quickly. Many proposals for a "precise" definition have been attempted, however, the term is being slowly replaced by the more descriptive **load-store architecture** (see below). Well known RISC families include Alpha, ARC, ARM, AVR, MIPS, PA-RISC, PIC, Power Architecture (including PowerPC), SuperH, and SPARC.

Harvard architecture - a computer architecture with physically separate storage and signal pathways for instructions and data

The von Neumann architecture **-** a processing unit and a single separate storage structure to hold both instructions and data

Pharse Lock Loop(PLL) - Use as Frequency Multiplier, for users who wish to use a lower frequency
oscillator circuit or to clock the device up to its highest rated frequency from a crystal oscillator

(From Wikipedia.org & PIC18F4620 datasheet)

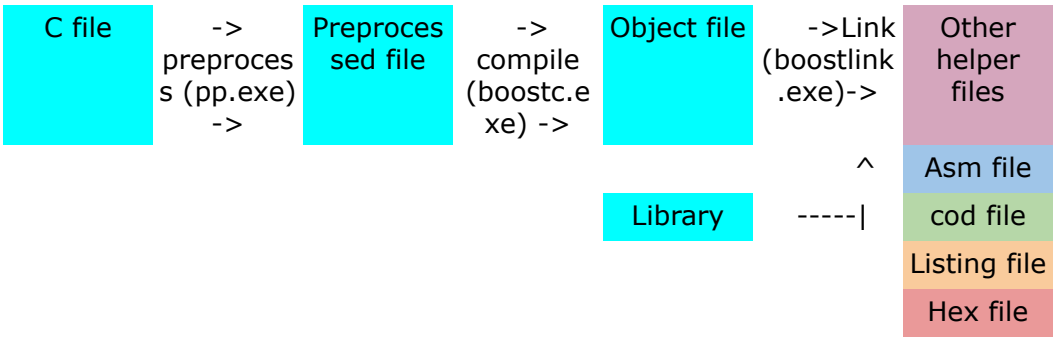
What is BoostC?

BoostC is one of the C compiler that works with PIC16, PIC18, and some PIC12 processors. BoostC is an ANSI C compatible
Compiler. However, it has some functions that is different than the standard C. It designed for Embedded Programming specifically

BoostC is a Higher-level programming language. Higher level programming language can

be divided into compiled language and interpreted language.
Interpreted language is the computer language will translate into machine language directly when the program executed. Compiled language like BoostC, requires preprocess, compile and link in order for computer or microprocessor to run.

Here is the compilation model and toolchain for BoostC



(From BoostC Manual p.13)
C Data Types:

In original C, there are few datatypes:

char	the data type capable of holding a character, a single byte.
int	an integer, the size of an integer data type is 2 bytes.
float	floating point, 4 bytes
double	double floating point

In addition, there are a number of qualifiers that can be applied to these basic types:

short int value1 -> 1 byte
long int value2 -> 4 bytes

(K&R 2.2 P.36)
However, the basic data type for BoostC is a little bit different...

Size	Type name	Specification
1 bit	bit, bool	boolean
8 bit	char	signed, unsigned
16 bit	short, int	signed, unsigned
32 bit	long	signed, unsigned

(BoostC manual Page.9)

Sometimes, You can use other For example PortD=0b00001000(Binary) you can write it as PortD = 0x08(Hex)
or PortD = 8 (Decimal), they mean the same thing. It depends on how programmer like to represent it

Here is the best use and formating for each type of data

Constant Definition	Formating	Best Use
Decimal	##	Default Value
Hexadecimal	0x##	Register Counter Value
Binary	0b########	Noncounter Register Value
ASCII	'#'	Human Interface Values

(123 PIC microcontroller experiments for the Evil genius. P.31)

In Class Assignment

1. Base on your observation, what devices are mount on Mt.SAC DEV. Board? List the components of Mt. SAC DEV. board and describe the function for each item.

2. Read PIC18F4620 datasheet and the article "What is Microcontroller" by Jacob Chirst. In your own word, describe the built-in functions of PIC18F4620.

Part II Start to do programming, Review from digital electronic

Before we learn to program the features of PICs, let's review TTL (or CMOS) IC from digital electronic class. In 7400 TTL ICs, there are chips contains AND gates, OR gates, and NOT gates...etc. Their inputs and outputs are fixed. PICs are programmable ICs which most of pins can be program as input or output. Logical gates can also be implemented to PICs, the sample below creates one AND gate, one OR gate and one NOT gate in PIC.

1. Type in the code below, name it **LogicalGate.c,** compile it and upload to the Mt.SAC DEV. Board.

```
#include <system.h>  //pic definition files required by boostc
   #pragma CLOCK_FREQ 32000000 //Set clock frequency

   //Configuration bits
   #pragma DATA _CONFIG1H, _OSC_INTIO67_1H & _FCMEN_OFF_1H & _IESO_OFF_1H
   #pragma DATA _CONFIG2L, _PWRT_OFF_2L & _BOREN_OFF_2L & _BORV_3_2L
   #pragma DATA _CONFIG2H, _WDT_OFF_2H & _WDTPS_1024_2H
   #pragma DATA _CONFIG3H, _MCLRE_ON_3H & _LPT1OSC_ON_3H & _PBADEN_OFF_3H
& _CCP2MX_PORTC_3H
   #pragma DATA _CONFIG4L, _STVREN_ON_4L & _LVP_ON_4L & _XINST_OFF_4L &
_DEBUG_OFF_4L
   #pragma DATA _CONFIG5L, _CP0_OFF_5L & _CP1_OFF_5L & _CP2_OFF_5L &
_CP3_OFF_5L
   #pragma DATA _CONFIG5H, _CPB_OFF_5H & _CPD_OFF_5H
   #pragma DATA _CONFIG6L, _WRT0_OFF_6L & _WRT1_OFF_6L & _WRT2_OFF_6L &
_WRT3_OFF_6L
   #pragma DATA _CONFIG6H, _WRTB_OFF_6H & _WRTC_OFF_6H & _WRTD_OFF_6H
   #pragma DATA _CONFIG7L, _EBTR0_OFF_7L & _EBTR1_OFF_7L & _EBTR2_OFF_7L &
_EBTR3_OFF_7L
   #pragma DATA _CONFIG7H, _EBTRB_OFF_7H

        void main()
        {
                 trisd = 0; //set all portd to output
                portd = 0; //set all portd to 0
```

```
		trisb = 1; //set all portb to input
		portb = 0; //set all portb to 0

		while( 1 )   // A endless while loop
		{
			portd.0 = portb.0 && portb.1;  //AND gate.  Input1->portb.0,
Input2->portb.1,Output->portd.0
			portd.1 = portb.2 || portb.3; //OR gate, Input1->portb.2,
Input2->portb.3,Output->portd.1
			portd.2 = ~portb.3; //NOT gate, Input->portb.3, Output->portd.2

		}
	}
```

In Class Assignment:

1. Create a program to emulate the diagram below in PIC.

Take Home Assignment:

1. Replace firmware in the PIC at home, and bring in board with different project running

2. Read "The Development of the C language" or "C (Programming Language)" from Wikipedia.org.

3. Search on-line, Find how people use PICs or MCUs in the real world application. how many different kind of PICs

Lab 2 - Digital I/O, aka Knight Rider + Buttons

<Knight Rider picture?>

Have you seen Knight Rider show on television? Knight Industry Three Thousand, or KITT is an artificial intelligence microprocessor installed in a highly advanced, very mobile, robot disguised as a 1982 Pontiac Firebird Trans Am. In this lab and next lab, we will show you how to program the KITT (Yes, I am serious! but only the light pattern part :-D). In order to learn programming the light pattern of KITT, we will show you the digital I/O functions of PIC18F4620.

Materials:

MT. SAC DEV BBOARD

References:

PIC18F2525/2620/4525/4620 data sheet

BoostC C Compiler for PICmicro Reference Manual

MT.SAC DEV Board schematic diagram

Goals:

Start from this Lab, you will learn many things not only about Microcontroller, but also practical use of electronic components and C programming.

For PIC, you will learn:

Control Digital I/O port output and Input in PIC

For Electronics and hardwares, you will learn:

LEDs

Buttons

For C Programming, you will learn:

1. Write your first C program in boostC

2. Grasp the purpose of the main() function, understand while loop and For loop in C programming, and variable array

3. Know the importance of documentation, indentations, and blank lines for readability of code

Part I - KnightRider

Study Note:

I/O pins is one of the most important feature of the PIC MCU.

<LED lights on Mt.SAC DEV board>

The PIC 18F4620 contains 5 I/O ports, portA, portB, portC, portD and portE. **Each port has two register**s associated with it **TRIS**(Tri state) and **port** register.

The TRIS register decides the port is configure as input or output. After the pins have been configured, you may read or write data to the port using the port register.

Writing one to a bit in tris register will make the corresponding bit or pin in port as input. Writing zero to a bit in tris register will make the corresponding bit or pin in port as output.

For example:

If we want to make PortD all outputs. We write the statement like this:

```
trisd = 0; //set all portd to output
```

If we want to set LED on PortD to off, we write the satatment like this:

```
portd = 0; //set all portd to 0
```

We can also set each pin individually to input and output:

For example:

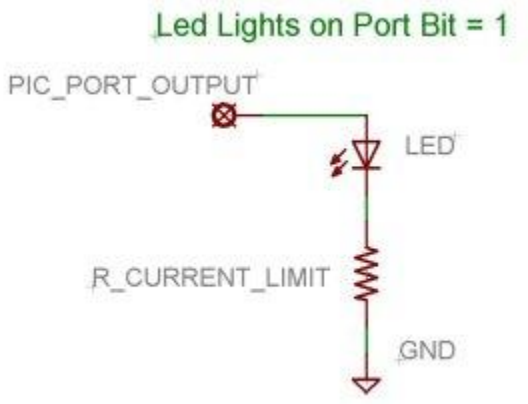
```
trisd = 0b10110100
```

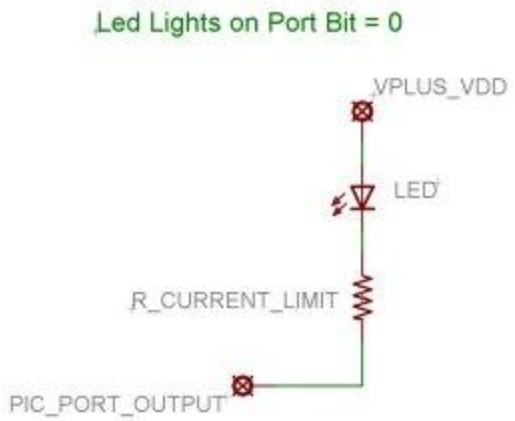
This statement will set PortD pin 3, pin5, pin6, and pin 8 to be input and pin1,pin2,pin4,pin7 to be output

(Read PIC 18F4620 Datasheet for detail)

Electronic Components:

LED - For microprocessor, The simplest form of output is LED. There are 2 ways to hook up LED





(http://www.opencircuits.com/A_Really_Basic_Guide_to_the_PIC_Microprocessor_and_BoostC)

Current Resistor for LED

From two diagram above, We see that besides LED itself, we also hook up an resistor to limit current. It is because Microchip Inc. states that the miaximum sink current of an ouput pin should not exceed 25ma. (Datasheet Page 1)

The maximum source current of an output pin also should be less or equal to 20ma.

The required value of the current limiting resistor for LED can be calculated as below:

R = (VS - VF)/IF

VS is the 5V supply voltage, VF is the Light Emit Diode voltage drop and IF is the LED forward current. If the LED current is 15mA, the
resistor will be:

R = (5 - 2)/15 = 200 ohm

C Programming Language:

Delay

Delay functions are not in standard C library. That are declared in boostc.h

void delay_us(unsigned char t) delay 1 micro second

void dekay_10us(unsigned char t) delay 10 micro second

void delay_100us(unsigned char t) delay 100 micro second

void dekay_ms(unsigned char t) delay 1 mili second

void delay_s(unsigned char t) delay 1 second

While Loop

A loop is the repeated execution of the same set of programming instruction until the relationship is false, the loop is ended. If you want to make infinite loop, never make the relationship false!

```
int LED_Count = 0;

while (LED_Count < 8)
{
   Portd=LED_Count;
   delay_ms(20);
        LED_Count++;
}
```

Here is the example for an infinite loop

```
int LED_Count = 0;
portd=0b00001111;
While (LED_Count < 1)
{
   portd =~portd;
   delay_ms(20);
}
```

Of course, the while loop we put in the main all the time in BoostC is also a endless while loop

```
while(1)  // It is always true
{
   //Your statement goes here
}
```

The I/O pins can be changed from input to output within a program by setting the tris register bit. For example, we write one

```
   void main()
   {
                while ( 1)
                {
                        trisd = 0b00000001;  //set trisd as input
                        while (portd.0 == 1)
                        {
                                //Do nothing until button on portd.0 pressed;
                        }
                        trisd = 0b00000000; ..set trisd as output
                        delay_s (3); //wait for 3 second
                        portd = 1; //turn on the LED on portd.0;
                }
   }
```

BoostC also allow us to choose individual bit in a register.
example: trisc.0 = 0; makes portc pin0 as output. trisd.3 = 1 makes portd

pin0 as input.

```
while ( 1 )
{
portd.0 = 1;
delay_ms(20);
portd.0 = 0;
delay_ms(20);
}
```

Procedure:

1. Create a sourceboost project called **KnightRider**

2. Make sure setting target to **PICF4620**(Settings->Target...), Active Toolsuite selection to **BoostC**(Setting->Toolsuite..)

3. Let's type in the C code below

```
#include <system.h>  //pic definition files required by boostc
#pragma CLOCK_FREQ 32000000 //Set clock frequency

//Configuration bits
#pragma DATA _CONFIG1H, _OSC_INTIO67_1H & _FCMEN_OFF_1H & _IESO_OFF_1H
#pragma DATA _CONFIG2L, _PWRT_OFF_2L & _BOREN_OFF_2L & _BORV_3_2L
#pragma DATA _CONFIG2H, _WDT_OFF_2H & _WDTPS_1024_2H
#pragma DATA _CONFIG3H, _MCLRE_ON_3H & _LPT1OSC_ON_3H & _PBADEN_OFF_3H
& _CCP2MX_PORTC_3H
#pragma DATA _CONFIG4L, _STVREN_ON_4L & _LVP_ON_4L & _XINST_OFF_4L &
_DEBUG_OFF_4L
#pragma DATA _CONFIG5L, _CP0_OFF_5L & _CP1_OFF_5L & _CP2_OFF_5L &
_CP3_OFF_5L
#pragma DATA _CONFIG5H, _CPB_OFF_5H & _CPD_OFF_5H
#pragma DATA _CONFIG6L, _WRT0_OFF_6L & _WRT1_OFF_6L & _WRT2_OFF_6L &
_WRT3_OFF_6L
#pragma DATA _CONFIG6H, _WRTB_OFF_6H & _WRTC_OFF_6H & _WRTD_OFF_6H
#pragma DATA _CONFIG7L, _EBTR0_OFF_7L & _EBTR1_OFF_7L & _EBTR2_OFF_7L &
_EBTR3_OFF_7L
#pragma DATA _CONFIG7H, _EBTRB_OFF_7H

void main()
{
   trisd = 0; //set all portd to output
        portd = 0; //set all portd to 0

        while( 1 )   // A endless while loop
        {

   // Turn on and Turn off LEDs and delay 20 mil-second, keep going and start over
again with a endless while loop

        portd = 0b00000001;  // Set Port d to 0b00000001;
  delay_ms(20);  // Delay 20 mil-second
        portd = 0b00000010;
```

```
        delay_ms(20);
        portd = 0b00000100;
  delay_ms(20);
        portd = 0b00001000;
        delay_ms(20);
        portd = 0b00000100;
  delay_ms(20);
        portd = 0b00100010;
        delay_ms(20);
        }
}
```

4. Compile the program and program the MT.SAC DEV Board and Check out the result

5. Create a New Project called **KnightRider2**, Change the codes in while loop to

```
while( 1 )
{
   portd = 0b10000001;
 delay_ms(20);
        portd = 0b01000010;
        delay_ms(20);
        portd = 0b00100100;
  delay_ms(20);
        portd = 0b00011000;
        delay_ms(20);
        portd = 0b00100100;
  delay_ms(20);
        portd = 0b01000010;
        delay_ms(20);
}
```

6. Compile the program and program the MT.SAC DEV Board and Check out the result

In Class Assignment:

1. Modified the program above and write your own LED flashing pattern

2. Understand how the program work and make clear comments for each pattern you write

Part II Introduce Array

Study Note:

Array is a data structure consisting of a group of elements that are accessed by indexing.

The example below create an variable array contains 5 student's grade in integer

```
int grade[5] = { 55, 75, 90, 40, 20};
```

In order to access the array

```
int a = grade [0];
```

a will be equal to 55

```
int b = grade [2];
 b will be equal to 90;
```

int will declare variable **grade** as integer variable. To decleare variable with [] means it is an Variable arrays. Array elements in C start at an index of zero, although we declare array to "5", the array will assign 55 to grade[0], 75 to grade[1], 90 to grade[2], 40 to grade[3], 20 to grade[4]. And there is no grade[5]

Note: To create an array without index number like **KnightRider3**, will create Dynamic Array, the size of array is decided by how many element you have in the array.

You can also declare char variable array this way

char MyName[] = "Jacob"

the array is placed in memory it looks like this:
[0] J
[1] a
[2] c
[3] o
[4] b
[5] \0

Procedure:

1. Create a New Project called **KnightRider3**, Change the codes in main()

```
void main()
{
    trisd = 0; //set all portd to output
    portd = 0; //set all portd to 0
    char Array[ ]={
                    0b10000001,
                       0b01000010,
                    0b00100100,
                    0b00011000,
                    0b00100100,
                    0b01000010,
                    0b10000001};
    while (1)
    {
        for (int i = 0; i<sizeof(Array);i++)
        {
            portd=Array[i];
            delay_ms(20);
        }
    }
}
```

In-Class Assignment:

1. Modified the program above and write your own LED flashing pattern

2. Understand how the program work and make clear comments for each pattern you write

PART III Binary Count

Study Note:

Binary Count

Here is the code in BoostC to make Pin5 of Port D(LED5) to turn on:

```
portd = 0b00010000;
```

however, you can also represent it in decimal;

```
portd = 16;
```

If you want to represent in hex, that is also fine

```
portd = 0x10;
```

Here is the conversion table for binary, octal, decimal and hexadecimal

Binary	0	1	10	11	100	101	110	111	1000	1001	1010	1011	1100	1101	1110	1111	10000	10001
Octal	0	1	2	3	4	5	6	7	10	11	12	13	14	15	16	17	20	21
Decimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Hexadecimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11

Binary Count is to count the number from 1 or 0 to any number you want in binary.

For Loop

To control a single for Loop the programmer need three expressions. The following is the format of a for Loops

```
for (startExpression; conditional; countExpression)
   {
      .. Block of one or more C statements
   }
```

Here is an example of For Loop

```
for (int count_down = 1; count_down >= 1; count_down --)
```

```
  {
   portd =count_down;
   }
```

Create a new project called **Binary Count**, here is the code for binary count

```
void main()
{
        trisd=0;
        char bc;
        while (1)
  {
           for( bc =0; bc<16; bc++)
           {
                   portd=bc;
                   delay_ms (500);
           }
   }
}
```

In Class Assignment

1. if you want to turn on LEDs on Pin2, pin 5 and pin 8, what do you do? Write the C code that turn on LEDs on Mt.SAC DEV. BOard.

2. Write BoostC codes that able to turn on LED 1, LED 3, LED 5, LED 7 and turn off LED2, LED4, LED6, LED 8 at same time, then turn on LED 2, LED 4, LED 6, LED 8 and turn off LED1, LED3, LED5, LED7. DON'T FORGET TO USEDELAY FUNCTION !

Part IV Input buttons, shift register, || and &&

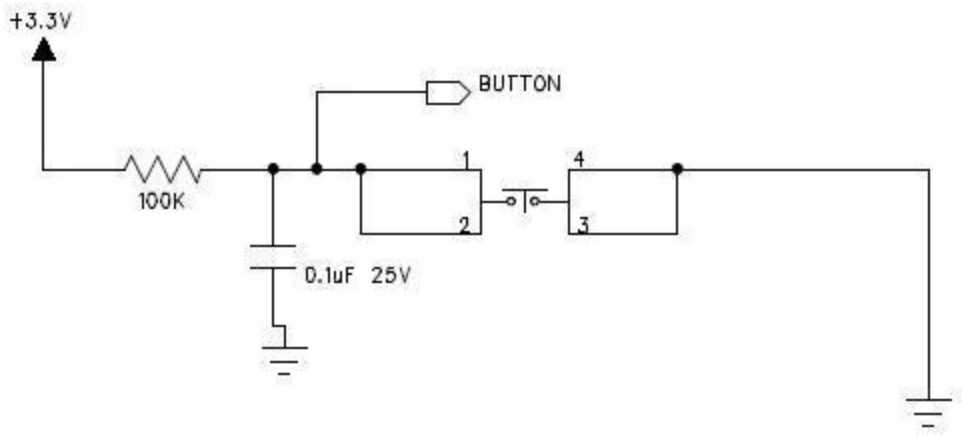
<Buttons on Mt.SAC DEV Board>

Study Note:

Hardware

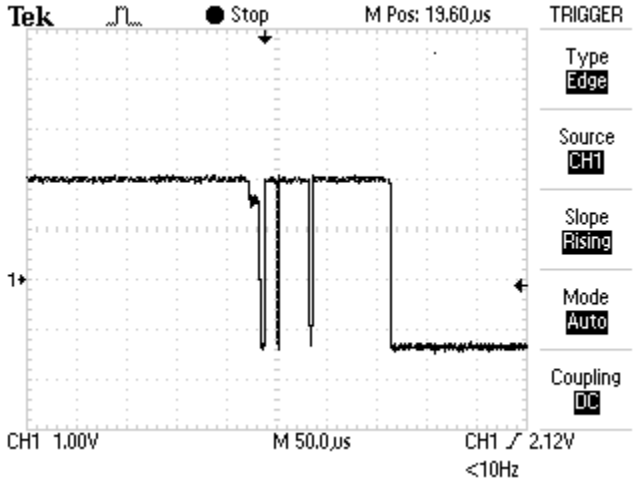
1. Switchs or buttons

The switch of MT.SAC DEV board hook up to MCU in this way



This simple switch setups allow you to change the voltage from high to low on your PIC pin when switch is pushed. we called it **pull-down**. If the circuit change the voltage from low to high, we call it **pull-up**

Every time a switch state changed from low to high or high to low, the contact bounce against each other. We can see that if when hook up Test Point for button on MT. SAC DEV board to oscilloscope. The picture is similar to below:



2. Shift Register

Shift Register >> and << will shift the bits in the the register the way you request

For example,

If port D is 0b00001100, and you have a statment in C:

portd =<< 2;

port D will be 0b00110000

If port B is 0b00001010, and you have a statment in C:

portb =>> 3;

portb will be 0b00000001

What happend to the 1 on the original 2nd digit? It has been erased!

3. || and &&

|| is just like "OR gate " you have learned from digital electronic class

Here is the truth table for || (OR gate)

A	0	1	0	1
B	0	0	1	1
X	0	1	1	1

For example

int A = 0;
int B = 1;

int X = (A || B)

X will contain 1

&& is just like "AND gate" you have learned from digital electronic class

A	0	1	0	1
B	0	0	1	1
X	0	0	0	1

For example

int A = 0;
int B = 1;

int X = (A && B)

X will contain 0

Note for Documentation

Good programs will contain comments as necessary to make the code easier interpret. It is important for other programmers will will need to understand the logic to make modifications to you code but also for yourself. You may have discovered in you past programming experince that if you go back to make changes to a program that you've written a while back, it may be hard to follow quickly some of the steps in the code.

A comment line can be included in C by bracketing all information that you don't want the compiler to taranslate with slashes and asterisks

/* this is a comment line and may go anywhere */

//This is also a comment line bu only a single line

Procedures:

1. In the previous examples and assignments, we only did output to LED. Here we do input from the buttons.

2. Create a project called **KnightRider-Buttons**.

3. Type in the source code below:

```
#include <system.h>  //pic definition files required by boostc

#pragma CLOCK_FREQ 32000000 //Set clock frequency


//Configuration bits

#pragma DATA _CONFIG1H, _OSC_INTIO67_1H & _FCMEN_OFF_1H & _IESO_OFF_1H
#pragma DATA _CONFIG2L, _PWRT_OFF_2L & _BOREN_OFF_2L & _BORV_3_2L
#pragma DATA _CONFIG2H, _WDT_OFF_2H & _WDTPS_1024_2H
#pragma DATA _CONFIG3H, _MCLRE_ON_3H & _LPT1OSC_ON_3H & _PBADEN_OFF_3H &
_CCP2MX_PORTC_3H
#pragma DATA _CONFIG4L, _STVREN_ON_4L & _LVP_ON_4L & _XINST_OFF_4L &
_DEBUG_OFF_4L
#pragma DATA _CONFIG5L, _CP0_OFF_5L & _CP1_OFF_5L & _CP2_OFF_5L & _CP3_OFF_5L
#pragma DATA _CONFIG5H, _CPB_OFF_5H & _CPD_OFF_5H
#pragma DATA _CONFIG6L, _WRT0_OFF_6L & _WRT1_OFF_6L & _WRT2_OFF_6L &
_WRT3_OFF_6L
#pragma DATA _CONFIG6H, _WRTB_OFF_6H & _WRTC_OFF_6H & _WRTD_OFF_6H
#pragma DATA _CONFIG7L, _EBTR0_OFF_7L & _EBTR1_OFF_7L & _EBTR2_OFF_7L &
_EBTR3_OFF_7L
#pragma DATA _CONFIG7H, _EBTRB_OFF_7H

void main()
{
   trisd = 0; //set all portd to output
   portd = 0; //set all portd to 0
   trisb = 1; //set all portb to input
   portb = 0; //set all portb to 0
   char Array1[ ]={
                                          0b00000001,
                                    0b00000010,
                                    0b00000100,
                                    0b00001000,
                                    0b00000100,
                                    0b00000010,
                                    0b00000001,
```

```
                                        0b00000000
                                        };
    char Array2[ ]={
                                        0b10000000,
                                        0b01000000,
                                        0b00100000,
                                        0b00010000,
                                        0b00100000,
                                        0b01000000,
                                        0b10000000,
                                        0b00000000
                                        };

    while (1)
    {

            if (portb.0 == 0)
            {
                    for (int i=0;i<sizeof(Array1);i++)
                    {
                    portd=Array1[i];
                    delay_ms(5);
                    }
            }

            if (portb.3 == 0)
            {
                    for (int i=0;i<sizeof(Array2);i++)
                    {
                    portd=Array2[i];
                    delay_ms(5);
                    }
            }
    }
}
```

4. Compile and program the MTSAC board.

5. Change the main program to the code below, this is the program demonstrates how shift registers work. Save the code and name it **ShiftRegister.c**

```
#include <system.h>  //pic definition files required by boostc

#pragma CLOCK_FREQ 32000000 //Set clock frequency


//Configuration bits

#pragma DATA _CONFIG1H, _OSC_INTIO67_1H & _FCMEN_OFF_1H & _IESO_OFF_1H
#pragma DATA _CONFIG2L, _PWRT_OFF_2L & _BOREN_OFF_2L & _BORV_3_2L
#pragma DATA _CONFIG2H, _WDT_OFF_2H & _WDTPS_1024_2H
#pragma DATA _CONFIG3H, _MCLRE_ON_3H & _LPT1OSC_ON_3H & _PBADEN_OFF_3H &
_CCP2MX_PORTC_3H
#pragma DATA _CONFIG4L, _STVREN_ON_4L & _LVP_ON_4L & _XINST_OFF_4L &
```

```
_DEBUG_OFF_4L
#pragma DATA _CONFIG5L, _CP0_OFF_5L & _CP1_OFF_5L & _CP2_OFF_5L & _CP3_OFF_5L
#pragma DATA _CONFIG5H, _CPB_OFF_5H & _CPD_OFF_5H
#pragma DATA _CONFIG6L, _WRT0_OFF_6L & _WRT1_OFF_6L & _WRT2_OFF_6L &
_WRT3_OFF_6L
#pragma DATA _CONFIG6H, _WRTB_OFF_6H & _WRTC_OFF_6H & _WRTD_OFF_6H
#pragma DATA _CONFIG7L, _EBTR0_OFF_7L & _EBTR1_OFF_7L & _EBTR2_OFF_7L &
_EBTR3_OFF_7L
#pragma DATA _CONFIG7H, _EBTRB_OFF_7H

void main()
{
   trisd = 0; //set all portd to output
   portd = 0b00000001; //set all portd to 0 except rd4
   trisb = 0xff; //set all portb to input
//    portb = 0; //set all portb to 0
//    adcon0.ADON = 0;

   while (1)
   {

                if (portb.0 == 0)
                {
                        if (portd == 0b00001000)
                        {
                                portd = 0b00000001;
                                delay_ms(5);
                        }
                        else
                        {
                                portd = portd << 1;
                                delay_ms(5);
                        }
                }

                if (portb.1 == 0)
                {
                        if (portd == 0b00000001)
                        {
                                portd = 0b00001000;
                                delay_ms(5);
                        }
                        else
                        {
                                portd = portd >> 1;
                                delay_ms(5);
                        }
                }
   }
}
```

6. Compile the source code and upload to the Mt.SAC DEV board. Press button 1 will shift the register up and press button 2 will shift the register dow

7. Create a program call ANDOR.c contains the code below, this code will demonstrate how C programming function || and && works

```
void main()
{
   trisd = 0; //set all portd to output
   portd = 0b00000001; //set all portd to 0 except rd4
   trisb = 0xff; //set all portb to input
//    portb = 0; //set all portb to 0
//    adcon0.ADON = 0;

   while (1)
   {

                if ((portb.0 == 0)&&(portb.1 == 0))  // here contains and  function
                {
                        if (portd == 0b00001000)
                        {
                                portd = 0b00000001;
                                delay_ms(5);
                        }
                        else
                        {
                                portd = portd << 1;
                                delay_ms(5);
                        }
                }

                if ((portb.6 == 0) || (portb.7 == 0)) // here contains or function
                {
                        if (portd == 0b00000001)
                        {
                                portd = 0b00001000;
                                delay_ms(5);
                        }
                        else
                        {
                                portd = portd >> 1;
                                delay_ms(5);
                        }
                }

   }
}
```

7. compile the source code and upload to the MTSAC DEV board, press button 1 & 2 will shift the LED up and press button 3 or 4 will shift the LED down.

In-Class Assignment

1. Create a program which store LED patterns in array, Using button 1 or 2 for shift up, 3 or 4 for shift down. Hold any two buttons for set the pattern back to power on stat.

Take-home Assignment

1. The pentagon ask you to do work for their nuclear weapon project. They want you to design a missile launching system require the president of the United States at the White House and General of Armies at pentagon to push the button at same time to launch the missile. Write the visual effect for led output as well, this project is due next week!

LAB 3. Knight Rider with Interrupt/ Subroutine and Function

Having fun for programming Knight Rider programs? In this lab, we are going to make Knight Rider program better. We are going to show you how to do subroutine and functions in Boost C. We will also show you how to implement digital I/O with interrupts

Materials:

MT. SAC DEV BBOARD

References:

PIC18F2525/2620/4525/4620 data sheet

BoostC C Compiler for PICmicro Reference Manual

MT.SAC DEV Board schematic diagram

Goals:

This chapter, you will learn:

1. PIC built-in function: Introduction to Interrupt, Understand the type of interrupts
2. Electronic Components: No topic for this Lab
3. C Programming Language: Understand Functions, Subroutines Subroutine and C Preprocessor

Study Notes:

What is subroutine?

Subroutine, or subprogram is a portion of code run under a larger program. It is also called function, routine, method, or procedure. Subroutine in C is referred to as "function".
(subroutine from www.wikipedia.org)

For example, If you want to write a function do an addition

```
int addition (int a, int b)
{
   return a+b;
}
```

Or, a function to turn on LED on Portd

```
void ledon (void)
{
   portd = 0xff;
}
```

You can have no input(just called the function) one input, many inputs. For output, you can have one output or no output.

Procedure:

Part I Subroutine and Function

1. Type in the code below, and call it subroutine.c , compile and upload to Mt.SAC DEV. Board

```
void Green_LED(void);
void Red_LED(char counter);
char Orange_LED(void);

void main()
{
   trisd = 0; //set all portd to output
   portd = 0; //set all portd to 0

   while( 1 )
   {
          Green_LED();
          Red_LED(0x05);
          portd = Orange_LED();
          delay_ms(10);
   }
}

void Green_LED(void)
{
   portd = 0b00000000;
   delay_ms(10);
   portd = 0b00000001;
   delay_ms(10);
   portd = 0b00000011;
   delay_ms(10);
   portd = 0b00000111;
   delay_ms(10);
   portd = 0b00001111;
   delay_ms(10);
}

void Red_LED(char counter)
{
   int i=0;
   portd = 0b00010000;
   delay_ms(10);

   while (counter>i)
   {
          portd = portd<<1;
          delay_ms(10);
          i++;
   }
```

```
}

char Orange_LED(void)
{
   return 0b10101010;
}
```

In-class Assignment

1. Write a function in C, take chars 'R','G',and 'O' and put return a value back to portd. If input is 'O' lit all LEDs in orange. If input is 'R', lit all LEDs in Red. If input is 'G' lit all LEDs in Green.

Part II. Introduction to PIC interrupt

Study Note:

PIC Built-in function

What is interrupt?

Interrupt is a very useful function for microcontroller or microprocessor. When microcontrollers and microprocessors are running, interrupt functions in microcontrollers and microprocessors avoid wasting the processor's valuable time in polling loops, waiting for external events. When interrupt occur, microcontrollers or microprocessors will save the state of main program and begin to execute the interrupt handler.

Interrupt in PIC18F4620

In PIC18F4620, there are 10 rigisters which are used to control interrpt operations, these registers are:

RCON
INTCON
INTCON2
INTCON3
PIR1, PIR2
PIE1, PIE2
IPR1, IPR2

Interrupt sources have 3 bits to control their operation, They are:

Flag bit to indicate that an interrupt event occurred

Enable bit that allows program execution to branch to the interrupt vector address when the flag bit is set

Priority bit to select high priority or low priority

Procedure:

Here is the example for MTSAC DEV. Board, type in the code below, name it **interrupt-button.c**:

```
void main()
{
   trisd=0x00; //configure portc as output
   portd=0x00;//initialize portc
   trisb=0xff;//configure porta as input, use porta.5 as IOC

//interrupt
   intcon.GIE=1;  //enable all interrupts
   intcon.PEIE=1; //enable all peripheral interrupt sources
   intcon.TMR0IE=0;
   //intcon.INT0E=1;
   intcon.RBIE=1; //enable portB Interrupt-on-Change

  // set_bit(intcon,7);//enable all interrupts
  // set_bit(intcon,6);//enable all peripheral interrupts
  // clear_bit(intcon,5);//disable the timer0 interrupt
  // clear_bit(intcon,4);// disable the RA2/INT exteranl interrupt
  //  set_bit(intcon,3);//enable the interrupt(RABIE)

   while(1)
   {
      delay_ms(200);
      portb=0x00;

            }
}


void interrupt()
{
   portd=0x01; // when interrupt happens, light up a led
   portb=0xff;//write to porta to clear the interrupt
   //clear_bit(intcon,0);// clear the interrupt flag(RABIF)
   intcon.RBIF=0;
   //intcon.INT0IF=0;
}
```

In order to use interrupt in PIC18F4620, you have to enable the interrupt by adding the lines below in the main() function

when IPEN bit is cleared, the interrupt priority feature is disabled and interrupts are compatible with PIC mid-range device.

Why we disable the interrupt priority? it is an advance topic and quite complicate, so we left it for later discussion.

At default, rcon.IPEN is cleared. However, if you like to make sure the interrupt priority feature is disabled. You can add the line below in the main() function:

```
rcon.IPEN = 0;
```

Here is the GIE bit, which enable/disable all interrupt sources.

```
intcon.GIE=1;  //enable all interrupts
```

After enable all interrupt sources, you should add another line

```
intcon.PEIE=1;  //enable all peripheral interrupt sources
```

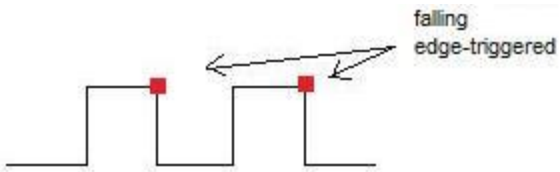
to enable all peripheral interupt sources

The bits of GIE or PEIE determine the interrupts are enable or not. Why? The diagram below shows the PIC18 interrupt logic. When an interrupt is responded to, it has to pass through couple AND gates. The bits of GIE or PEIE have to be set in order for interrupts able to response to the interrupt vector address(00018h). Once in the Interrupt Service Routine, the source(s) of the interrupt can be determining by polling the interrupt flag bit. The interrupt flag bit must be cleared before re-enabling interrupts to avoid recursive interrupts.

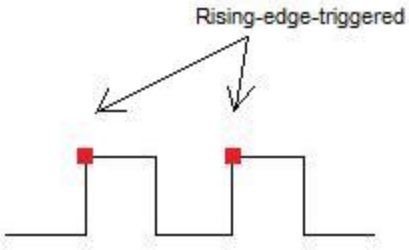
External Pin interrupts

1. External interrupt on the RB0/INT0, RB1/INT1, and RB2/INT2 pin are edge-triggered. What does that mean? Let me show you what is edge-triggered means below: (The red point is the triggered point)

If the corresponding INTEDGx bit in the INTCON2 is set (=0), the interrupt is triggered by a falling edge like below



If the corresponding INTEDGx bit in the INTCON2 is set (=1), the interrupt is triggered by a raising edge like below



2. To enable INT0, type the line below:

```
intcon.INT0 = 1;
```

To enable INT1, type the line below:

```
intcon3.INT1 = 1;
```

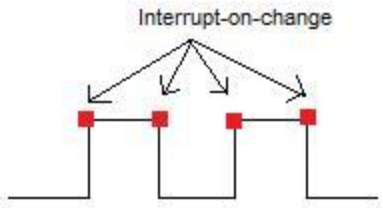
To enable INT2, type the line below:

```
intcon3.INT2 = 1;
```

NOTE: INT0 is on intcon, INT1 and INT2 are on intcon3. CHECK THE DATASHEET FOR

DETAIL

3. Besides External Interrupt, there is also an Interrupt called "Interrupt-on-Change". It enables the flag bit when RB4-RB7 change the level of voltage, the oscilloscope diagram shown below:



4. to enable Interrupt-on-Change, add the line below to the main():

```
intcon.RBIE = 1; // enable interrupt-on-Change RB7:RB4
```

5. After Enable the interrupts, you create a subroutine outside the main call interrupt. When one of the flag bit has been enabled
this subroutine will be executed.

```
void interrupt()
{
//you code here
***flag bit*** = 0;  //don't forget to clear your flag bit softwarely
}
```

If you have multiple interrupt set up, you can have if statement to detect which flag bit has been enabled, like below:

```
void interrupt()
{
   if(intcon.INT0IF)
   {
         portd.1=1; // when interrupt happens, light up a led 1
         portb=0xff;//write to portb to clear the interrupt
         intcon.INT0IF = 0; //clear flag bit softwarely
   }
}
```

2. Here is the example of code, please create a new project name **Interrupt-External.c,** compile the source code and upload to the MT.SAC DEV board

```
void main()
{
   trisd = 0; //set all portd to output
   portd = 0; //set all portd to 0
   trisb = 0xff; //set all portb to input

   //enable interrupts
   intcon.GIE = 1;
   intcon.PEIE = 1;
```

```
    //enable Int0 external interrupt enable bit
    intcon.INT0IE = 1;
    intcon3.INT1IE = 1;  //WATCH OUT!! INT1 and INT2 are on intcon3
    //set INT0 to rising edge-triggered
    intcon2.INTEDG0 = 0; //INT0 is fall edge-triggered
    intcon2.INTEDG1 = 1; //INT1 is raise edge-triggered

    intcon.RBIE = 1; // enable interrupt-on-Change RB7:RB4


    while( 1 )
    {
      //waiting for the interrupt
    }
}

void interrupt()
{
    if(intcon.INT0IF)
    {
            portd.1=1; // when interrupt happens, light up a led 1
            portb=0xff;//write to portb to clear the interrupt
            intcon.INT0IF = 0; //clear flag bit softwarely
    }

    if(intcon3.INT1IF)
    {
            portd.0=1; // when interrupt happens, light up a led 1
            portb=0xff; //write to portb to clear the interrupt
            intcon.INT1IF = 0; //clear flag bit softwarely
    }

    if(intcon.RBIF)
    {
            portd = 0; // turn off portd LEDs
            portb=0xff; //write to portb to clear the interrupt
            intcon.RBIF = 0; //clear flag bit softwarely
    }
}
```

In-class-Assignment:

1. Create a program which store LED patterns in array, Using button with External-Interrupt to shift up LED patterns.

Take-home Assignment

1. Turn on LED4 in green by default. Enable INT0 and INT1 and set them into falling-edged triggered. When press Button1, turn off LED 4 and turn on LED 3, Push button 2, turn on LED 4 and turn off LED 3. Press Button2, nothing happen. When anyone press button3 or 4, reset led to LED 4

Lab 4 - Introduction to Asynchronous Serial

"Did you see the green light on?", "Oh, NO! IT IS RED LIGHT AGAIN!!", "I just saw the green light flash 7 times, is that an 'A'?". These were typical conversations when programmers doing embedded programming in the early days. You do not receive any text message or feedback from microprocessors or microcontrollers. Besides that, assembly language maybe the only choice for the programming language. Writing embedded programs and debugging were not easy tasks. Today, mid-range PIC offer you at least one or two Universal Synchronous Receiver Transmitter (USART). It can be used for debugging and output text messages. With USART, programmers can even create parser, just like a little operating systems for user to input commands! You should feel lucky you are doing embedded programming in C and You have nice output from PICs, let get started!

Materials:

1. MSAC002 Dev Board

2. Dump terminal or Computer with serial port and terminal software is installed(such as Hyperterminal for Windows, or Gtkterm for Linux).

References:

PIC18F2525/2620/4525/4620 data sheet

BoostC C Compiler for PICmicro Reference Manual

MT.SAC DEV Board schematic diagram

Goals:
1. Understanding Asynchronous Serial Communications
1. Analyze FTDI to PIC Schematic
2. Study PIC USART Datasheet
3. Study PIC USART Transmission Code
4. Understand how to call, pass and return values from C Functions

At the end of this Lab, you will know how to send or receive characters to another MTSAC DEV Board, the terminals, PC or anything come with serial port.

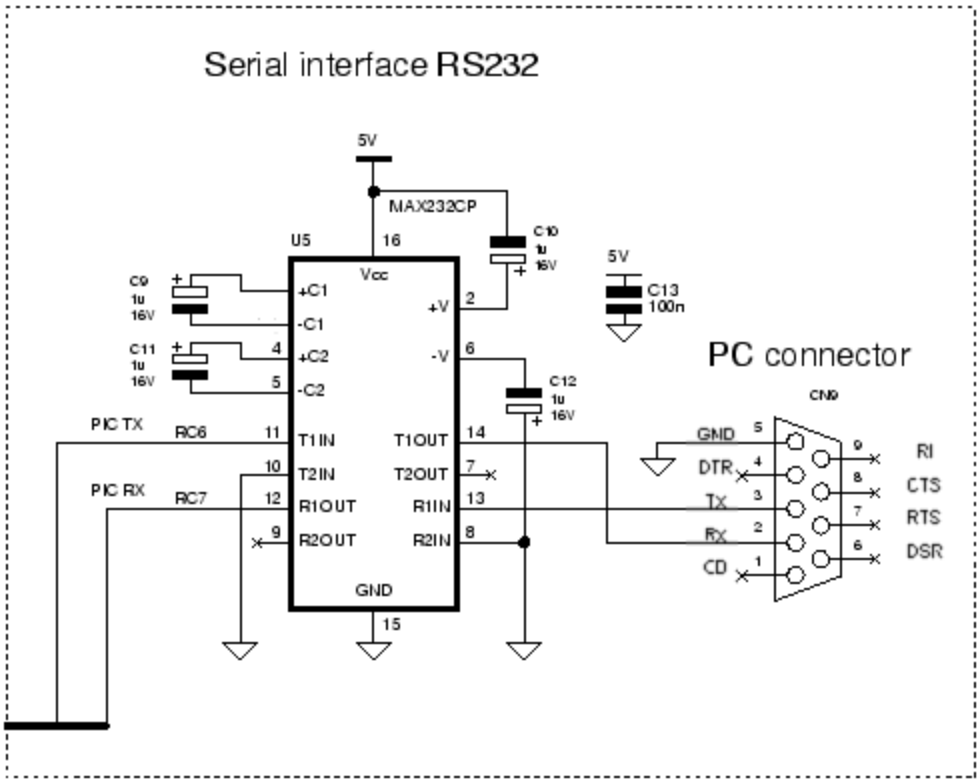
Before start this laboratory.....

In today's computer technology, it is hard not to see serial communication around. Universal Serial Bus a.k.a USB is a big player in the computer market. Bluetooth devices are also use a form of serial communication to talk to each other. The connection between mouse and keyboard are using serial(PS/2 or USB). Many components within computer are also communicated each other in forms of serial. On the MT.SAC DEV Board, there are also many devices talk to PIC in serial (Serial Peripheral Interface). We will have more discussions about Serial Peripheral Interface in Lab 8, 9 and 11. The bottom line, it is very important to learn serial communication. In this laboratory, we will introduce you the the simplest form of serial communications - Asynchronous Serial.

Part I . Asynchronous Serial Communications - Hardware prospect

Study Note:

The diagram below show how we hook up asynchronous serial from PIC to a serial port on PC. The Max232 chip between PIC and PC is the voltage converter in order for PIC and PC to communicate each other.



In Asynchronous serial communication, We only use Transmit Date Line(TXD) and Receive data(RXD) and Ground(GND). The most common use of asynchronous serial communication is to transfer ASCII over RS-232.

If you like to transfer character 'A' in ASCII to another device (Another MCUs, Terminals, or computers). The TX signal will be like the diagram below:

To start transfer, the signal start with a start bit. A start bit signals the beginning of each character frame. Its duration in seconds is the reciprocal of the baud rate. If we're transmitting at 115200 baud, then the duration of the start bit and each subsequent bit will be about 8 us. The entire character frame of eleven bits would be transmitted in about 60 us.

The start bit is always a zero. After start bit, The data bits come right after it. Check the ASCII chart in Appendix. Character 'A' in Hex is 41, the binary representation of 41 is 0100 0001. Since USART send Least Significant Bit(LSB) first. The representation on the oscilloscope is 1000 0010.

After data bit is parity bit, but it is optional. The example from this book, the configuration of parity bit is disable (txsta.TX9 = 0).

Finally, there is a stop bit. it is always represented in a negative voltage(In PIC, it is inverted). If no further characters are transmitted, the line stays in the negative condition.

What is 11520/8-N-1?

11520 is the speed of communication (bits per second). **8-N-1** is a common parameter setting or configuration in asynchronous mode in serial, in which there are eight (**8**) data bits, no (**N**) parity bit, and one (**1**) stop bit.

8-N-1 is the most common configuration for PC serial communications today.

The speed includes bits for framing (stop bits, parity, etc.) and the effective data rate is lower than the bit transmission rate. For 8-N-1 encoding, only 80% of the bits are available for data (8 data bits/(1 start bit + 8 data bits + 1 stop bit) = 80%).

Serial on MT. SAC DEV board



Serial Port



USB Port

However, modern computer do not have traditional serial port anymore but USB ports. That is why MT. SAC DEV Board does not come serial port but mini USB port, and a FTDI Chip translate the serial signal into USB signals.

What is FTDI chip?

FTDI is a company's name. Future Technology Devices International, a semiconductor company from Scottland. Their chip and their software drivers converts RS-232 to USB signals. This chip allow support for RS-232 legacy devices with modern computer.

In-Class Assignment:

1. Read FTDI Datasheet, write a short paragraph describe that what is the function of FT232RL chip.

2. Check schematic diagram of Mt. SAC development board and read FTDI Datasheet and PIC18F4620 Datasheet. Draw a diagram about how to hook up FTDI chips to the PIC and write a short paragraph why MTSAC DEV-Board is designed this way.

Part II. Start Programming Asynchronous Serial Communication

Study Note:

From PIC18F4620 datasheet, We find the section for enabling USART module(p.203)

Here is the information from PIC18F4620 Datasheet:

A. Enable EUSART module

The pins of the Enhanced USART are multiplexed
with PORTC. In order to configure RC6/TX/CK and
RC7/RX/DT as a USART:
• SPEN bit (RCSTA<7>) must be set (= 1)
• TRISC<7> bit must be set (= 1)
• TRISC<6> bit must be set (= 1)
The operation of the Enhanced USART module is
controlled through three registers:

• Transmit Status and Control (TXSTA)
• Receive Status and Control (RCSTA)
• Baud Rate Control (BAUDCON)

Here is the code in BoostC to do the steps above:

```
trisc.6         = 1; //Set TX Pin
trisc.7         = 1; //Set RX Pin

rcsta.SPEN    = 1; //Serial Port Enable bit
```

To set Serial asynchronous, clear the bit below:

```
//Enable Asynchronous Serial
txsta.SYNC    = 0; //Enable the asynchronous serial port by clearing bit, SYNC
```

To setup baud rate:

```
      baudcon.BRG16 = 1; //
txsta.BRGH    = 1;   //
```

We also need to figure out baud rate. The baud rate should set to 16, we will discuss how to calculate the baud rate setting in next part of lab

```
spbrg    = 16; // Baud rate setting
```

To set serial transmit with not interrupt, 8 bits transmission(no parity), and then enable TX transmission:

```
//Enable TX
pie1.TXIE    = 0; //Interrupt is not desire. TXIE bit is cleared
txsta.TX9    = 0; //8-bit transmission is selected TX9 bit is cleared
txsta.TXEN    = 1; //Enable TX transmission
```

To enable RX with interrupt:

```
   //Enable RX
   pie1.RCIE = 0; //Interrupt is not desire. TXIE bit is cleared
   rcsta.CREN = 1; // Contineous Receive Enable bit
```

Procedure:

Okay, it is the time for the real work! To enable EUSART on PIC18F4620, please fellow the steps below:

1. The code below show how to send a character through TX line. type in the code below and create a file called **Sendchar.c**

```
#include <system.h>

#pragma CLOCK_FREQ 32000000
#pragma DATA _CONFIG1H, _OSC_INTIO67_1H & _FCMEN_OFF_1H & _IESO_OFF_1H
#pragma DATA _CONFIG2L, _PWRT_OFF_2L & _BOREN_OFF_2L & _BORV_3_2L
#pragma DATA _CONFIG2H, _WDT_OFF_2H & _WDTPS_1024_2H
#pragma DATA _CONFIG3H, _MCLRE_ON_3H & _LPT1OSC_ON_3H & _PBADEN_OFF_3H &
_CCP2MX_PORTC_3H
#pragma DATA _CONFIG4L, _STVREN_ON_4L & _LVP_ON_4L & _XINST_OFF_4L &
_DEBUG_OFF_4L
#pragma DATA _CONFIG5L, _CP0_OFF_5L & _CP1_OFF_5L & _CP2_OFF_5L & _CP3_OFF_5L
#pragma DATA _CONFIG5H, _CPB_OFF_5H & _CPD_OFF_5H
#pragma DATA _CONFIG6L, _WRT0_OFF_6L & _WRT1_OFF_6L & _WRT2_OFF_6L &
_WRT3_OFF_6L
#pragma DATA _CONFIG6H, _WRTB_OFF_6H & _WRTC_OFF_6H & _WRTD_OFF_6H
#pragma DATA _CONFIG7L, _EBTR0_OFF_7L & _EBTR1_OFF_7L & _EBTR2_OFF_7L &
_EBTR3_OFF_7L
#pragma DATA_CONFIG7H,_EBTRB_OFF_7H

void Rs_Send (char);

void main()
{
   osccon =  0b01110000;    //enable PLL
   osctune = 0b01000000;
   delay_ms(2); //wait for PLL to be stablized

   //To setup Asynchronous Transmission

   trisc.6          = 0; //Set TX Pin as Output

   baudcon.BRG16 = 1; //
   txsta.BRGH    = 1;   //   When BRG16 = 1 and BRGH = 1, set BRG counter clock rate to
FOSC/32
   spbrg          = 16; // Baud rate setting

   txsta.SYNC    = 0; //Enable the asynchronous serial port by clearing bit, SYNC
   rcsta.SPEN    = 1; //Serial Port Enable bit
   pie1.TXIE    = 0; //Interrupt is not desire. TXIE bit is cleared
   txsta.TX9    = 0; //8-bit transmission is selected TX9 bit is cleared
   txsta.TXEN    = 1; //Enable TX transmission
```

```
   while( 1 )
   {
         Rs_Send('A');
   }
}

void Rs_Send (char tx_char)
{
   while(!pir1.TXIF);
   txreg = tx_char;
}
```

2. How about to make PIC receiving signal from Computer? type in the code below, and name it **receivechar.c**, compile it and uploado Mt.SAC DEV board:

```
void main()
{
        osccon =  0b01110000;    //enable PLL
   osctune = 0b01000000;
   delay_ms(2); //wait for PLL to be stablized

   //To setup Asynchronous Transmission

   trisd = 0;
   portd = 0;
   trisc.6         = 0; //Set TX Pin as Output

   baudcon.BRG16 = 1; //
   txsta.BRGH    = 1;   //   When BRG16 = 1 and BRGH = 1, set BRG counter clock rate to
FOSC/32
   spbrg          = 25; // Baud rate setting

   txsta.SYNC    = 0; //Enable the asynchronous serial port by clearing bit, SYNC
   rcsta.SPEN    = 1; //Serial Port Enable bit

   pie1.RCIE = 0; //Interrupt is not desire. TXIE bit is cleared
   rcsta.CREN  = 1; //Continous Receive Enable bit


   while( 1 )
   {
         while(!pir1.RCIF);

         portd = rcreg;
   }
}
```

This code will display the data bits receive from RX and display on LEDs. rcsta.CREN bit has to be enabled. it will store the receiving bits at rcreg

3. Now, let put transmit and Receive together!!Type in the source code below and create a file called **SendReceive.c**:

```
void Tx_Send (char);
void Rx_Recv (void);

void main()
{

   //To setup Asynchronous Transmission

   trisc.6         = 1; //Set TX Pin
   trisc.7         = 1; //Set RX Pin
   trisd = 0 ; //  Set portD to output
   portd = 0 ; // Set LEDs on portd off

   //Set Baud rate
   baudcon.BRG16 = 1; //
   txsta.BRGH    = 1;   //   When BRG16 = 1 and BRGH = 1, set BRG counter clock rate to
FOSC/32
   spbrg         = 25; // Baud rate setting

   //Enable Asynchronous Serial
   txsta.SYNC    = 0; //Enable the asynchronous serial port by clearing bit, SYNC
   rcsta.SPEN    = 1; //Serial Port Enable bit

   //Enable TX
   pie1.TXIE    = 0; //Interrupt is not desire. TXIE bit is cleared
   txsta.TX9    = 0; //8-bit transmission is selected TX9 bit is cleared
   txsta.TXEN    = 1; //Enable TX transmission

   //Enable RX
   pie1.RCIE = 0; //Interrupt is not desire. TXIE bit is cleared
   rcsta.CREN = 1; // Contineous Receive Enable bit

   char ch;

   while( 1 )
   {
         Rx_Recv();
         Tx_Send(rcreg);
   }
}

void Tx_Send (char tx_char)
{
   while(!pir1.TXIF);

   txreg = tx_char;
}

void Rx_Recv ()
{
   while(!pir1.RCIF);
```

```
   portd = rcreg;
}
```

This program will receive a character from terminal and send back directly back to the terminal

In-Class Assignment:

1. Type in the code below, and name it **USART-Scope.c**, compile and upload the hex file to Mt.SAC DEV board. Run the program and use the oscilloscope to probe on Button1 Test

```
 #include <system.h>

#pragma CLOCK_FREQ 32000000
#pragma DATA _CONFIG1H, _OSC_INTIO67_1H & _FCMEN_OFF_1H & _IESO_OFF_1H
#pragma DATA _CONFIG2L, _PWRT_OFF_2L & _BOREN_OFF_2L & _BORV_3_2L
#pragma DATA _CONFIG2H, _WDT_OFF_2H & _WDTPS_1024_2H
#pragma DATA _CONFIG3H, _MCLRE_ON_3H & _LPT1OSC_ON_3H & _PBADEN_OFF_3H &
_CCP2MX_PORTC_3H
#pragma DATA _CONFIG4L, _STVREN_ON_4L & _LVP_ON_4L & _XINST_OFF_4L &
_DEBUG_OFF_4L
#pragma DATA _CONFIG5L, _CP0_OFF_5L & _CP1_OFF_5L & _CP2_OFF_5L & _CP3_OFF_5L
#pragma DATA _CONFIG5H, _CPB_OFF_5H & _CPD_OFF_5H
#pragma DATA _CONFIG6L, _WRT0_OFF_6L & _WRT1_OFF_6L & _WRT2_OFF_6L &
_WRT3_OFF_6L
#pragma DATA _CONFIG6H, _WRTB_OFF_6H & _WRTC_OFF_6H & _WRTD_OFF_6H
#pragma DATA _CONFIG7L, _EBTR0_OFF_7L & _EBTR1_OFF_7L & _EBTR2_OFF_7L &
_EBTR3_OFF_7L
#pragma DATA_CONFIG7H,_EBTRB_OFF_7H

void Rs_Send (char);

void main()
{
   //To setup Asynchronous Transmission

        osccon =  0b01110000;    //enable PLL
   osctune = 0b01000000;
   delay_ms(2); //wait for PLL to be stabilized

   trisc.6          = 0; //Set TX Pin as Output

   baudcon.BRG16 = 1; //
   txsta.BRGH    = 1;   //   When BRG16 = 1 and BRGH = 1, set BRG counter clock rate to
FOSC/32
   spbrg          = 16; // Baud rate setting

   txsta.SYNC    = 0; //Enable the asynchronous serial port by clearing bit, SYNC
   rcsta.SPEN    = 1; //Serial Port Enable bit
   pie1.TXIE    = 0; //Interrupt is not desire. TXIE bit is cleared
   txsta.TX9    = 0; //8-bit transmission is selected TX9 bit is cleared
   txsta.TXEN    = 1; //Enable TX transmission
```

```
   while( 1 )
   {
         Rs_Send('A');
   }
}

void Rs_Send (char tx_char)
{
   while(!pir1.TXIF);
   txreg = tx_char;
}
```

Take-home Assignment

1. a. Write a program to convert a character sent from terminal from Upper case to Lower case, and lower case to Upper case and return the character back to terminal

Lab 5 - Introduction to Asynchronous Serial (Continued)

This week we continue to study Asynchronous Serial. In this lab, we learn how to configure Baud Rate Generator for PICs with BoostC. Adding interupt for USART. Sending and receiving data with character arrays.

Goals:

1. Understand how to setup speed for Baud Rate Generator
2. Improve PIC USART Receive Code by adding interrupt
3. Parse Incoming Data
4. Understand char arrays

Part I. Setup speed for USART

Study Note:

A Baud Rate Generator

How do you configure baud rate for the PIC? It is a good question. Different PICS with different crystals will require different configurations.
What is Baud Rate? You might heard it when you purchase a modem from a computer store. You may heard about it for when working at telecommunication field. Or you are a ham radio operator....etc. **Baud rate is the unit of symbol rate. Bps mean Baud Per Second. 2400Bps means 2400 bytes per second**.

Here is the Baud rate Formula from page 205 from PIC18F4620 Datasheet.

	Configuration Bits			
SYNC	BRG16	BRGH	BRG/EUSART	Baud Rate

			Mode	Formula
0	0	0	8-bit/Asynchronous	FOSC/[64 (n+1)]
0	0	1	8-bit/Asynchronous	FOSC/[16 (n+1)]
0	1	0	16-bit/Asynchronous	FOSC/[16 (n+1)]
0	1	1	16-bit/Asynchronous	FOSC/[16 (n+1)]
1	0	x	8-bit/Synchronous	FOSC/[4 (n+1)]
1	1	x	16-bit/Synchronous	FOSC/[4 (n+1)]

Legend: x = Don't care, n = value of SPBRGH:SPBRG register pair

How do we use it? On page 206, it shows:

For a device with FOSC of 16 MHz, desired baud rate of 9600, Asynchronous mode, 8-bit BRG:

= FOSC/(64 ([SPBRGH:SPBRG] + 1))

Desired Baud Rate

Solving for SPBRGH:SPBRG:

X = ((FOSC/Desired Baud Rate)/64) – 1

= ((16000000/9600)/64) – 1

= [25.042] = 25

Calculated Baud Rate = 16000000/(64 (25 + 1))

= 9615

Error = (Calculated Baud Rate – Desired Baud Rate)/Desired Baud Rate

= (9615 – 9600)/9600 = 0.16%

In examples of Lab 4:

We set BRG16 and BRGH to:

```
baudcon.BRG16 = 0;

txsta.BRGH    = 1;
```

And it is asynchronous serial

```
txsta.SYNC    = 0
```

We want to set Buad Rate to 115200

115200 = FOSC/[16 ([SPBRGH:SPBRG] +1)]

115200 = 32000000/[16([SPBRGH:SPBRG] +1)]

115200(16([SPBRGH:SPBRG] +1)) = 32000000

16([SPBRGH:SPBRG] +1) = 32000000 / 115200

16[SPBRGH:SPBRG] + 16 = 32000000 / 115200

16[SPBRGH:SPBRG] = (32000000 / 115200) - 16

[SPBRGH:SPBRG] = [32000000 / 115200) -16] / 16
[SPBRGH:SPBRG] = 16.36.....

SPBRG is equal to 16

In-Class Assignment:

1. Select 2 PICs from PIC16 family, such as Pic16F627A, Get the datasheet and look up Buad rate generator section, and find out what is SPBRG should set and other parameter should set to get 9600bp on USART

2. Instructor will assign you exercise for baud rate generator.

Part II. Adding Interrupt to USART

Procedure:

1. We want to write a program which is capable to flash an LED every 3 seconds and receive a character from terminal and send back to terminal. Without interrupts, it is an impossible mission. This program demonstrates how flashing LED and asynchrnous serial create trouble for each other..

```
void main()
{
   //To setup Asynchronous Transmission


   trisc.6         = 1; //Set TX Pin
   trisc.7         = 1; //Set RX Pin

   trisd           = 0; //  Set portD to output
//    portd = 0b00000011 ; // Set LEDs on portd off except portd.0

   osccon =  0b01110000;
   osctune = 0b01000000;

   //enable interrupts
   intcon.GIE = 1;      //enable all interrupts
   intcon.PEIE = 1;   //enable all peripheral interrupt sources


   //Set Baud rate
   baudcon.BRG16 = 0; //
   txsta.BRGH    = 1;   //   When BRG16 = 0 and BRGH = 1, set BRG counter clock rate to
FOSC/32
   spbrg           = 16; // Baud rate setting

   //Enable Asynchronous Serial
   txsta.SYNC    = 0; //Enable the asynchronous serial port by clearing bit, SYNC
   rcsta.SPEN    = 1; //Serial Port Enable bit

   //Enable TX
//    pie1.TXIE    = 1; //Interrupt is needed. TXIE bit is enabled
   txsta.TX9    = 0; //8-bit transmission is selected TX9 bit is cleared
```

```
   txsta.TXEN    = 1; //Enable TX transmission

   //Enable RX
   pie1.RCIE   = 0; //Interrupt is needed. RCIE bit is enabled
   rcsta.CREN = 1; // Contineous Receive Enable bit

   while( 1 )
   {
          portd.1 = ~portd.1;
          delay_s(3);

          if (!pir1.RCIF){
                 txreg=rcreg;
          }
   }
}
```

Is it working good?? NO.....That is why we introduce USART Interrupt here:

2. Here is the code with interrupt which works fine, type the code below:

```
void main()
{
   //To setup Asynchronous Transmission


   trisc.6         = 1; //Set TX Pin
   trisc.7         = 1; //Set RX Pin

   trisd           = 0; //  Set portD to output
//    portd = 0b00000011 ; // Set LEDs on portd off except portd.0

   osccon =  0b01110000;
   osctune = 0b01000000;

   //enable interrupts
   intcon.GIE = 1;      //enable all interrupts
   intcon.PEIE = 1;   //enable all peripheral interrupt sources


   //Set Baud rate
   baudcon.BRG16 = 0; //
   txsta.BRGH    = 1;   //   When BRG16 = 0 and BRGH = 1, set BRG counter clock rate to
FOSC/32
   spbrg          = 16; // Baud rate setting

   //Enable Asynchronous Serial
   txsta.SYNC    = 0; //Enable the asynchronous serial port by clearing bit, SYNC
   rcsta.SPEN    = 1; //Serial Port Enable bit

   //Enable TX
//    pie1.TXIE    = 1; //Interrupt is needed. TXIE bit is enabled
   txsta.TX9    = 0; //8-bit transmission is selected TX9 bit is cleared
   txsta.TXEN    = 1; //Enable TX transmission
```

```
   //Enable RX
   pie1.RCIE   = 1; //Interrupt is needed. RCIE bit is enabled
   rcsta.CREN = 1; // Contineous Receive Enable bit

   while( 1 )
   {
          portd.1 = ~portd.1;
          delay_ms(200);
   }
}

// Interrupt for TX/RX

void interrupt()
{
  portd.0 = ~portd.0;

   if (!pir1.RCIF);
          txreg=rcreg;
}
```

In-Class Assignment:

1. Write a program using Receive Interrupt to receive character and change LED pattern with they key type in. For example '1' for pattern 1 and '2' for pattern 2.

2. Bonus question: Use other methods to write the example program for Lab 5 part 2.

Part III the introduction to pointer

Procedure:

In C or C++ programming, pointer is an important topic and need to pay special attention. Whether you knew it or not, every program you wrote so far has used pointer. Every code we wrote in C will be converted into code involving the address of the target variable. In the laboratory, We will show you that an array name can stand for the address of the start of the array. This laboratory won't teach you all the features of pointer, but it is a good start for you to understand how to use pointer in BoostC in PIC.

Here is the example of swaping char using pointer, type the code below and name it **pointer.c**, compile and run on Mt.SAC DEV board:

```
typedef char * charptr;

void swapchars(charptr a_ptr, charptr b_ptr)
{
   char temp;
   temp = *a_ptr;
   *a_ptr = *b_ptr;
   *b_ptr = temp;
}

main() {
```

```
    trisd = 0;
    portd = 0;

    char green = 0b00001111;
    char red = 0b11110000;

    while(1)
    {
            portd = green;
            swapchars( &green, &red);
            delay_ms(20);
    }
}
```

We see * and &, the symbol have not been used before

&green tell us where is address location of green is.

Because the address is a pointer, and it is the address of a char. We can declare variables to hold such values using the indirection sumbol, *, as in

char * green_addr;

The diagram below shows the function which using swapchar works:

In-Class Assignment

1. Write a program that using pointer to switch between 3 patterns(All in green, all in red, and all in orange)

Part IV Transmits a word or sentences to terminals and the introduction to pointer(continue)

Study Note:

In C programming language, The connection between pointers and arrays is very important.

```
char *charptr;
charptr = array;
```

the codes above passes the array's address to the pointer; it does not copy the entire array.

Procedure:

1. type in the code below to understand the concept clearly. name the file **pointer2.c**

```
void Rs_Send (char);

void main()
{
    //To setup Asynchronous Transmission
```

```
   trisc.6         = 1; //Set TX Pin as Output
   trisc.7         = 1;

   osccon =  0b01110000;
   osctune = 0b01000000;
   delay_ms(2); //wait for PLL to be stablized

   baudcon.BRG16 = 0;
   txsta.BRGH    = 1;
   spbrg         = 16; // Baud rate setting

   txsta.SYNC    = 0; //Enable the asynchronous serial port by clearing bit, SYNC
   rcsta.SPEN    = 1; //Serial Port Enable bit
   pie1.TXIE     = 0; //Interrupt is not desire. TXIE bit is cleared
   txsta.TX9     = 0; //8-bit transmission is selected TX9 bit is cleared
   txsta.TXEN    = 1; //Enable TX transmission

   char array[6] = {'E','L','E','C','7','4'}; //Put a string(6 characters) in 6 elements array one
by one
   char *charptr;
   charptr = array;

   Rs_Send(array[0]);
   Rs_Send(array[1]);
   Rs_Send(array[2]);
   Rs_Send(array[3]);
   Rs_Send(array[4]);
   Rs_Send(array[5]);
   Rs_Send('\n');
   Rs_Send('\r');
   Rs_Send(*charptr);
   Rs_Send(*(charptr+1));
   Rs_Send(*(charptr+2));
   Rs_Send(*(charptr+3));
   Rs_Send(*(charptr+4));
   Rs_Send(*(charptr+5));

   while( 1 )
   {
   }
}

void Rs_Send (char tx_char)
{
   while(!pir1.TXIF);
   txreg = tx_char;
}
```

In C, the connection between pointers and array is very important. In previous example, it shows the relationship between Array and pointer. We can really use this feature to transmit words and sentences without send characters one by one.

```
void Rs_Send (char tx_char)     // Send 1 byte in txreg
{
```

```
   while(!pir1.TXIF);          // wait until tx register is empty
   txreg = tx_char;            // Place the tx_char in the txreg for sending

}

void rs232_printf(char *source)        // Send a string 1 byte at a time to txreg
{
   while (*source != 0)        //Loop through variable until end
          Rs_Send(*source++);
      Rs_Send(0x0d);           //Send A Carriage Return (\r)
      Rs_Send(0x0a);           //Send A Line Feed LF (\n)
}

void Rs_Send (char);
void rs232_printf (char *string);

void main()
{
   //To setup Asynchronous Transmission
   trisb = 0xff;              //set trisb as input
   trisc.6          = 1; //Set TX Pin as Output
   trisc.7          = 1;


   //Enable Interrupt
   intcon.GIE = 1;
   intcon.PEIE = 1;

   intcon.INT0IE = 1;//enable Int0 external interrupt enable bit
   intcon2.INTEDG0 = 0; // INT0 is fall edge-triggered

   osccon =  0b01110000;
   osctune = 0b01000000;
   delay_ms(2); //wait for PLL to be stablized

   baudcon.BRG16 = 0;
   txsta.BRGH    = 1;
   spbrg          = 16; // Baud rate setting

   txsta.SYNC    = 0; //Enable the asynchronous serial port by clearing bit, SYNC
   rcsta.SPEN    = 1; //Serial Port Enable bit
   pie1.TXIE    = 0; //Interrupt is not desire. TXIE bit is cleared
   txsta.TX9    = 0; //8-bit transmission is selected TX9 bit is cleared
   txsta.TXEN    = 1; //Enable TX transmission


   while( 1 )
   {
          //waiting for interrupt
   }
}

void interrupt ()
{
```

```
   rs232_printf("Hello World");
   portb=0xff; //write to portb to clear the interrupt
   intcon.INT0IF = 0; //clear flag big softwarely
}

void Rs_Send (char tx_char)
{
   while(!pir1.TXIF);
   txreg = tx_char;
}

void rs232_printf (char *string)
{
   while (*string != 0)
   {
         Rs_Send(*string++);
   }
   Rs_Send(0x0d);
   Rs_Send(0x0a);
}
```

In-Class Assignment:

1. Write a program using pointer to point to character arrays contains "MTSAC"(array1) and "ELEC74 and print the arrays to terminal.

Take-Home Assignment:

1. For PIC18F4620, if the clock speed is 20Mhz, what is X should be?
Write down the formula and do the calculation, pull out the datasheet if necessary

```
#include <system.h>  //pic definition files required by boostc

#pragma CLOCK_FREQ 32000000 //Set clock frequency

//Configuration bits
#pragma DATA _CONFIG1H, _OSC_INTIO67_1H & _FCMEN_OFF_1H & _IESO_OFF_1H
#pragma DATA _CONFIG2L, _PWRT_OFF_2L & _BOREN_OFF_2L & _BORV_3_2L
#pragma DATA _CONFIG2H, _WDT_OFF_2H & _WDTPS_1024_2H
#pragma DATA _CONFIG3H, _MCLRE_ON_3H & _LPT1OSC_ON_3H & _PBADEN_OFF_3H &
_CCP2MX_PORTC_3H
#pragma DATA _CONFIG4L, _STVREN_ON_4L & _LVP_ON_4L & _XINST_OFF_4L &
_DEBUG_OFF_4L
#pragma DATA _CONFIG5L, _CP0_OFF_5L & _CP1_OFF_5L & _CP2_OFF_5L & _CP3_OFF_5L
#pragma DATA _CONFIG5H, _CPB_OFF_5H & _CPD_OFF_5H
#pragma DATA _CONFIG6L, _WRT0_OFF_6L & _WRT1_OFF_6L & _WRT2_OFF_6L &
_WRT3_OFF_6L
#pragma DATA _CONFIG6H, _WRTB_OFF_6H & _WRTC_OFF_6H & _WRTD_OFF_6H
#pragma DATA _CONFIG7L, _EBTR0_OFF_7L & _EBTR1_OFF_7L & _EBTR2_OFF_7L &
_EBTR3_OFF_7L
//#pragma DATA _CONFIG7H, _EBTRB_ON_7H &
void main()
{
//To setup Asynchronous Transmission
```

```
        osccon =  0b01110000;    //enable PLL
   osctune = 0b01000000;
   delay_ms(2); //wait for PLL to be stabilized

 trisd = 0; //set all portd to output
 portd = 0; //set all portd to 0

        baudcon.BRG16 = 1; //
   txsta.BRGH    = 1;   //   When BRG16 = 1 and BRGH = 1, set BRG counter clock rate to
FOSC/32
   spbrg         = X; // Baud rate setting

   txsta.SYNC    = 0; //Enable the asynchronous serial port by clearing bit, SYNC
   rcsta.SPEN    = 1; //Serial Port Enable bit
   pie1.TXIE    = 0; //Interrupt is not desire. TXIE bit is cleared
   txsta.TX9    = 0; //8-bit transmission is selected TX9 bit is cleared
   txsta.TXEN    = 1; //Enable TX transmission
 while( 1 )
 {
  ..............
  ..............
  ..............
 }
}
```

2. Create another interrupt button, which print "Hello World" is backward order, when you push it.

3. Challenge: Using interrupt buttons, to select the character arrays "Mt. SAC"(array1) and "ELEC 74" using pointer and print it out.

Lab 6 - Introduction to Asynchronous Serial - Parser

In this lab, the full potential of USART is going to be revealed! We are going to introduce parser, a Dos-like environment to control PIC microcontroller! Parser can control more complicate embedded programming devices that buttons and leds can not handle. Parser is a very useful and powerful tool to communicate with complicate embedded devices. Furthermore, parser is very useful to debug embedded system using more commplicate protocollike SPI or I^2 C. Want to learn it? Don't miss this lab!

Materials:

1. Mt.SAC Dev Board

2. Dump terminal or Computer with serial port and terminal software is installed(such as Hyperterminal for Windows, or Gtkterm for Linux).

References:

PIC18F2525/2620/4525/4620 data sheet

BoostC C Compiler for PICmicro Reference Manual

MT.SAC DEV Board schematic diagram

Goals:

1. Make PIC understand the string send from computers or other devices
2. Convert an int/long to a string
3. Convert a string to an int/long
4. Understand signed vs unsigned values
5. Understand Parser

Procedure:

Part I. Receive command, execute it and response result

In previous Lab, we are able to transmit a string instead of one character at time. Here is the code able to reconize the command receive from USART.

```
#include <system.h>
#include <string.h>

#pragma CLOCK_FREQ 32000000
#pragma DATA _CONFIG1H, _OSC_INTIO67_1H & _FCMEN_OFF_1H & _IESO_OFF_1H
#pragma DATA _CONFIG2L, _PWRT_OFF_2L & _BOREN_OFF_2L & _BORV_3_2L
#pragma DATA _CONFIG2H, _WDT_OFF_2H & _WDTPS_1024_2H
#pragma DATA _CONFIG3H, _MCLRE_ON_3H & _LPT1OSC_ON_3H & _PBADEN_OFF_3H &
_CCP2MX_PORTC_3H
#pragma DATA _CONFIG4L, _STVREN_ON_4L & _LVP_ON_4L & _XINST_OFF_4L &
_DEBUG_OFF_4L
#pragma DATA _CONFIG5L, _CP0_OFF_5L & _CP1_OFF_5L & _CP2_OFF_5L & _CP3_OFF_5L
#pragma DATA _CONFIG5H, _CPB_OFF_5H & _CPD_OFF_5H
#pragma DATA _CONFIG6L, _WRT0_OFF_6L & _WRT1_OFF_6L & _WRT2_OFF_6L &
_WRT3_OFF_6L
#pragma DATA _CONFIG6H, _WRTB_OFF_6H & _WRTC_OFF_6H & _WRTD_OFF_6H
#pragma DATA _CONFIG7L, _EBTR0_OFF_7L & _EBTR1_OFF_7L & _EBTR2_OFF_7L &
_EBTR3_OFF_7L
//#pragma DATA_CONFIG7H,_EBTRB_OFF_7H

void Tx_Send (char);
char Rx_Recv (void);
void rs232_printf (char *string);

void main()
{
   //To setup Asynchronous Transmission

   trisc.6         = 1; //Set TX Pin
   trisc.7         = 1; //Set RX Pin
   trisd = 0 ; //  Set portD to output
   portd = 0 ; // Set LEDs on portd off

   osccon =  0b01110000;
   osctune = 0b01000000;
   delay_ms(2); //wait for PLL to be stablized
```

```
   //Set Baud rate
   baudcon.BRG16 = 0; //
   txsta.BRGH    = 1;   //   When BRG16 = 0 and BRGH = 1, set BRG counter clock rate to
FOSC/32
   spbrg         = 16; // Baud rate setting

   //Enable Asynchronous Serial
   txsta.SYNC    = 0; //Enable the asynchronous serial port by clearing bit, SYNC
   rcsta.SPEN    = 1; //Serial Port Enable bit

   //Enable TX
   pie1.TXIE     = 0; //Interrupt is not desire. TXIE bit is cleared
   txsta.TX9     = 0; //8-bit transmission is selected TX9 bit is cleared
   txsta.TXEN    = 1; //Enable TX transmission

   //Enable RX
   rcsta.CREN = 1; // Contineous Receive Enable bit

   bit parser_ready = 0;

   rs232_printf("MTSAC DEV BOARD PARSER Lite V1.0\n");
   rs232_printf("MTSAC DEV>");


   while( 1 )
   {
          char *memory_array;
          char mem_spot;
          char input;
          parser_ready = 1;
          mem_spot=0;

          while (parser_ready)
          {
                 if (pir1 & 1 <<RCIF)
                 {
                        input = Rx_Recv();
                        memory_array[mem_spot] = input;
                        Tx_Send(memory_array[mem_spot]);
                        if (input == 0x0D) //0x0D is EENTER in ASCII code
                        {
                               if (!strncmp ( "LED1", memory_array,4))
                               {
                                      if (portd.0 == 0)
                                      {
                                             rs232_printf("LED 1 is ON now\n");
                                      }
                                      else
                                      {
                                             rs232_printf("LED 1 is OFF now\n");
                                      }
                                      portd.0 = ~portd.0;
```

```
                char i;
                for ( i = 0 ; mem_spot != i ; i++ )
                {
                        memory_array[i]=0;
                }
                mem_spot = 0;
                parser_ready = 0;
                rs232_printf("MTSAC DEV>");
        }
        else if (!strncmp ( "LED2", memory_array,4))
        {
                if (portd.0 == 0)
                {
                        rs232_printf("LED 2 is ON now\n");
                }
                else
                {
                        rs232_printf("LED 2 is OFF now\n");
                }
                portd.1 = ~portd.1;

                char i;
                for ( i = 0 ; mem_spot != i ; i++ )
                {
                        memory_array[i]=0;
                }
                mem_spot = 0;
                parser_ready = 0;
                rs232_printf("MTSAC DEV>");
        }
        else
        {
                if (mem_spot==0)
                {
                        rs232_printf("MTSAC DEV>");
                }
                else
                {
                        char i;
                        for ( i = 0 ; mem_spot != i ; i++ )
                        {
                                Tx_Send(memory_array[i]);
                                memory_array[i]=0;
                        }
                        rs232_printf(": command not found\n");
                        mem_spot = 0;
                        parser_ready = 0;
                        rs232_printf("MTSAC DEV>");
                }
        }
}
else
{
        mem_spot++;
```

```
                        }
                    }
                }
    }
}

void Tx_Send (char tx_char)
{
    while(!pir1.TXIF);
    txreg = tx_char;
}

char Rx_Recv ()
{
    while(!pir1.RCIF);
        return rcreg;
}

void rs232_printf (char *string)
{
    while (*string != 0)
    {
        Tx_Send(*string++);
    }
}
```

In-class Assignment:

1. Use parser to control array of LEDs pattern, command **start** will make LEDs flash, command **stop** will make LEDs stop

Part II Convert from String to integer/long and from integer/long to String - C Programming

So, we can receive strings from computers, terminals and other devices.

1. Create a new project call **Lab5** and import source files from Parser100 to Lab5. Make sure you include all the files from Parser100.
The C source files include: **main.c, rs232.c, filters.c**
The C header files include: **global.h, main.h, filters.h, rs232.h**
Don't

Procedure:

main.c

```
#include <system.h>
#include <string.h>
#include "rs232.h"
```

```
#pragma CLOCK_FREQ 32000000
#pragma DATA _CONFIG1H, _OSC_INTIO67_1H & _FCMEN_OFF_1H & _IESO_OFF_1H
#pragma DATA _CONFIG2L, _PWRT_OFF_2L & _BOREN_OFF_2L & _BORV_3_2L
#pragma DATA _CONFIG2H, _WDT_OFF_2H & _WDTPS_1024_2H
#pragma DATA _CONFIG3H, _MCLRE_ON_3H & _LPT1OSC_ON_3H & _PBADEN_OFF_3H &
_CCP2MX_PORTC_3H
#pragma DATA _CONFIG4L, _STVREN_ON_4L & _LVP_ON_4L & _XINST_OFF_4L &
_DEBUG_OFF_4L
#pragma DATA _CONFIG5L, _CP0_OFF_5L & _CP1_OFF_5L & _CP2_OFF_5L & _CP3_OFF_5L
#pragma DATA _CONFIG5H, _CPB_OFF_5H & _CPD_OFF_5H
#pragma DATA _CONFIG6L, _WRT0_OFF_6L & _WRT1_OFF_6L & _WRT2_OFF_6L &
_WRT3_OFF_6L
#pragma DATA _CONFIG6H, _WRTB_OFF_6H & _WRTC_OFF_6H & _WRTD_OFF_6H
#pragma DATA _CONFIG7L, _EBTR0_OFF_7L & _EBTR1_OFF_7L & _EBTR2_OFF_7L &
_EBTR3_OFF_7L
//#pragma DATA _CONFIG7H, _EBTRB_ON_7H &

void main()
{
        trisb = 0;
        portb = 0;
        trisd = 0;

        rs232_init(25); //2400 @ 32MHZ
        rs232_printf("All right! I finished the assignment for Lab 5! \n");

        while ( 1 )
        ;
}
```

RS-232.c

```
#include "rs232.h"
#include "filters.h"

us8 ParserReady;
us8 rx_buffer[RS232_BUFSIZE];
us8 rx_index;

void rs232_init( us8 brg ) {
   rx_index    = 0;
   ParserReady    = 0;

   trisc.6         = 0;
   trisc.7         = 1;

   // Transmit control register
   txsta.TX9    = 0;
   txsta.TXEN    = 1;
   txsta.SYNC    = 0;
   txsta.BRGH    = 1;

   // Receive control register
   rcsta.SPEN    = 1;
```

```
    rcsta.RX9    = 0;
    rcsta.CREN    = 1;
    rcsta.ADDEN    = 0;

    // Baud rate setting
    spbrg          = brg;

    // EUSART Interupts
    pie1.RCIE    = 1;
    pie1.TXIE    = 0;

    return;
}

//us8 rs232_interrupt(void)
void rs232_interrupt( void ) {
    us8 x;
    //pir1.RCIF = 0;
    if ( rcsta.OERR ) {
            rcsta.CREN = 0;
            rcsta.CREN = 1;
    }
    else if ( rcsta.FERR ) {
            rx_index = 0; // 1
            rx_buffer[0] = rcreg;
            //rx_buffer[0] = rcreg1; // for PIC18F65J90
    }
    else {
            x = rcreg;
            //x = rcreg1; // for PIC18F65J90
            //rs232_send_byte(x); // Testing
            if ( filter_ascii( x ) ) {
                    if ( x == ';' ) { // binary command
                            //return 2;
                            return;
                    }
                    if ( rx_index < RS232_BUFSIZE ) {
                            rx_buffer[rx_index] = x;
                            rx_index++;
                            // make sure RCREG is empty, Peek RCIF
                            if ( pir1.RCIF ) {
                                    x = rcreg;
                                    //x = rcreg1; // for PIC18F65J90
                                    rx_buffer[rx_index] = x;
                                    rx_index++;
                                    if ( pir1.RCIF ) {
                                            // Raise Error
                                    }
                            }
                    }
                    else { // buffer is full
                            rx_index = 0;
                    }
            }
```

```
        else if ( x == 0x0A || x == 0x0D ) { // Carriage Return or Line Feed
                ParserReady = 1;
                //return 1;
                //return;
        }
        else if ( x == 0x08 ) { // DELETE
                // remove last us8 by de-incrementing index
                if ( rx_index > 0 ) {
                        rx_index--;
                }
        }
        else { // do nothing with other chars
                asm nop;
        }
   }
   //return 0;
   //return;
}

//
// Pulls charactors from RX Buffer
//

us8 rs232_getchar()
{
   if (rcsta.OERR){
        rs232_printf("OERR");
        rcsta.CREN = 0;
        rcsta.CREN = 1;
   }
   while (!(pir1 & 1 <<RCIF));
   return rcreg;
}

us8 rs232_getchar( us16 index ) {
   if ( index <= rx_index ) {
        return rx_buffer[index];
   }
   else {
        return 0;
   }
}

void rs232_send_byte( us8 value ) {
   while ( ( txsta & 1 << TRMT ) == 0 );
   txreg = value;
}

void rs232_printf( const char* text ) {
   us8 i = 0;
   while ( text[i] != 0 )
        rs232_send_byte( text[i++] );
}
```

```
void rs232_printf( us8 value ) {
   us8 hex;

   hex = nibble_to_hexchar( ( char )( value >> 4 ) );
   rs232_send_byte( hex );
   hex = nibble_to_hexchar( ( char )( value & 0x0F ) );
   rs232_send_byte( hex );

}

void rs232_printf( us16 value ) {
   rs232_printf( ( char )( value >> 8 ) );
   rs232_printf( ( char )( value ) );
}

void rs232_printf( us32 value ) {
   rs232_printf( ( char )( value >> 24 ) );
   rs232_printf( ( char )( value >> 16 ) );
   rs232_printf( ( char )( value >> 8 ) );
   rs232_printf( ( char )( value ) );
}

void rs232_print_dec( us8 value ) {
   if ( value > 99 ) {
         rs232_send_byte( ( ( value / 100 ) % 10 ) + '0' );
   }

   if ( value > 9 ) {
         rs232_send_byte( ( ( value / 10 ) % 10 ) + '0' );
   }

   rs232_send_byte( ( value % 10 ) + '0' );
}
```

filter.h

```
#include "filters.h"

//
// Filters ASCII Charactors     rs232_printf("All right! I finished the assignment for Lab 5!
\n");

        while ( 1 )
        ;
}
```

RS-232.c

```
#include "rs232.h"
#include "filters.h"

us8 ParserReady;
us8 rx_buffer[RS232_BUFSIZE];
us8 rx_index;
```

```
void rs232_init( us8 brg ) {
   rx_index    = 0;
   ParserReady    = 0;

   trisc.6         = 0;
   trisc.7         = 1;

   // Transmit control register
   txsta.TX9    = 0;
   txsta.TXEN    = 1;
   txsta.SYNC    = 0;
   txsta.BRGH    = 1;

   // Receive control register
   rcsta.SPEN    = 1;
   rcsta.RX9    = 0;
   rcsta.CREN    = 1;
   rcsta.ADDEN    = 0;

   // Baud rate setting
   spbrg          = brg;

   // EUSART Interupts
   pie1.RCIE    = 1;
   pie1.TXIE    = 0;

   return;
}

//us8 rs232_interrupt(void)
void rs232_interrupt( void ) {
   us8 x;
   //pir1.RCIF = 0;
   if ( rcsta.OERR ) {
          rcsta.CREN = 0;
          rcsta.CREN = 1;
   }
   else if ( rcsta.FERR ) {
          rx_index = 0; // 1
          rx_buffer[0] = rcreg;
          //rx_buffer[0] = rcreg1; // for PIC18F65J90
   }
   else {
          x = rcreg;
          //x = rcreg1; // for PIC18F65J90
          //rs232_send_byte(x); // Testing
          if ( filter_ascii( x ) ) {
                 if ( x == ';' ) { // binary command
                        //return 2;
                        return;
                 }
                 if ( rx_index < RS232_BUFSIZE ) {
                        rx_buffer[rx_index] = x;
```

```
                        rx_index++;
                        // make sure RCREG is empty, Peek RCIF
                        if ( pir1.RCIF ) {
                                x = rcreg;
                                //x = rcreg1; // for PIC18F65J90
                                rx_buffer[rx_index] = x;
                                rx_index++;
                                if ( pir1.RCIF ) {
                                        // Raise Error
                                }
                        }
                }
                else { // buffer is full
                        rx_index = 0;
                }
        }
        else if ( x == 0x0A || x == 0x0D ) { // Carriage Return or Line Feed
                ParserReady = 1;
                //return 1;
                //return;
        }
        else if ( x == 0x08 ) { // DELETE
                // remove last us8 by de-incrementing index
                if ( rx_index > 0 ) {
                        rx_index--;
                }
        }
        else { // do nothing with other chars
                asm nop;
        }
   }
   //return 0;
   //return;
}

//
// Pulls charactors from RX Buffer
//

us8 rs232_getchar()
{
   if (rcsta.OERR){
        rs232_printf("OERR");
        rcsta.CREN = 0;
        rcsta.CREN = 1;
   }
   while (!(pir1 & 1 <<RCIF));
   return rcreg;
}

us8 rs232_getchar( us16 index ) {
   if ( index <= rx_index ) {
        return rx_buffer[index];
   }
```

```
    else {
        return 0;
    }
}

void rs232_send_byte( us8 value ) {
    while ( ( txsta & 1 << TRMT ) == 0 );
    txreg = value;
}

void rs232_printf( const char* text ) {
    us8 i = 0;
    while ( text[i] != 0 )
        rs232_send_byte( text[i++] );
}

void rs232_printf( us8 value ) {
    us8 hex;

    hex = nibble_to_hexchar( ( char )( value >> 4 ) );
    rs232_send_byte( hex );
    hex = nibble_to_hexchar( ( char )( value & 0x0F ) );
    rs232_send_byte( hex );

}

void rs232_printf( us16 value ) {
    rs232_printf( ( char )( value >> 8 ) );
    rs232_printf( ( char )( value ) );
}

void rs232_printf( us32 value ) {
    rs232_printf( ( char )( value >> 24 ) );
    rs232_printf( ( char )( value >> 16 ) );
    rs232_printf( ( char )( value >> 8 ) );
    rs232_printf( ( char )( value ) );
}

void rs232_print_dec( us8 value ) {
    if ( value > 99 ) {
        rs232_send_byte( ( ( value / 100 ) % 10 ) + '0' );
    }

    if ( value > 9 ) {
        rs232_send_byte( ( ( value / 10 ) % 10 ) + '0' );
    }

    rs232_send_byte( ( value % 10 ) + '0' );
}
```

filter.h

```
#include "filters.h"
```

```
//
// Filters ASCII Charactors
//
us8 filter_ascii( us8 x ) {
    if ( 0x20 <= x && x <= 0x7F ) {
            return 1;
    }
    return 0; // Error, Bad Value
}

//
// Filters HEX Charactors
//
us8 filter_hexchar( us8 x ) {
    if ( ( '0' <= x && x <= '9' ) || ( 'A' <= x && x <= 'F' ) ) { // hex filter
            return 1;
    }
    return 0; // Error, Bad Value
}

//
// Make HEX Charactor Uppercase
//
us8 uppercase_hexchar( us8 x ) {
    if ( 'a' <= x && x <= 'z' ) {    // make upper case
            x -= 0x20;
    }
    return x;
}

//
// Converts Hex Charactors to Hex value
//
// Notes:
// Expects Upper Case ASCII Charactors
//
us8 hexchar_to_hex( us8 msb, us8 lsb ) {
    us8 x;

    if ( '0' <= msb && msb <= '9' ) {
            msb -= 0x30;

    }
    else if ( 'A' <= msb && msb <= 'F' ) {
            msb -= 0x37;
    }

    if ( '0' <= lsb && lsb <= '9' ) {
            lsb -= 0x30;
    }
    else if ( 'A' <= lsb && lsb <= 'F' ) {
            lsb -= 0x37;
    }
```

```
    x = ( msb << 4 );
    x |= lsb;

    return x;
}

//
// Converts Dec Charactors to Hex value
//
us8 decchar_to_hex( us8 p[], us8 start, us8 stop ) {
    us8 x;
    x = 0;
    return x;
}

//
// Converts Nibble to Hex Charactor
//
us8 nibble_to_hexchar( us8 value ) {
    value &= 0x0F;

    if ( value < 16 ) {

            if ( value < 10 )
                    value += '0';
            else
                    value += ( 'A' - 10 );

            return value;
    }
    return 0;
}

// Converts Nibble to Decimal Charactor
us8 nibble_to_decchar( us8 value ) {
    value &= 0x0F;
```

Goals:

1. Convert int/long to a string
2. Convert a string to an int/long
3. Understand signed vs unsigned values

Procedure:

1. Create a new project call **Lab5** and import source files from Parser100 to Lab5. Make sure you include all the files from Parser100.
The C source files include: **main.c, rs232.c, filters.c**
The C header files include: **global.h, main.h, filters.h, rs232.h**
Don't

```
    if ( value < 10 ) {
            return value + '0';
    }
    return 0;
```

```
}

// Meters/second -> Feet/minute
s32 meters_to_feet( s32 meters ) {
   //if ( -1000 < meters < 1000 ) {
   s32 temp;
   temp = meters * 32808L;
   temp /= 10000L;
   return temp;
   //}
   //return 0;
}

// BCD Functions?
// Binary Functions?
```

global.h

```
#ifndef _GLOBAL_H_
#define _GLOBAL_H_
#warning "GLOBAL.H"

#ifdef __18CXX
   #warning "Compiling for C18"
#elif __30CXX
   #warning "Compiling for C30"
#elif _BOOSTC
   #warning "Compiling for Boost C"
   #include <system.h>
#else
   #error "Compiling for unknown compiler"
#endif

#define NULL                    0

typedef signed char     s8;
typedef signed int      s16;
typedef signed long     s32;

typedef unsigned char           us8;
typedef unsigned int    us16;
typedef unsigned long           us32;

typedef unsigned char           bcd8;

typedef union {
   us8 value;
} uus8;

typedef struct {
   uus8 lo;
   uus8 hi;
} sus16;
```

```
typedef union {
   us16 value;
   sus16 parts;
} uus16;

typedef struct {
   uus16 lo;
   uus16 hi;
} sus32;

typedef union {
   us32 value;
   sus32 parts;
} uus32;

//uus32.value = 0x12345678
//uus32.parts.hi.value = 0x1234
//uus32.parts.lo.value = 0x5678

//uus32.parts.hi.parts.hi = 0x12
//uus32.parts.hi.parts.lo = 0x34
//uus32.parts.lo.parts.hi = 0x56
//uus32.parts.lo.parts.lo = 0x78
#include "filters.h"
#include "rs232.h"
#endif
```

Take-home Assignments:

1. Use parser to control array of LEDs pattern, command **start** will make LEDs flash, command **stop** will make LEDs stop, **enable** to show first LED pattern on LED. **forward** to show next pattern, and **previous** to show last pattern.

2. Bonus: Use parser to write a little adventure game, You can printer out response through terminal, or using buttons and LEDs on Mt.SAC DEV board.

Lab 7 Interfacing to the analog world -Analog to Digital Converter , Potentiometer, Temperture Sensor, and Light Sensor

In the previous labs, all the PICs features we learned are digital. In today's world, analog electronic is still very important. PIC18F4620 has 13 Channel 10 bit adc converter. Many PICs even have special module to process analog signals such as audio & speech. In this Lab, we will learn how to set the various registers that controls the analog to digital conversion for PIC18F4620.

Materials:

1. MSAC Dev Board

References:

PIC18F2525/2620/4525/4620 data sheet

BoostC C Compiler for PICmicro Reference Manual

MT.SAC DEV Board schematic diagram

NTC Thermistors Datasheet

Photo Diode Datasheet

Goals:

1. Understand how Analog to Digital Converter module work

2. Understand how temperature sensor(thermistor) work

2. Understand how photo-diode

Part I. ADC Basic

Study Note:

ADC module has five registers associate with it.

adresh - Result High Register
adresl - Result Low Register
adres - A/D result register
adcon0 - A/D control register0
adcon1 - A/D control register1

REGISTER 19-1: ADCON0:A/D CONTROL REGISTER 0

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
-	-	CHS3	CHS2	CHS1	CHS0	GO/DONE	ADON
bit 7							bit 0

Legend:

R = Readable bit W=Writable bit U=Unimplemented bit, read as '0'

-n=Value at POR '1'=Bit is set '0'=Bit is cleared x=Bit is unknown

bit 7-6 **Unimplemented:** Read as '0'
bit 5-2 CHS3:CHS0: Analog Channel Select bits

0000 = Channel0(AN0)
0001 = Channel1(AN1)
0010 = Channel2(AN2)
0011 = Channel3(AN3)
0100 = Channel4(AN4)

0101 = Channel5(AN5)
0110 = Channel6(AN6)
0111 = Channel7(AN7)
1000 = Channel8(AN8)
1001 = Channel9(AN9)
1010 = Channel10(AN10)
1011 = Channel11(AN11)
1100 = Channel12(AN12)
1101 = Unimplemented
1110 = Unimplemented
1111 = Unimplemented

bit1 **GO/DONE**: A/D Conversion Status bit
When ADON =1;
1 = A/D conversion in progress
0 = A/D Idle

bit0 ADON:A/D On bit
1=A/D Converter module is enabled
2=A/D Converter module is disabled

REGISTER 19-2: ADCON1: A

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-q	R/W-q	R/W-q
-	-	VCFG1	VCFG0	PCFG3	PCFG2	PCFG1	PCFG0
bit 7							bit 0

Legend:

R = Readable bit W=Writable bit U=Unimplemented bit, read as '0'

-n=Value at POR '1'=Bit is set '0'=Bit is cleared x=Bit is unknown

bit 7-6 Unimplemented: Read as '0'
bit 5 VCFG1: Voltage Referemce Cpmfogiratopm bit (VREF-source)
1 = VREF -(AN2)
0 = VSS
bit 4 VCFG0: Voltage Referemce Cpmfogiration bit(VREF+source)
1 = VREF+(AN3)
0 = VDD
bit3-0 PCFG3:PCFG0: A/D Port Configuration Control bits:

PCFG 3: PCFG 0	AN12	AN11	AN10	AN9	AN8	AN7	AN6	AN5	AN4	AN3	AN2	AN1	AN0
0000	A	A	A	A	A	A	A	A	A	A	A	A	A
0001	A	A	A	A	A	A	A	A	A	A	A	A	A
0010	A	A	A	A	A	A	A	A	A	A	A	A	A
0011	D	A	A	A	A	A	A	A	A	A	A	A	A
0100	D	D	A	A	A	A	A	A	A	A	A	A	A

0101	D	D	D	A	A	A	A	A	A	A	A	A	A
0110	D	D	D	D	A	A	A	A	A	A	A	A	A
0111	D	D	D	D	D	A	A	A	A	A	A	A	A
1000	D	D	D	D	D	D	A	A	A	A	A	A	A
1001	D	D	D	D	D	D	D	A	A	A	A	A	A
1010	D	D	D	D	D	D	D	D	A	A	A	A	A
1011	D	D	D	D	D	D	D	D	D	A	A	A	A
1100	D	D	D	D	D	D	D	D	D	D	A	A	A
1101	D	D	D	D	D	D	D	D	D	D	D	A	A
1110	D	D	D	D	D	D	D	D	D	D	D	D	A
1111	D	D	D	D	D	D	D	D	D	D	D	D	D

Procedure:

1. Analog-Digital Converter is very easy to understand. Type in the code below and see how easy is it:

```
void adc_init(void);

void main()
{
   trisd = 0; //set all portd to output LEDS
   portd = 0;

   trisa = 0b000000001; //set port RA0 as Input, rest of pins are output

   usart_init();
   adc_init();
   //adcon1 = 0b00111110; //Bit 0 is an analog input for the A/D, the 1's are digital input
   //adcon0 = 0b00000001;
   delay_10us(2); //Wait the required acquisition time (datasheet)
   //adcon0.ADON = 1;

   int i = 0;

   char Array0[ ]={   0b00000000,
                                    0b00000001,
                                    0b00000011,
                                    0b00000111,
                                    0b00001111,
                                    0b00011110,
                                    0b00111100,
                                    0b01111000,
                                    0b11110000,
                                    0b11110001,
                                    0b11110011,
                                    0b11110111,
                                    0b11111111,
                                    0b11101110,
```

```
                                        0b11001100,
                                        0b10001000};

   while (1)
   {
         adcon0.GO = 1;
         while (adcon0.GO);

         short retval;
         retval = (short)adresh;
         retval = retval << 8;
         retval |=adresl;

         portd=Array0[retval%sizeof(Array0)];
   }
}

void adc_init(void){
//Following Datasheet P.227
//1. Configure the A/D module
// . Configure analog pins, voltage reference and digital I/O (ADCON1)
   clear_bit(adcon1, VCFG1);    //Voltage Reference Configuration bit (VREF - source) VSS
   clear_bit(adcon1, VCFG0);    //Voltage Reference Configuration bit(VREF + source) VDD


   clear_bit(adcon1, PCFG0);        //AN0 is analog input and rest are digital
   set_bit(adcon1, PCFG1);     // Vref+ = Vdd
   set_bit(adcon1, PCFG2);     // Vref- = Vss
   set_bit(adcon1, PCFG3);

   // .Select A/D input channel(adcon0)
   adcon0.CHS3 = 0;   //channel 0
   adcon0.CHS2 = 0;
   adcon0.CHS1 = 0;
   adcon0.CHS0 = 0;

   // Select A/D acquisition time(adcon2)
   // Select A/D conversion clock(adcon2)

   set_bit(adcon2, ADFM); // AD result needs to be right justified
   clear_bit(adcon2, ACQT2); //  TAD
   set_bit(adcon2, ACQT1);
   clear_bit(adcon2, ACQT0);

   set_bit(adcon2, ADCS2);  //FOSC/4
   set_bit(adcon2, ADCS1);
   set_bit(adcon2, ADCS0);

   //turn on A/D module
   set_bit(adcon0, ADON);
}
```

In-Class Assignment:

1. What is the maximum number can 10-bit represent?

2. If you want to 9 channels works, what value will you set to Adcon1?

Part II. Temperature Sensing and Light Sensing

There are many type of Temperature sensors:

Take-home Assignment:

1. Design you own LED patterns represent temperature, display the result of temperature sensor on LEDs.

2. Your family ask the electrical engineer at home to design a "Night light" at bathroom.

Lab 8 - Introduction to Serial Peripheral Interface/Digital Analog Converter

PIC microcontrollers are . Have you ever think to talk withWe have digital Analog Converter. The first SPI device we are going to learn is AD5302. It will adjust it own voltage level when a valid SPI signal sent in.

Materials:

MT. SAC DEV BBOARD

References:

PIC18F2525/2620/4525/4620 data sheet

BoostC C Compiler for PICmicro Reference Manual

MT.SAC DEV Board schematic diagram

AD5302/AD5312/AD5322 Datasheet

Goals:

1. Understand how SPI protocol works

2. Understand how Digital-Analog Converter works

3. Understand how Master Synchronous Serial Port module work in SPI work

Part I. Introduction to SPI

Let us review what we have learned for PIC so far. We interfaced PIC with buttons, LEDs, potentiometers,light sensor and temperture sensor. We learned how to connect PIC to computer or communicate with other PICs with USART. However, The technology of PIC we learned so far is the built-in function of PICs and simple digital and analog interfaces to

LEDs, buttons and potentiometers. PICs can also communicate with other special purpose ICs or devices such as MP3 decoder, touchscreens, SD/MMC Cards, Sony Memory sticker and many powerful Ics or devices.
Most of mid-range and high-end PICs have capability to communicate with other ICs and devices. PIC18F4620 has MASTER SYNCHRONOUS
SERIAL PORT (MSSP) MODULE to communicate with other ICs and devices. Under Mster Synchronous serial port, it has two modes: SPI and I2C.

Using SPI and I2C, PIC can communicate with the ICs below:

Device Name	Manufacturer	Model Number	Interfacing protocol
Mp3 Decoder	STMicroelectronic	STA013, STA015, STA016 and etc...	I2C, SPI
	VLSI Technology	VLSI - VS1001,VS1002, VS1003,VS1011, VS1033,VS1053 and etc..	SPI
MMC/SD Card	Sannidisk, Trendsend, Panasonic, Toshiba ...etc	Secure Digital Card Micro Secure Digital Card Mini Secure Digital Card	SPI, SD Native mode
Pressure Sensor	Natio	LM74	SPI

	nal Semiconductor		
	Infineon	KP100	SPI
Real Timer Clock(RTC)	Dallas	DS1305 DS1306	SPI
ADC + Temperature Sensor	Analog Device Inc.	AD7816 AD7817 AD7818	SPI
Camera lenses	Canon	Canon EF lens mount	SPI
EEPROM	Microchip	25LC1024	SPI

SPI VS I2C

I2C advantages:
- Requires only 2 I/O pins (SPI requires 3 + 1 select line for every device).
- Multiple masters possible on the same bus.

SPI advantages:
- Faster (higher bitrates and no overhead for addressing etc.)
- Easier protocol (no multi-master, etc).

At this LAB, we will introduce SPI first.

The SPI bus specifies four logic signals.

- SCLK — Serial
- MOSI/SIMO — Master Output, Slave Input
- MISO/SOMI — Master Input, Slave Output
- SS — Slave Select

Different manufacturers may have different naming conventions:

- SCK, CLK — Serial Clock
- SDI, DI, SI — Serial Data In
- SDO, DO, SO — Serial Data Out
- nCS, CS, nSS, STE — Chip Select, Slave Transmit Enable

In-class Assignment:

1. Read the MT.SAC DEV schematic diagram, PIC18F2525/2620/4525/4620 datasheet, and AD5302/AD5312/AD5322 datasheet. Describe why MT.SAC DEV board connect to AD5302 in this way in briefly.

Part II. Digital Analog Converter - control with bit banging method

This program using bit-banging method controling SPI device will help you to understand SPI protocol more easily.

Procedure:

1. Type in the program below, save it as **soft-SPI-DAC.c.** Compile the sourcecode and run on Mt.SAC DEV board.

```
volatile bit spi_clk_tris     @ TRISC.3;
volatile bit spi_do_tris      @ TRISC.5;
volatile bit spi_cs_tris      @ TRISE.1;
volatile bit spi_ldac_tris    @ TRISE.2;

volatile bit spi_clk       @ PORTC.3;
volatile bit spi_do        @ PORTC.5;
volatile bit spi_cs        @ PORTE.1;
volatile bit spi_ldac      @ PORTE.2;

void init_4620(void)
{
   spi_clk_tris=0;
   spi_do_tris=0;
   spi_cs_tris=0;
   spi_ldac_tris=0;

   spi_clk=1;
   spi_cs=1;
   spi_do=1;
   spi_ldac=0;
}

void DAC5032(unsigned int out)
{
   unsigned char i;
   unsigned int dump=0;

//    init_4620();
   delay_us(5);
   spi_cs=1;
```

```
spi_clk=1;
delay_us(5);
spi_clk=0;
delay_us(5);
spi_clk=1;
delay_us(5);
spi_clk=0;
delay_us(3);
spi_cs=0;
delay_us(5);

//-----------------------------------


for(i=0;i<16;i++)
{
	dump=(out) & 0x8000;
	if(dump==0x8000)
	{
		spi_do=1;
	}
	else
	{
		spi_do=0;
	}
	spi_clk=1;
	delay_us(5);
	spi_clk=0;
	out=(out<<1);
	delay_us(5);
}
delay_us(5);
//-----------------------------------
spi_cs=1;
spi_clk=1;
delay_us(5);
spi_clk=0;
delay_us(5);
spi_clk=1;
delay_us(5);
spi_clk=0;
delay_us(5);
spi_clk=1;
delay_us(5);
spi_clk=0;
spi_ldac=0;
delay_us(5);
spi_clk=1;
delay_us(5);
spi_clk=0;
delay_us(5);
spi_clk=1;
delay_us(5);
spi_ldac=1;
```

```
}

void main()
{
   init_4620();
   while(1)
   {
         DAC5032(0x0ff0);
         DAC5032(0x0000);
   }
}
```

In-Class Assignment:

1. Using ossoliscope to probe the SPI_do signal, on the interval between chip selected and chip unselected. Draw the diagram and describe what is the signal represented.

Part III. Digital Analog Converter - Control DAC5032 with PIC MSSP module(SPI)

After we successfully control DAC5032 with bit-banging method. we are going to show you how to control DAC5032 with hardware come with PIC18F4620

Procedure:

1. Type in the program below, save it as **DAC.c.** Compile the sourcecode and run on Mt.SAC DEV board.

```
volatile bit spi_clk_tris     @ TRISC.3;
volatile bit spi_do_tris      @ TRISC.5;
volatile bit spi_di_tris      @ TRISC.4;
volatile bit spi_cs_tris      @ TRISE.1;
volatile bit spi_ldac_tris    @ TRISE.2;

volatile bit spi_clk        @ PORTC.3;
volatile bit spi_do         @ PORTC.5;
volatile bit spi_cs         @ PORTE.1;
volatile bit spi_ldac       @ LATE.2;



int spi_init( void ) {
//    rs232_printf("SPI is initializing...\r\n");
   spi_cs_tris     = 0;
   spi_clk_tris    = 0;
   spi_do_tris             = 0;
   spi_di_tris     = 1;
   spi_ldac_tris    = 0;


//#ifdef PIC18_SPIx1
   sspstat                 = 0b00000000;
   sspstat.SMP             = 1;
   sspstat.CKE             = 1;
   sspcon1       = 0b00000000;    // clock = Fosc/4
```

```
//    sspcon1     = 0b00000001;    // clock = Fosc/16
//    sspcon1     = 0b00000010;    // clock = Fosc/64
    //sspcon1             = 0x20;
    sspcon1.CKP           = 0;

    pir1.SSPIF    = 0;    // clear interrupt flag
    pie1.SSPIE            = 0;    // enable interrupt flag

    sspcon1.WCOL    = 0;
    sspcon1.SSPEN    = 1;

    return 0;
}


void spi_put( unsigned char data ) {
    do {
            sspcon1.WCOL = 0;
            sspbuf = data;
    } while( sspcon1.WCOL );
    while( !sspstat.BF ) {};
}


unsigned char spi_get( void ) {
    spi_put( 0x00 );
    return sspbuf;
}


unsigned char spi_swap(unsigned char data) {
    spi_put(data);
    return sspbuf;
}


void DAC5032 (unsigned char channel, unsigned char value)
{
    unsigned char output_1, output_2;

    spi_ldac = 0;
    spi_cs=0;

    //-------------------------------------
    if (channel == 1)
    {
            output_1=0x80;
    }
    else
    {
            output_1=0x00;
    }

    output_2 = 0x00;
```

```
   output_1 = output_1 | ( value >> 4 );
   output_2 = value & 0x0f;
   output_2 = output_2 << 4;
   spi_put(output_1);
   spi_put(output_2);

   //-----------------------------------
   spi_cs=1;

}

void main()
{
   osccon =  0b01110000;
   osctune = 0b01000000;
   delay_ms(2);    // Allow PLL to stablize
   spi_init();
   //spi_do_tris          = 0;

//    unsigned char i = 0;
//    unsigned char j = 0x80;
   while(1)
   {

                        //Square Wave
          DAC5032(0x0ff0);
           delay_us(5);
          DAC5032(0x0000);
           delay_us(5);
   }
}
```

In-Class Assignment

1. Use the code from class, draw the diagram below on oscilloscope:

2. Add the function below to the main program, this is the function that able to draw picture under oscilloscope X-Y mode:

```
void dac_xy (unsigned char x, unsigned char y)
{
   DAC5032(0, x);
   DAC5032(1, y);

   delay_us(50);
//    spi_ldac=0;
   delay_us(50);
//    spi_ldac=1;
   delay_us(50);
}
```

Draw a Rectangle on Oscilloscope in X-Y mode

3. Bonus question: Draw your name on Oscilloscope (Retired Professor Harry Smith at Mt.

SAC said: If you can draw your name on Oscilloscope, you are good at it)

Take-Home Assignment

1. Use the code you have, draw

Lab 9 - Introduction to Serial Peripheral Interface/ LCD Display and Internal EEPROM

When you turn on a new electronic device you just bought from store, most of time it will ask you to configure the system. When you turn it on again, it will not ask you anymore. Because the configuration already been save into the device memory. This lab will teach you how to design a product like that with PIC's internal memory. Besides, this lab will also introduce you how to display result on the LCD display.

Materials:

MT. SAC DEV BBOARD

References:

PIC18F2525/2620/4525/4620 data sheet

BoostC C Compiler for PICmicro Reference Manual

MT.SAC DEV Board schematic diagram

Goals:

1. Review how SPI protocol works

2. Understand how to use Internal EEPROM of PIC

2. Understand the methods to display data on LCD

Part I. Internal Memory

A Microcontroller is a functional computer system-on-a-chip. It usually comes with two types of ram. The program memory, where programmers compiled the source code and store the machine code at. Besides program memory, PICs usually have a small a mount of memory for microcontroller to store data. PIC18F4620 has **4 mega bytes date memory**, and **1 mega byte EEPROM memory**. It is very useful for project require to store and retrive data from memory or EEPROM.

Procedure:

1. In order to use EEPROM, The initial data of EEPROM has to be included in the source code. EEPROM initialization data is set using the pragma directive like the code below.

#pragma DATA_EEPROM,12,34,56,"HELLO",0xFE,0b10011001

2. Here is the code for EEPROM reading.

```
char eeprom_read(char address){

   // Load address
   eeadr = address;
   //Point to EEPROM memory
   eecon1.EEPGD = 0;

   // 18F devices also need the CFGS bit to be cleared

   eecon1.CFGS = 0;

   // Read, data is available in eedata the next cycle
   eecon1.RD = 1;

   //return value
   return eedata;
}
```

3. Here is the code for EEPROM writing

```
void eeprom_write(us8 address, us8 data){

   //18F devices also need the CFGS bit to be cleared

   eecon1.CFGS = 0;

   // Wait for write to complete
   while (eecon1.WR ){};

   // Load address and data
   eeadr = address;
   eedata = data;

   //Point to EEPROM memory
   eecon1.EEPGD = 0;
   // Disable interrupts;
   //clear_bit(intcon, GIE);

   eecon1.WREN = 1;

   // Required write sequence
   eecon2 = 0x55;
   eecon2 = 0xAA;

   eecon1.WR = 1;

   //Disable write
   eecon1.WREN = 0;
   // Enable interrupts
   //set_bit(intcon, GIE);
}
```

4. Use two functions for reading EEPROM and writing EEPROM above, we can write a simple EEPROM reading/writing program

Please include source files **filters.c** and **rs232.c**, header files **filters.h**, **rs232.h** and **global.h**

```
#include <system.h>
#include <string.h>
#include "rs232.h"

#pragma CLOCK_FREQ 32000000
#pragma DATA _CONFIG1H, _OSC_INTIO67_1H & _FCMEN_OFF_1H & _IESO_OFF_1H
#pragma DATA _CONFIG2L, _PWRT_OFF_2L & _BOREN_OFF_2L & _BORV_3_2L
#pragma DATA _CONFIG2H, _WDT_OFF_2H & _WDTPS_1024_2H
#pragma DATA _CONFIG3H, _MCLRE_ON_3H & _LPT1OSC_ON_3H & _PBADEN_OFF_3H &
_CCP2MX_PORTC_3H
#pragma DATA _CONFIG4L, _STVREN_ON_4L & _LVP_ON_4L & _XINST_OFF_4L &
_DEBUG_OFF_4L
#pragma DATA _CONFIG5L, _CP0_OFF_5L & _CP1_OFF_5L & _CP2_OFF_5L & _CP3_OFF_5L
#pragma DATA _CONFIG5H, _CPB_OFF_5H & _CPD_OFF_5H
#pragma DATA _CONFIG6L, _WRT0_OFF_6L & _WRT1_OFF_6L & _WRT2_OFF_6L &
_WRT3_OFF_6L
#pragma DATA _CONFIG6H, _WRTB_OFF_6H & _WRTC_OFF_6H & _WRTD_OFF_6H
#pragma DATA _CONFIG7L, _EBTR0_OFF_7L & _EBTR1_OFF_7L & _EBTR2_OFF_7L &
_EBTR3_OFF_7L
//#pragma DATA _CONFIG7H, _EBTRB_ON_7H &

volatile bit spi_clk_tris     @ TRISC.3;
volatile bit spi_do_tris      @ TRISC.5;
volatile bit spi_di_tris      @ TRISC.4;
volatile bit spi_cs_tris      @ TRISE.1;
volatile bit spi_ldac_tris    @ TRISE.2;
volatile bit lcd_cs_tris    @ TRISE.0;

volatile bit spi_clk        @ PORTC.3;
volatile bit spi_do         @ PORTC.5;
volatile bit spi_cs         @ PORTE.1;
volatile bit spi_ldac       @ LATE.2;
volatile bit lcd_cs                 @ LATE.0;

char eeprom_read(char address){

   // Load address
   eeadr = address;
   //Point to EEPROM memory
   eecon1.EEPGD = 0;

   // 18F devices also need the CFGS bit to be cleared

   eecon1.CFGS = 0;

   // Read, data is available in eedata the next cycle
   eecon1.RD = 1;

   //return value
   return eedata;
}
```

```
void eeprom_write(us8 address, us8 data){

   //18F devices also need the CFGS bit to be cleared

   eecon1.CFGS = 0;

   // Wait for write to complete
   while (eecon1.WR ){};

   // Load address and data
   eeadr = address;
   eedata = data;

   //Point to EEPROM memory
   eecon1.EEPGD = 0;
   // Disable interrupts;
   //clear_bit(intcon, GIE);

   eecon1.WREN = 1;

   // Required write sequence
   eecon2 = 0x55;
   eecon2 = 0xAA;

   eecon1.WR = 1;

   //Disable write
   eecon1.WREN = 0;
   // Enable interrupts
   //set_bit(intcon, GIE);
}

void main()
{
   trisb = 0;
   portb = 0;

   osccon =  0b01110000;
   osctune = 0b01000000;
   delay_ms(2); //wait for PLL to be stablized

   rs232_init( 16); // 115200 @32MHZ /4 clock //
   rs232_printf("MTSAC DEV BOARD PARSER V1.0\nType 'help' for more information\n");
   rs232_printf("MTSAC DEV>");

   bit parser_ready = 0;


   while( 1 )
   {
          us8 input_string[32];
          us8 mem_spot;
          us8 input;
```

```
	bit parser_ready = 1;
	mem_spot=0;



	while (parser_ready)
	{
		if (pir1 & 1 <<RCIF)
		{
			// If it was, go get it....
			input = rs232_getchar();
			input_string[mem_spot] = input;
			rs232_send_byte(input_string[mem_spot]);

			if (input == 0x0D) //0x0D is ENTER in ASCII code
			{

				if (!strncmp ( "help", input_string,4))
				{
					rs232_printf("help		to show all
commands\n");
					rs232_printf("ver		System
version\n");
					rs232_printf("ls		list directory\n");
					us8 i;
					for ( i = 0 ; mem_spot != i ; i++ )
					{
						input_string[i]=0;
					}
					mem_spot = 0;
					parser_ready = 0;
					rs232_printf("MTSAC DEV>");
				}
				else if (!strncmp ( "ver", input_string,3))
				{
					rs232_printf("MT.SAC DEV-BOARD PARSER
V1.0\nProlinear/Pontech Inc.\n");
					rs232_printf("Author: John C. Tsai\n");
					us8 i;
					for ( i = 0 ; mem_spot != i ; i++ )
					{
						input_string[i]=0;
					}
					mem_spot = 0;
					parser_ready = 0;
					rs232_printf("MTSAC DEV>");
				}
				else if (!strncmp ( "porta", input_string,5))
				{
					rs232_printf(porta);
						us8 i;
					for ( i = 0 ; mem_spot != i ; i++ )
					{
						input_string[i]=0;
```

```
        }
        mem_spot = 0;
        parser_ready = 0;
        rs232_printf("\nMTSAC DEV>");
}
else if (!strncmp ( "portb", input_string,5))
{
        rs232_printf(portb);
                us8 i;
        for ( i = 0 ; mem_spot != i ; i++ )
        {
                input_string[i]=0;
        }
        mem_spot = 0;
        parser_ready = 0;
        rs232_printf("\nMTSAC DEV>");
}
        else if (!strncmp ( "portc", input_string,5))
{
        rs232_printf(portc);
                us8 i;
        for ( i = 0 ; mem_spot != i ; i++ )
        {
                input_string[i]=0;
        }
        mem_spot = 0;
        parser_ready = 0;
        rs232_printf("\nMTSAC DEV>");
}
else if (!strncmp ( "portd", input_string,5))
{
        rs232_printf(portd);
                us8 i;
        for ( i = 0 ; mem_spot != i ; i++ )
        {
                input_string[i]=0;
        }
        mem_spot = 0;
        parser_ready = 0;
        rs232_printf("\nMTSAC DEV>");
}
else if (!strncmp ( "porte", input_string,5))
{
        rs232_printf(porte);
                us8 i;
        for ( i = 0 ; mem_spot != i ; i++ )
        {
                input_string[i]=0;
        }
        mem_spot = 0;
        parser_ready = 0;
        rs232_printf("\nMTSAC DEV>");
}
else if (!strncmp ( "write", input_string,5))
```

```
		{
			eeprom_write(0x00, input_string[6]);


			us8 i;
			for ( i = 0 ; mem_spot != i ; i++ )
			{
				input_string[i]=0;
			}
			mem_spot = 0;
			parser_ready = 0;
			rs232_printf("\nMTSAC DEV>");
		}
		else if (!strncmp ( "read", input_string,4))
		{

			rs232_send_byte(eeprom_read(0x00));
			rs232_send_byte('\n');

			us8 i;

			for ( i = 0 ; mem_spot != i ; i++ )
			{
				input_string[i]=0;
			}
			mem_spot = 0;
			parser_ready = 0;
			rs232_printf("\nMTSAC DEV>");
		}
		else
		{
			if (mem_spot==0)
			{
				rs232_printf("MTSAC DEV>");
			}
			else if(mem_spot>32)
			{
				rs232_printf("String is too long!\n");
			}
			else
			{
				us8 i;
				for ( i = 0 ; mem_spot != i ; i++ )
				{
					rs232_send_byte(input_string[i]);
					input_string[i]=0;
				}
				rs232_printf(": command not found\n");
				mem_spot = 0;
				parser_ready = 0;
				rs232_printf("MTSAC DEV>");
			}
		}
	}
```

```
                        else
                        {
                                mem_spot++;
                        }
                }
        }
   }
}
```

5. After you compile the source code, the program should run like this on MT.SAC DEV Board

```
MTSAC DEV BOARD PARSER V1.0

Type 'help' for more information
MTSAC DEV>write 1
MTSAC DEV>read
1

MTSAC DEV>write M
MTSAC DEV>read
M

MTSAC DEV>write B
MTSAC DEV>read
B

MTSAC DEV>
```

In-Class Assignment

1. Write a knight rider program which store and read sequence of LED patterns from Internal memory.

Part II. LCD Display

Liquid Crystal Display(LCD) display is very common on a lot of electronic device today. There are many ways to interfacing LCD display depends on the applications or the manufacturer of LCD you choose. Serial, Parallel, SPI...etc. For detail, you can check out article about Liquid Crystal Display from wikipedia.org. The LCD Display on Mt.SAC DEV board is controlled by CPLD. The program for CPLD to drive LCD display was written by Jacob Christ. In this CLPD program, Jacob defines SPI protocol as standard protocol for programmers to control LCD display.

Procedure:

1. Type in the code below to the programming environment you have and named **LCD-sample.c**

```
#include <system.h>  //pic definition files required by boostc

#pragma CLOCK_FREQ 32000000 //Set clock frequency
```

```
//Configuration bits

#pragma DATA _CONFIG1H, _OSC_INTIO67_1H & _FCMEN_OFF_1H & _IESO_OFF_1H
#pragma DATA _CONFIG2L, _PWRT_OFF_2L & _BOREN_OFF_2L & _BORV_3_2L
#pragma DATA _CONFIG2H, _WDT_OFF_2H & _WDTPS_1024_2H
#pragma DATA _CONFIG3H, _MCLRE_ON_3H & _LPT1OSC_ON_3H & _PBADEN_OFF_3H &
_CCP2MX_PORTC_3H
#pragma DATA _CONFIG4L, _STVREN_ON_4L & _LVP_ON_4L & _XINST_OFF_4L &
_DEBUG_OFF_4L
#pragma DATA _CONFIG5L, _CP0_OFF_5L & _CP1_OFF_5L & _CP2_OFF_5L & _CP3_OFF_5L
#pragma DATA _CONFIG5H, _CPB_OFF_5H & _CPD_OFF_5H
#pragma DATA _CONFIG6L, _WRT0_OFF_6L & _WRT1_OFF_6L & _WRT2_OFF_6L &
_WRT3_OFF_6L
#pragma DATA _CONFIG6H, _WRTB_OFF_6H & _WRTC_OFF_6H & _WRTD_OFF_6H
#pragma DATA _CONFIG7L, _EBTR0_OFF_7L & _EBTR1_OFF_7L & _EBTR2_OFF_7L &
_EBTR3_OFF_7L
//#pragma DATA _CONFIG7H, _EBTRB_ON_7H &

volatile bit spi_clk_tris     @ TRISC.3;
volatile bit spi_do_tris      @ TRISC.5;
volatile bit spi_di_tris      @ TRISC.4;
volatile bit spi_cs_tris      @ TRISE.1;
volatile bit spi_ldac_tris    @ TRISE.2;

volatile bit lcd_cs_tris    @ TRISE.0;

volatile bit spi_clk        @ PORTC.3;
volatile bit spi_do         @ PORTC.5;
volatile bit spi_cs         @ PORTE.1;
volatile bit spi_ldac       @ LATE.2;

volatile bit lcd_cs                 @ LATE.0;

int spi_init( void ) {
//    rs232_printf("SPI is initializing...\r\n");
   spi_cs_tris    = 0;
   spi_clk_tris    = 0;
   spi_do_tris            = 0;
   spi_di_tris    = 1;
   spi_ldac_tris    = 0;


//#ifdef PIC18_SPIx1
   sspstat                = 0b00000000;
   sspstat.SMP            = 1;
   sspstat.CKE            = 1;
   sspcon1       = 0b00000000;    // clock = Fosc/4
//    sspcon1     = 0b00000001;    // clock = Fosc/16
//    sspcon1     = 0b00000010;    // clock = Fosc/64
   //sspcon1              = 0x20;
   sspcon1.CKP            = 0;

   pir1.SSPIF    = 0;    // clear interrupt flag
```

```
   pie1.SSPIE            = 0;    // enable interrupt flag

   sspcon1.WCOL    = 0;
   sspcon1.SSPEN    = 1;

   return 0;
}


void spi_put( unsigned char data ) {
   do {
         sspcon1.WCOL = 0;
         sspbuf = data;
   } while( sspcon1.WCOL );
   while( !sspstat.BF ) {};
}


unsigned char spi_get( void ) {
   spi_put( 0x00 );
   return sspbuf;
}


unsigned char spi_swap(unsigned char data) {
   spi_put(data);
   return sspbuf;
}

void main()
{
   unsigned char i;
   osccon =  0b01110000;
   osctune = 0b01000000;
   delay_ms(2);    // Allow PLL to stablize
   spi_init();
   lcd_cs_tris = 0;


   unsigned long segments;
   while(1)
   {

                lcd_cs = 0;
                spi_put(segments >>  16);
                spi_put(segments >>  8);
                spi_put(segments >>  0);

                lcd_cs = 1;
                delay_s(1);
                segments <<= 1;
   }
}
```

2. Compile and run the program on MT.SAC DEV board

In-Class Assignment

1. Base on your observation from the display of Mt.SAC DEV board, Figure out how to display each segment on LCD display

2. Base on the result from your In-Class Assignment 1, Write a function which is able to display digits 0 - 9 on MT.SAC DEV board LCD

Take-Home Assignment

1. Here is the source code for driving the LCD on MT.SAC DEV board. However, you need write the source code for functions **eeprom_read**, **eeprom_write** and **display_digits**. Write the program behave like the instructor shown in the class.

```
#include <system.h>
#include <string.h>
#include "rs232.h"

#pragma CLOCK_FREQ 32000000
#pragma DATA _CONFIG1H, _OSC_INTIO67_1H & _FCMEN_OFF_1H & _IESO_OFF_1H
#pragma DATA _CONFIG2L, _PWRT_OFF_2L & _BOREN_OFF_2L & _BORV_3_2L
#pragma DATA _CONFIG2H, _WDT_OFF_2H & _WDTPS_1024_2H
#pragma DATA _CONFIG3H, _MCLRE_ON_3H & _LPT1OSC_ON_3H & _PBADEN_OFF_3H &
_CCP2MX_PORTC_3H
#pragma DATA _CONFIG4L, _STVREN_ON_4L & _LVP_ON_4L & _XINST_OFF_4L &
_DEBUG_OFF_4L
#pragma DATA _CONFIG5L, _CP0_OFF_5L & _CP1_OFF_5L & _CP2_OFF_5L & _CP3_OFF_5L
#pragma DATA _CONFIG5H, _CPB_OFF_5H & _CPD_OFF_5H
#pragma DATA _CONFIG6L, _WRT0_OFF_6L & _WRT1_OFF_6L & _WRT2_OFF_6L &
_WRT3_OFF_6L
#pragma DATA _CONFIG6H, _WRTB_OFF_6H & _WRTC_OFF_6H & _WRTD_OFF_6H
#pragma DATA _CONFIG7L, _EBTR0_OFF_7L & _EBTR1_OFF_7L & _EBTR2_OFF_7L &
_EBTR3_OFF_7L
//#pragma DATA _CONFIG7H, _EBTRB_ON_7H &

volatile bit spi_clk_tris     @ TRISC.3;
volatile bit spi_do_tris      @ TRISC.5;
volatile bit spi_di_tris      @ TRISC.4;
volatile bit spi_cs_tris      @ TRISE.1;
volatile bit spi_ldac_tris    @ TRISE.2;
volatile bit lcd_cs_tris    @ TRISE.0;

volatile bit spi_clk        @ PORTC.3;
volatile bit spi_do         @ PORTC.5;
volatile bit spi_cs         @ PORTE.1;
volatile bit spi_ldac       @ LATE.2;
volatile bit lcd_cs                 @ LATE.0;

char eeprom_read(char address){

  //*************************************************************
   //Put your code here
```

```
 //************************************************************
}

char is_digit(us8 input){
   if((input<0x3a)&&(input>0x2f))
   {
         return 1;
   }
   else
   {
         return 0;
   }
}

void eeprom_write(us8 address, us8 data){
  //************************************************************
   //Put your code here
 //************************************************************
}


us8 display_digit (us8 digit)
{
  //************************************************************
   //Put your code here
 //************************************************************
}



int spi_init( void ) {
//    rs232_printf("SPI is initializing...\r\n");
   spi_cs_tris    = 0;
   spi_clk_tris    = 0;
   spi_do_tris          = 0;
   spi_di_tris    = 1;
   spi_ldac_tris    = 0;
   lcd_cs_tris = 0;


//#ifdef PIC18_SPIx1
   sspstat              = 0b00000000;
   sspstat.SMP          = 1;
   sspstat.CKE          = 1;
   sspcon1       = 0b00000000;    // clock = Fosc/4
//    sspcon1     = 0b00000001;    // clock = Fosc/16
//    sspcon1     = 0b00000010;    // clock = Fosc/64
   //sspcon1            = 0x20;
   sspcon1.CKP          = 0;

   pir1.SSPIF    = 0;    // clear interrupt flag
   pie1.SSPIE           = 0;    // enable interrupt flag

   sspcon1.WCOL    = 0;
```

```
   sspcon1.SSPEN    = 1;

   return 0;
}


void spi_put( unsigned char data ) {
   do {
          sspcon1.WCOL = 0;
          sspbuf = data;
   } while( sspcon1.WCOL );
   while( !sspstat.BF ) {};
}


unsigned char spi_get( void ) {
   spi_put( 0x00 );
   return sspbuf;
}


unsigned char spi_swap(unsigned char data) {
   spi_put(data);
   return sspbuf;
}



void main()
{
   trisb = 0;
   portb = 0;

   osccon =  0b01110000;
   osctune = 0b01000000;
   delay_ms(2); //wait for PLL to be stablized

   rs232_init( 16); // 115200 @32MHZ /4 clock //
   rs232_printf("MTSAC DEV BOARD PARSER V1.0\nType 'help' for more information\n");
   rs232_printf("MTSAC DEV>");

   bit parser_ready = 0;


   while( 1 )
   {
          us8 input_string[32];
          us8 mem_spot;
          us8 input;
          bit parser_ready = 1;
          mem_spot=0;
```

```
	while (parser_ready)
	{
		if (pir1 & 1 <<RCIF)
		{
			// If it was, go get it....
			input = rs232_getchar();
			input_string[mem_spot] = input;
			rs232_send_byte(input_string[mem_spot]);

			if (input == 0x0D) //0x0D is ENTER in ASCII code
			{

				if (!strncmp ( "help", input_string,4))
				{
					rs232_printf("help                    to show all
commands\n");
					rs232_printf("ver                     System
version\n");
					rs232_printf("ls                      list directory\n");
					us8 i;
					for ( i = 0 ; mem_spot != i ; i++ )
					{
						input_string[i]=0;
					}
					mem_spot = 0;
					parser_ready = 0;
					rs232_printf("MTSAC DEV>");
				}
				else if (!strncmp ( "ver", input_string,3))
				{
					rs232_printf("MT.SAC DEV-BOARD PARSER
V1.0\nProlinear/Pontech Inc.\n");
					rs232_printf("Author: John C. Tsai\n");
					us8 i;
					for ( i = 0 ; mem_spot != i ; i++ )
					{
						input_string[i]=0;
					}
					mem_spot = 0;
					parser_ready = 0;
					rs232_printf("MTSAC DEV>");
				}
				else if (!strncmp ( "porta", input_string,5))
				{
					rs232_printf(porta);
						us8 i;
					for ( i = 0 ; mem_spot != i ; i++ )
					{
						input_string[i]=0;
					}
					mem_spot = 0;
					parser_ready = 0;
					rs232_printf("\nMTSAC DEV>");
				}
```

```
else if (!strncmp ( "portb", input_string,5))
{
        rs232_printf(portb);
                us8 i;
        for ( i = 0 ; mem_spot != i ; i++ )
        {
                input_string[i]=0;
        }
        mem_spot = 0;
        parser_ready = 0;
        rs232_printf("\nMTSAC DEV>");
}
        else if (!strncmp ( "portc", input_string,5))
{
        rs232_printf(portc);
                us8 i;
        for ( i = 0 ; mem_spot != i ; i++ )
        {
                input_string[i]=0;
        }
        mem_spot = 0;
        parser_ready = 0;
        rs232_printf("\nMTSAC DEV>");
}
else if (!strncmp ( "portd", input_string,5))
{
        rs232_printf(portd);
                us8 i;
        for ( i = 0 ; mem_spot != i ; i++ )
        {
                input_string[i]=0;
        }
        mem_spot = 0;
        parser_ready = 0;
        rs232_printf("\nMTSAC DEV>");
}
else if (!strncmp ( "porte", input_string,5))
{
        rs232_printf(porte);
                us8 i;
        for ( i = 0 ; mem_spot != i ; i++ )
        {
                input_string[i]=0;
        }
        mem_spot = 0;
        parser_ready = 0;
        rs232_printf("\nMTSAC DEV>");
}
else if (!strncmp ( "spi_init", input_string,8))
{
        spi_init();
                us8 i;
        for ( i = 0 ; mem_spot != i ; i++ )
        {
```

```
                input_string[i]=0;
        }
        mem_spot = 0;
        parser_ready = 0;
        rs232_printf("\nMTSAC DEV>");
}
else if (!strncmp ( "is_digit", input_string,8))
{
        if (is_digit(input_string[9]))
        {
                rs232_printf("It is a digit\n\r");
        }
        else
        {
                rs232_printf("It is not a digit\n\r");
        }
                us8 i;
        for ( i = 0 ; mem_spot != i ; i++ )
        {
                input_string[i]=0;
        }
        mem_spot = 0;
        parser_ready = 0;
        rs232_printf("\nMTSAC DEV>");
}
else if (!strncmp ( "write", input_string,5))
{

if((is_digit(input_string[6]))&&(is_digit(input_string[7]))&&(is_digit(input_string[8])))
        {
                eeprom_write(0x00, input_string[6]);
                eeprom_write(0x01, input_string[7]);
                eeprom_write(0x02, input_string[8]);
        }
        else
        {
                rs232_printf("The input is not valid\r\n");
        }

        us8 i;
        for ( i = 0 ; mem_spot != i ; i++ )
        {
                input_string[i]=0;
        }
        mem_spot = 0;
        parser_ready = 0;
        rs232_printf("\nMTSAC DEV>");
}
else if (!strncmp ( "read", input_string,4))
{

        rs232_send_byte(eeprom_read(0x00));
        rs232_send_byte(eeprom_read(0x01));
        rs232_send_byte(eeprom_read(0x02));
```

```
                rs232_send_byte('\n');

                us8 i;

                for ( i = 0 ; mem_spot != i ; i++ )
                {
                        input_string[i]=0;
                }
                mem_spot = 0;
                parser_ready = 0;
                rs232_printf("\nMTSAC DEV>");
        }
        else if (!strncmp ( "display", input_string,7))
        {

                lcd_cs = 0;

                spi_put(display_digit(eeprom_read(0x00)));
                spi_put(display_digit(eeprom_read(0x01)));
                spi_put(display_digit(eeprom_read(0x02)));


                lcd_cs = 1;

                us8 i;

                for ( i = 0 ; mem_spot != i ; i++ )
                {
                        input_string[i]=0;
                }
                mem_spot = 0;
                parser_ready = 0;
                rs232_printf("\nMTSAC DEV>");
        }
        else
        {
                if (mem_spot==0)
                {
                        rs232_printf("MTSAC DEV>");
                }
                else if(mem_spot>32)
                {
                        rs232_printf("String is too long!\n");
                }
                else
                {
                        us8 i;
                        for ( i = 0 ; mem_spot != i ; i++ )
                        {
                                rs232_send_byte(input_string[i]);
                                input_string[i]=0;
                        }
                        rs232_printf(": command not found\n");
                        mem_spot = 0;
```

```
                                        parser_ready = 0;
                                        rs232_printf("MTSAC DEV>");
                                }
                        }
                }
                else
                {
                        mem_spot++;
                }
        }
    }
  }
}
```

2. Write a program to store E74 at Internal memory, and display the result on LCD

3. Extra Credit: Base on Assignment 1, create scrolling effect on LCD.

Lab 10 – Timer and Real Time Clock

Timer module is very useful feature of PICs. It can use for clock, stopwatch, metronome and many useful things beyond your imagination. Timer modules is also a better way to produce accurate timing compare to delay method. In PIC18F4620, there are 4 built-in timers and each of them have different functions and different purposes. It is TIME to get started!

Materials:

MT. SAC DEV BBOARD

References:

PIC18F2525/2620/4525/4620 data sheet

BoostC C Compiler for PICmicro Reference Manual

MT.SAC DEV Board schematic diagram

Goals:

1. Understand how Internal Timers of PIC works

2. Understand how to use Real-time clock for PIC

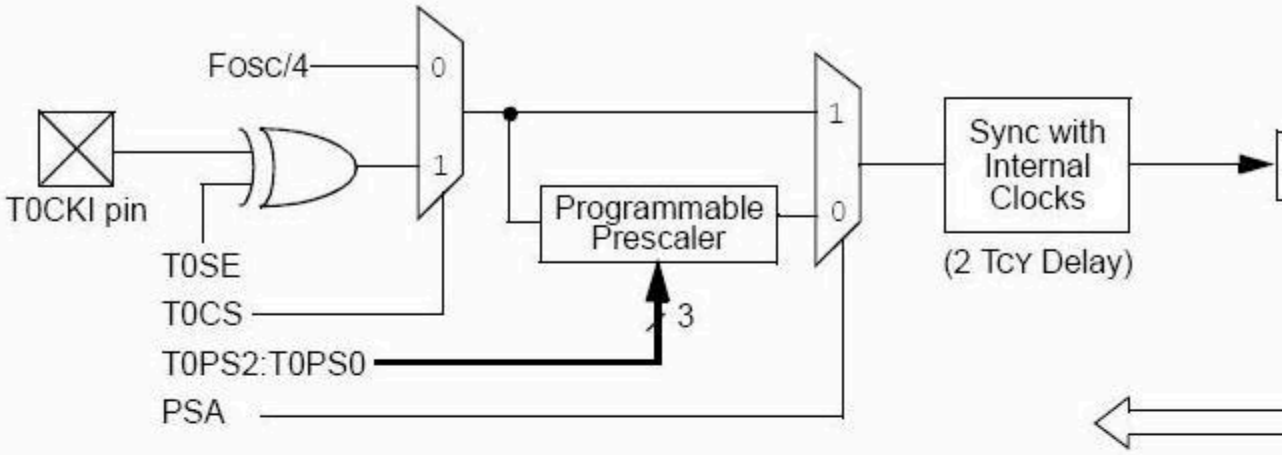
Part I. Timer

The Timer 0 if PIC18F4620 has following features:

Software selectable operation as a time or counter in both 8-bit or 16-bit modes
Readable and writable registers
Dedicated 8-bit, software programmable prescaler
Selectable clock source (internal or external)
Edge select for external clock
Interrupt-on-overflow

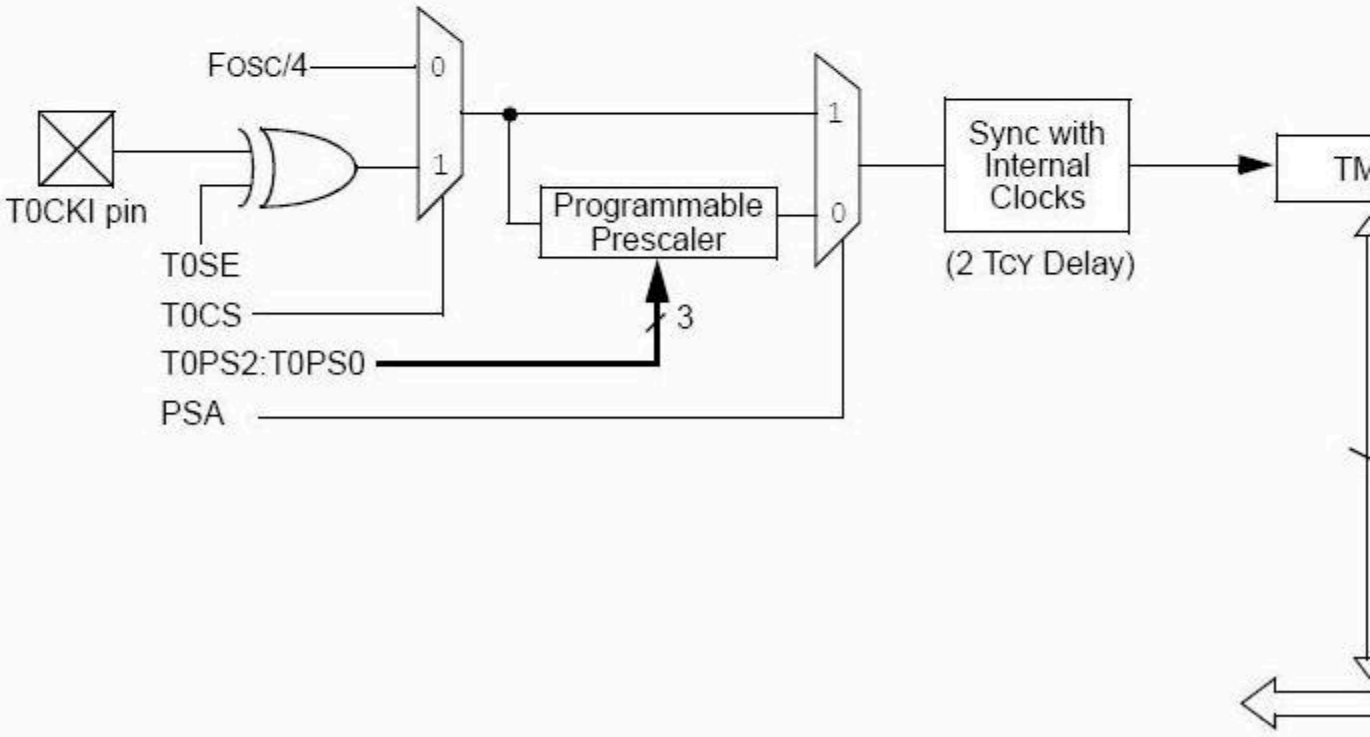
(from PIC18F2525/2620/4525/4620 Datasheet)

FIGURE 11-1: TIMER0 BLOCK DIAGRAM (8-BIT MODE)



Note: Upon Reset, Timer0 is enabled in 8-bit mode with clock input from T0CKI max. pre

FIGURE 11-2: TIMER0 BLOCK DIAGRAM (16-BIT MODE)



Note: Upon Reset, Timer0 is enabled in 8-bit mode with clock input from T0CKI max. prescale.

Timer0 can operate in either 8 bit or 16-bit modes.From the Diagram above, we can conclude that Timer-0 has four components:1.The clock input (either from the TMR0 pin or from the internal instruction clock).2. the TMR0 prescaler,3. the TMR0 register4. the TMR0 interrupt flags (GIE, T0IE, and T0IF)

THE CLOCK INPUT FOR TIMER0 —MT.SAC DEV Board is set to use 32MHZ. The instruction clock occur at 8 Mhz rate because there is an internal divide-by-4 between the oscillator frequency and the instruction Clock

THE TMR0 PRESCALER - The TMR0 prescaler is set via 3 bits register

T0PS2:T0PS0: Timer0 Prescaler Select bits

111 = 1:256 Prescale value
110 = 1:128 Prescale value
101 = 1:64 Prescale value
100 = 1:32 Prescale value
011 = 1:16 Prescale value
010 = 1:8 Prescale value
001 = 1:4 Prescale value
000 = 1:2 Prescale value

The TMR0 prescale is passed to the prescaler, whose divided-down output is then fed to the TMR0 register.

If the TMR0 is set to divide-by-8 and the PIC is running at 32 MHZ, the prescaler will send a 1 Mhhz clock to the TMR0 register ([32/4]/8)

TMR0 REGISTER – The TMR0 can be preloaded with any 8-bit value you like. When the value in the TMR0 register rolls over from 0xFF to 0x00, the T0IF flag is set.

TMR0 INTERRUPT FLAG – GIE and T0IE flags have to be set when the T0IF flag is set.

Procedure:

1. The code below show you how to turn on Timer0 and the overflow of timer toggle LED (portd.0). Save the code to **Timer0.c** . Compile the source code and upload it to Mt.SAC DEV board

```
void Timer0_Init(void);

void main()
{
trisd = 0;
portd = 0;

Timer0_Init();
```

```
while (1)
{

if (intcon.TMR0IF == 1){
portd.0 =~portd.0;
clear_bit(intcon, TMR0IF);
}
}


}

void Timer0_Init(void)
{
clear_bit(intcon, TMR0IF); // clear roll-over interrupt flag
// clear_bit(intcon2, TMR0IP); // Timer0 is low priority interrupt
t0con = 0x08; //T0CON = b00001000; no prescale - increments every instruction clock
// t0con = 0x01; //T0CON = 0b00000001; // prescale 1:4 - four times the delay
tmr0h = 0x00; //clear timer - always write upper byte first
tmr0l = 0x00;

set_bit(t0con, TMR0ON); // start timer
}
```

In-Class Assignment:

1. Use Timer0 to drive array of LED patterns.

2. Use Timer0 to count from 1 to 10 and display it on LCD

Part II. Other Timers (Timer1, Timer2, and Timer3)

In part I, we introduced you timer module and we know how to configure and turn on timer0. In part II, We will show you how other timer modules work. Some are similar, and some are totally different.

Timer1

For timer1, it is very similar to timer0, but it has a little bit different

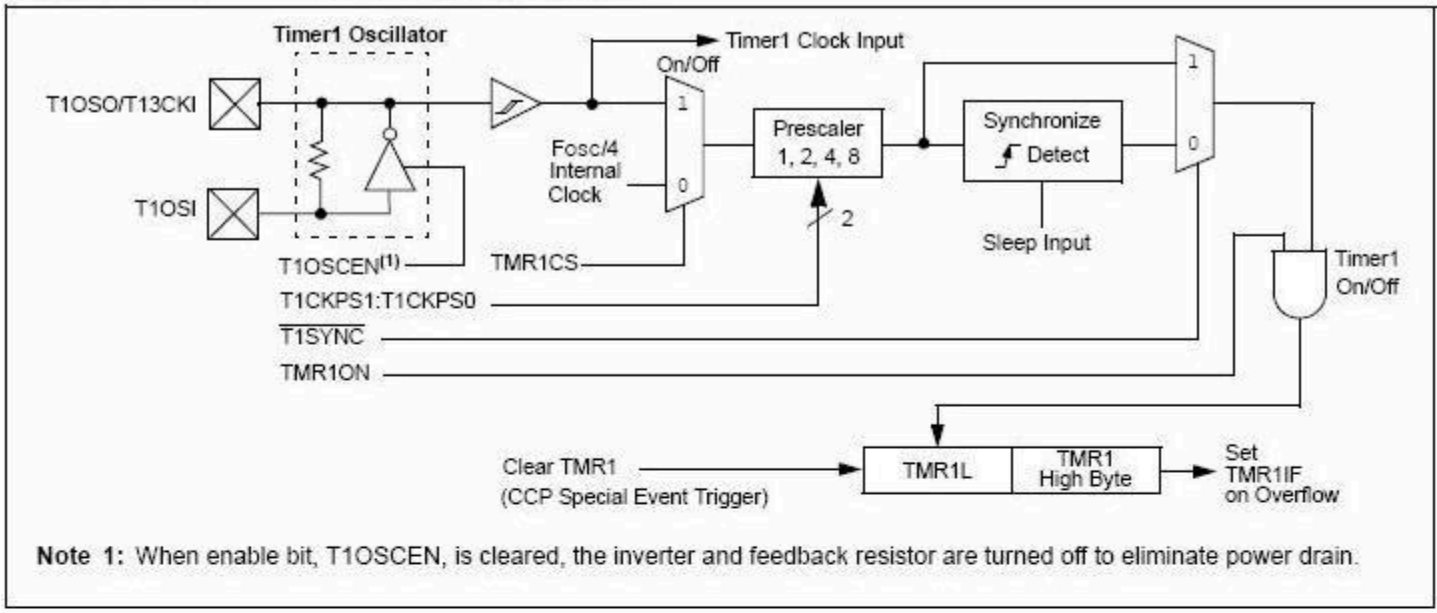
1.The clock input is quite different. (Compare between diagram for Timer0 and Timer1)2. the TMR1 prescaler only have 2 select bits, compare to 3 select bits to TMR0

11 1:8 Prescale value
10 1:4 Prescale value
01 1:2 Prescale value
00 1:1 Prescale value

3. the TMR1 have same register bit as TMR0. But TMR1 has special even trigger to clear TMR0 Register4. the way interrupt flag of TMR0 and TMR1 operate in the same way.

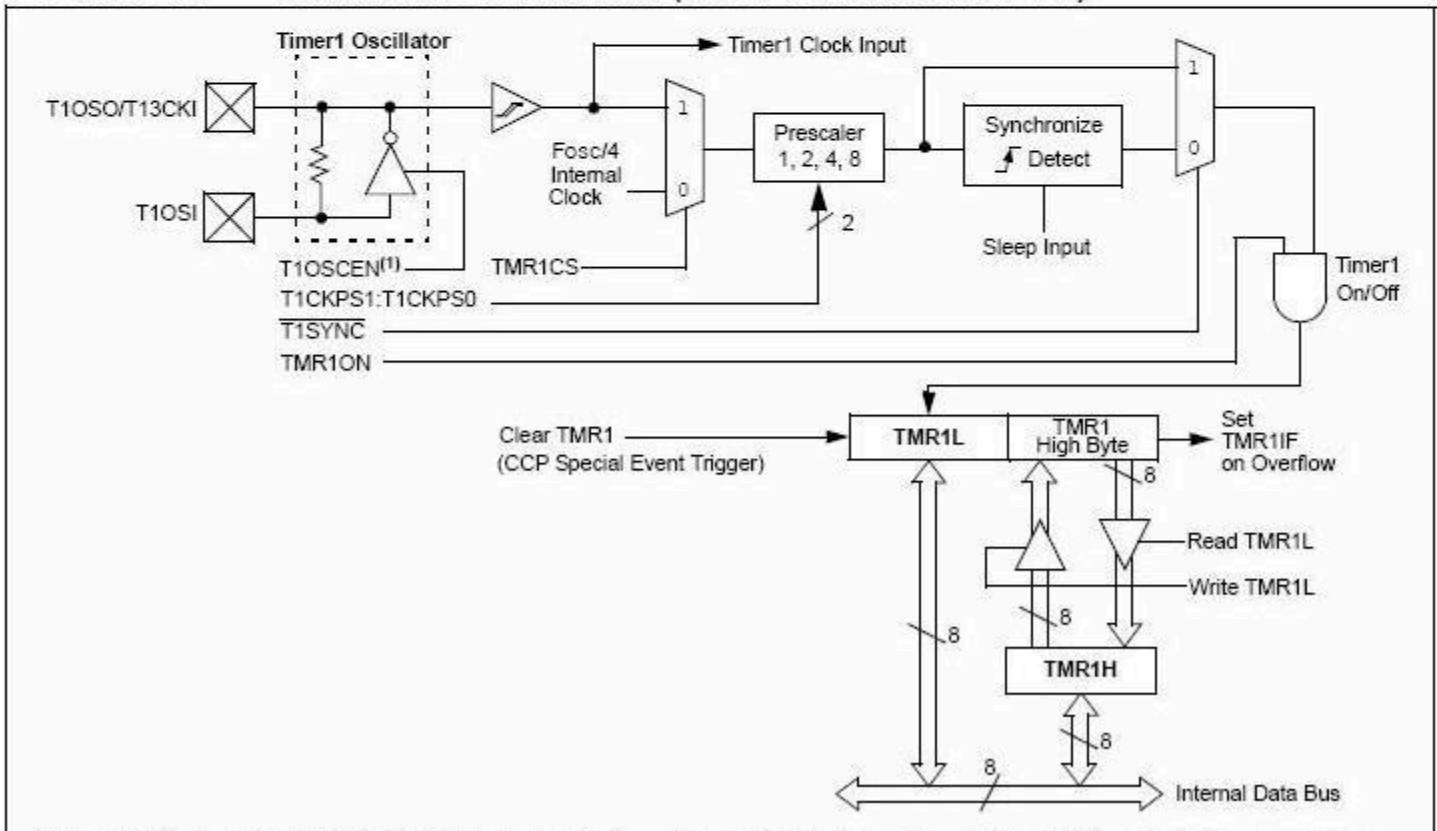
The diagram of Timer1 is quite different than the Timer0, The block diagram for Timer1 is showing below:

FIGURE 12-1: TIMER1 BLOCK DIAGRAM



Note 1: When enable bit, T1OSCEN, is cleared, the inverter and feedback resistor are turned off to eliminate power drain.

FIGURE 12-2: TIMER1 BLOCK DIAGRAM (16-BIT READ/WRITE MODE)



Note 1: When enable bit, T1OSCEN, is cleared, the inverter and feedback resistor are turned off to eliminate power drain.

However, the way to enable timer1 is very similar to timer0, except the prescale select bits

```
void main()
{
trisd = 0;
portd = 0;

Timer0_Init();
Timer1_Init();

while (1)
{

if (intcon.TMR0IF == 1){
portd.0 =~portd.0;
clear_bit(intcon, TMR0IF);
}


if (pir1.TMR1IF == 1){
portd.1 =~portd.1;
clear_bit(pir1, TMR1IF);
}
}


}

void Timer0_Init(void)
{
clear_bit(intcon, TMR0IF); // clear roll-over interrupt flag
// clear_bit(intcon2, TMR0IP); // Timer0 is low priority interrupt
t0con = 0x08; //T0CON = b00001000; no prescale - increments every instruction clock
// t0con = 0x01; //T0CON = 0b00000001; // prescale 1:4 - four times the delay
tmr0h = 0x00; //clear timer - always write upper byte first
tmr0l = 0x00;

set_bit(t0con, TMR0ON); // start timer
}

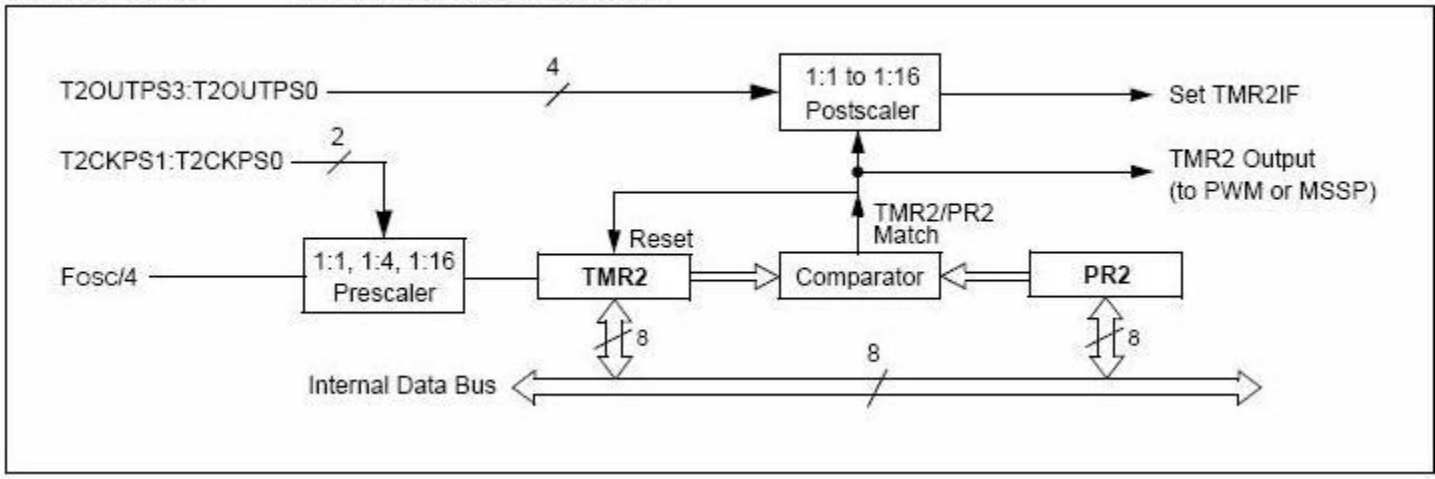
void Timer1_Init(void)
{
//Init Timer1
clear_bit(pir1, TMR1IF);
t1con = 0b01011100;
tmr1h = 0x00;
tmr1l = 0x00;

set_bit(t1con, TMR1ON);
}
```

How about Timer2?? The Timer2 of PIC18F4620 is dedicate for PWM, the more detail we will

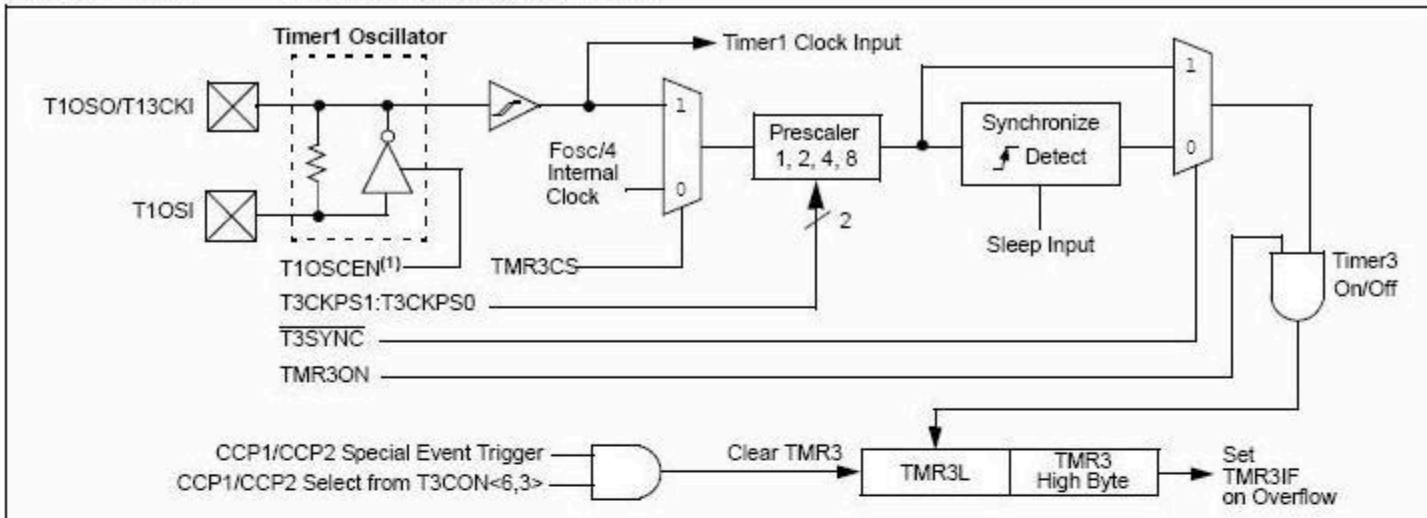
discuss about this timer in next lab.

FIGURE 13-1: TIMER2 BLOCK DIAGRAM



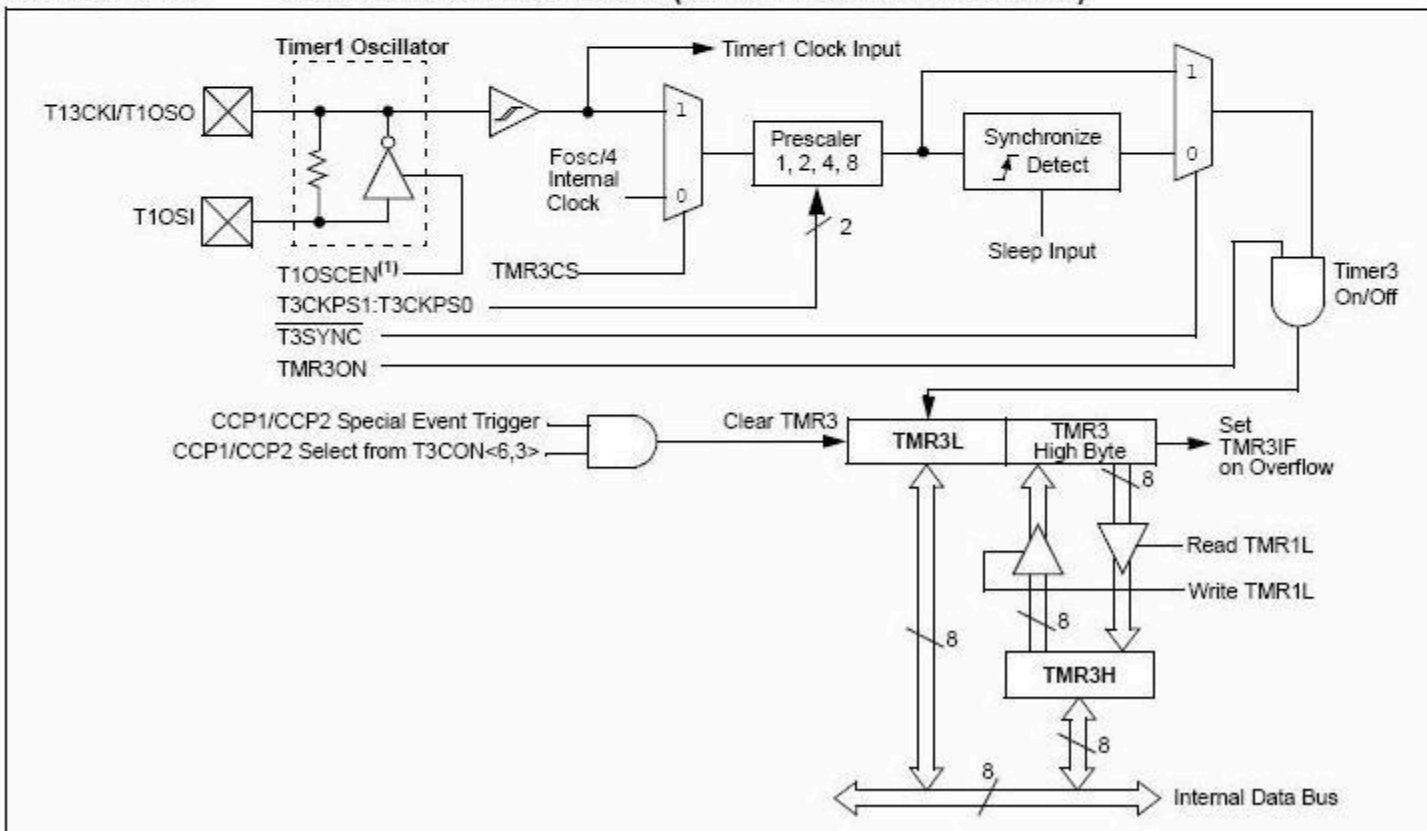
The last timer(timer3) is very similar to Timer1. however, Timer3 have more option for special event trigger, It can either clear by CCP1 or CCP2 (Select from T3CON<6,3>

FIGURE 14-1: TIMER3 BLOCK DIAGRAM



Note 1: When enable bit, T1OSCEN, is cleared, the inverter and feedback resistor are turned off to eliminate power drain.

FIGURE 14-2: TIMER3 BLOCK DIAGRAM (16-BIT READ/WRITE MODE)



Note 1: When enable bit, T1OSCEN, is cleared, the inverter and feedback resistor are turned off to eliminate power drain.

In-Class Assignment:

1. Enable Timer0 and Timer1 just like Lab exercise, read datasheet and the codes from exercise. Enable Timer3 and set prescale to 1:8 and toggle LED 3. It is not an easy exercise. (Hint: Read 14.3 and read other related part carefully as well)
2. Bonus Assignment: After enable Timer0, Timer1, and Timer3, enable Timer2 and make Timer2 toggle LED 4 and set it correspond to LED1, LED2, LED3

Part III. Timer Interrupt

Timer interrupt should be easy after knowing how to setup Timer, external interrupt, and interrupt on change. After Timer overflow, the interrupt flag will set and interrupt occurs

Anyway, here is the code for how to setup Timer Interrupt:

```
void Timer0_Init(void);

void main()
{
trisd = 0;
portd = 0;

Timer0_Init();

while (1)
{
}
}

void Timer0_Init(void)
{
//enable interrupts
intcon.GIE = 1;
intcon.PEIE = 1;
intcon.TMR0IE = 1; //Enable Timer 0 Interrupt

clear_bit(intcon, TMR0IF); // clear roll-over interrupt flag
// clear_bit(intcon2, TMR0IP); // Timer0 is low priority interrupt
t0con = 0x08; //T0CON = b00001000; no prescale - increments every instruction clock
// t0con = 0x01; //T0CON = 0b00000001; // prescale 1:4 - four times the delay
tmr0h = 0x00; //clear timer - always write upper byte first
tmr0l = 0x00;

set_bit(t0con, TMR0ON); // start timer
}

void interrupt(void)
{
portd.0 =~portd.0;
clear_bit(intcon, TMR0IF);
}
```

In-Class Assignment

1. Enable Timer0 and Timer1 at same time and toggle LED1 and LED2 (set with different prescales)

Part IV Real-Time Clock

The real time clock on Mt.SAC DEV board is used for many common digital watch or digital clocks. Learn to use PIC to control real-time clock device, you will be able to make digital watch or digital clocks your self!!

procedure:

1. Type in the code below and name it **real-time-clock.c**, compile it and upload to the Mt.SAC DEV board

****

to be developed....

****

In-Class Assignment

1. Use real-time clock to count from 1 to 10 and display on LCD

Take-Home Assignment

1. Use real-time clock to output serial string telling how much time has been passed since the program start running.

LAB 11 Pulse-Width Modulation (PWM), Buzzer and Servo Motor

Pulse-Width Modulation can be used for many real-life applications. In telecommunications, PWM can be used as clock signal. For industrial electronics, PWM can be use for power delivery and voltage regulator to drive servo motors. PWM can also use for audio effect (such as buzzer) and amplification. PWM is an extremely useful knowledge have to learn in the field of electronic. Many PICs has hardware to generate Pulse-Width Modulation. Why not learn the way using PIC to generate PWM now!

Materials:

MT. SAC DEV BBOARD

References:

PIC18F2525/2620/4525/4620 data sheet

BoostC C Compiler for PICmicro Reference Manual

MT.SAC DEV Board schematic diagram

Goals:

1. Understand how to generate PWM signal

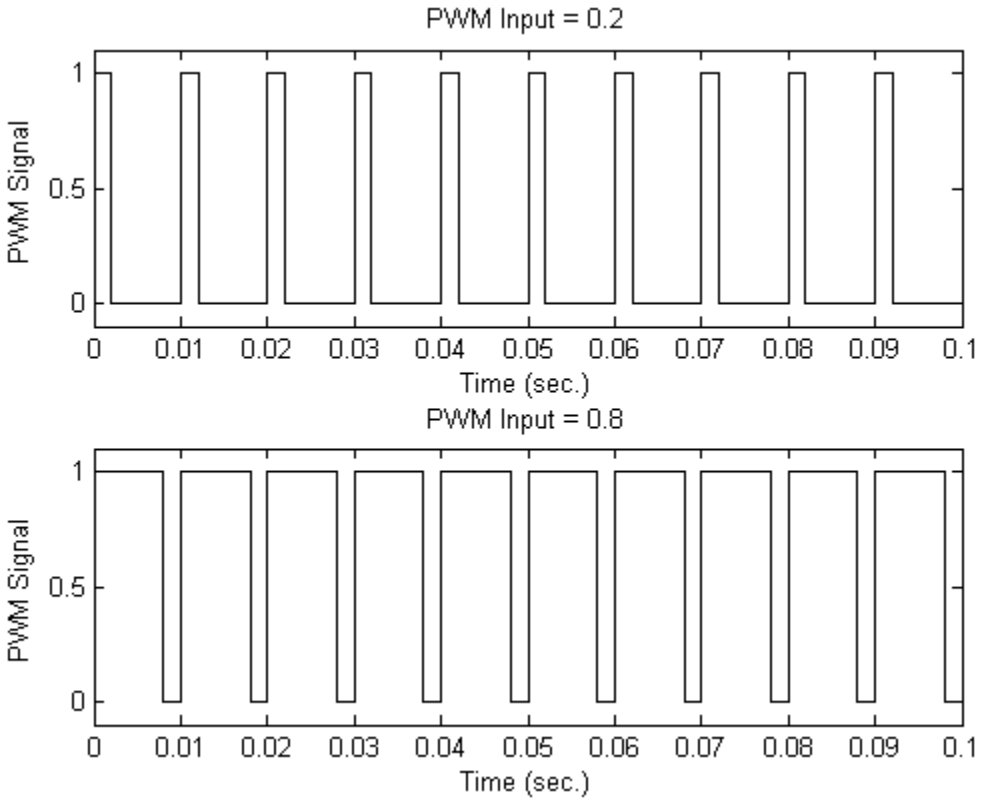
2. Understand how Servo motor works

3 Understnad how Buzzer works

Part I. What is Pulse-Width Modulation?

Pulse-Width Modulation is a signal or power source involves the modulation of its duty cycle.

It looks like square wave like below:



PWM period and **Duty cycle** are two variables decide your PWM signal look like.

<PWM Picture>

In-Class Assignment

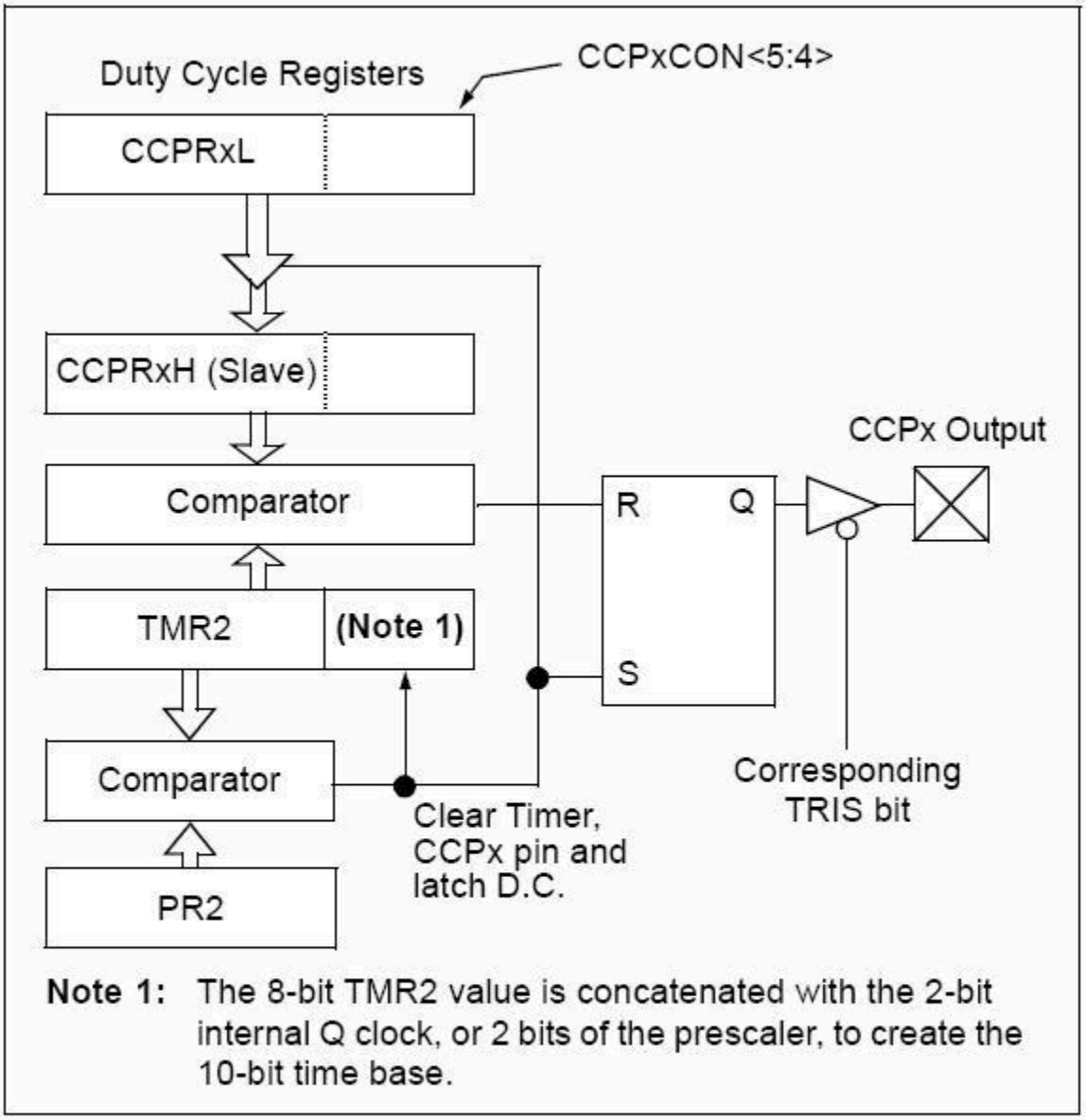
1. Use bit-banging method to generate PWM signal.

2. Use Parser to change duty cycle and period of PWM

Part II. How to enable PWM for PIC?

PIC18F4620 have two CCP (Capture/Compare/PWM) modules and Timer2 is the dedicate timer resource for PWM. The block diagram below shows How Timer2, PR2 (Period Register) and CCPRxl (Duty Cycle Register) produce PWM

FIGURE 15-3: SIMPLIFIED PWM BLOCK DIAGRAM



Note 1: The 8-bit TMR2 value is concatenated with the 2-bit internal Q clock, or 2 bits of the prescaler, to create the 10-bit time base.

A PWM output (Figure 15-4) has a time base (period) and a time that the output stays high (duty cycle). The frequency of the PWM is the inverse of the period (1/period).

The code below show you how to enable CCP1. For more detail, read Datasheet (15.4.4 SETUP FOR PWM OPERATION):

```
void main()
{
      trisc.2 = 0; // PWM1 output

      osccon = 0b01110000;
      osctune = 0b01000000;
      delay_ms(2); //wait for PLL to be stablized



 volatile bit t2ckps0@T2CON.0;
 volatile bit t2ckps1@T2CON.1;

 pr2 = 0xFF; // PR2 = Timer2 period register.
 ccpr1l = 0x80;    // CCPR1L = Capture/Compare/PWM Register1 Low Byte
 t2con = 0; // clear bits for t1con

 set_bit(t2con,2); // Turn on timer2

 t2ckps0=1; // Try changing these bits...
 t2ckps1=0; // ...see what happens.

 ccp1con = 0x0F;   // CCP1CON = Capture/Compare/PWM Control -- put CCP1 into PWM
mode
 clear_bit(trisc,2); // Portb, pin3 is the PWM output.

      while( 1 )
      {
      }
}
```

Compile and upload the code to MTSAC DEV BOARD, Check Datasheet and figure out which pin is RC2. Use Ossiliscope to probe RC2 and make sure it has PWM output.

How about CCP2? The code below shows how to setup

```
void main()
{
trisc.2 = 0; // PWM1 output
trisc.1 = 0; // PWM2 output

osccon = 0b01110000;
osctune = 0b01000000;
delay_ms(2); //wait for PLL to be stablized



volatile bit t2ckps0@T2CON.0;
volatile bit t2ckps1@T2CON.1;

pr2 = 0xFF; // PR2 = Timer2 period register.
```

```
ccpr1l = 0x80; // CCPR1L = Capture/Compare/PWM Register1 Low Byte
ccpr2l = 0xa0; // CCPR2L = Coature/Compare/PWM Register2 Low Byte
t2con = 0; // clear bits for t2con

set_bit(t2con,2); // Turn on timer2

t2ckps0=1; // Try changing these bits...
t2ckps1=0; // ...see what happens.

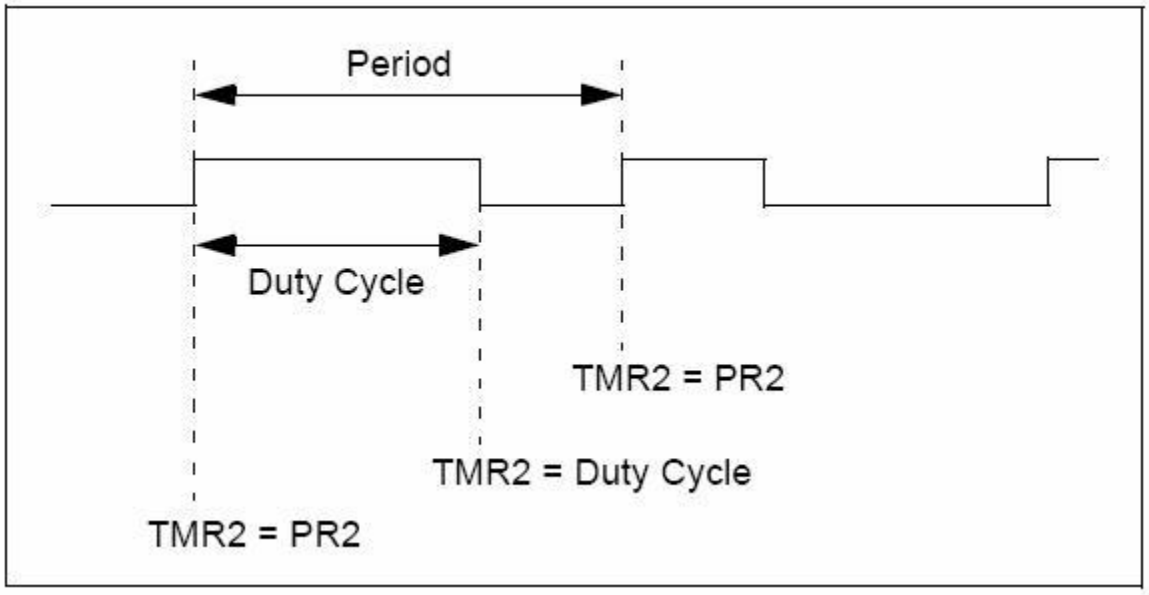
ccp1con = 0x0F; // CCP1CON = Capture/Compare/PWM Control -- put CCP1 into PWM mode
clear_bit(trisc,2); // PortC, pin2 is the PWM output.

ccp2con = 0x0F;
clear_bit(trisc,1);

while( 1 )
{
}
}
```

After enable PWM modules, it is important to learn how to adjust PWM PERIOD and Duty Cycle

FIGURE 15-4: PWM OUTPUT



A period is duty cycle + non-duty cycle.

The Equation for PWM Period is:

PWM Period = [(PR2)+1]*4*TOSC*(TMR2 Prescale Value)

The Equation for PWM duty cycle is:

PWM Duty Cycle = (CCPRxL:CCPxCON<5:4>)*Tosc*(TMR2 Prescale Value)

In-Class Assignment:

1. Write a program that push button 1 to enable PWM, and push button 1 again to disable PWM.

2. Bonus question: Write a program for CCP1 that will auto-shutdown PWM.

Part III. Buzzer

To enable the Buzzer on MTSAC DEV board. Change one of the Configuration bit in 3H to...

_CCP2MX_PORTBE_3H

```
void main()
{
trisb.3 = 0;

osccon = 0b01110000;
osctune = 0b01000000;
delay_ms(2); //wait for PLL to be stablized

volatile bit t2ckps0@T2CON.0;
volatile bit t2ckps1@T2CON.1;

pr2 = 0xFF; // PR2 = Timer2 period register.
ccpr2l = 0x80; // CCPR1L = Capture/Compare/PWM Register1 Low Byte
t2con = 0; // clear bits for t2con

set_bit(t2con,2); // Turn on timer2

t2ckps0=1; // Try changing these bits...
t2ckps1=0; // ...see what happens.

ccp2con = 0x0F; // CCP2CON = Capture/Compare/PWM Control -- put CCP2 into PWM mode
clear_bit(trisb,3); // Portb, pin3 is the PWM output.

while( 1 )
{
}
}
```

Yeah, it buzzed. Don't you feel it is a little bit boring? Let add some tunes.....

he keyss are defined as:

C 1,046Hz PP = 240

D 1,174Hz PP = 212
E 1,318Hz PP = 200
F 1,396Hz PP = 180
G 1,568Hz PP = 160
A 1,760Hz PP = 142
B 1846Hz PP = 130

Here is the code that play an octave and each note for 1 second

```
void C_note(void);
void D_note(void);
void E_note(void);
void F_note(void);
void G_note(void);
void A_note(void);
void B_note(void);


void main()
{
trisb.3 = 0;

osccon = 0b01110000;
osctune = 0b01000000;
delay_ms(2); //wait for PLL to be stablized

volatile bit t2ckps0@T2CON.0;
volatile bit t2ckps1@T2CON.1;

pr2 = 0xFF; // PR2 = Timer2 period register.
ccpr2l = 0x80; // CCPR1L = Capture/Compare/PWM Register1 Low Byte
t2con = 0; // clear bits for t2con

set_bit(t2con,2); // Turn on timer2

t2ckps0=1; // Try changing these bits...
t2ckps1=0; // ...see what happens.

ccp2con = 0x0F; // CCP2CON = Capture/Compare/PWM Control -- put CCP2 into PWM mode
clear_bit(trisb,3); // Portb, pin3 is the PWM output.

while( 1 )
{
C_note();
D_note();
E_note();
F_note();
G_note();
A_note();
B_note();
}
}

void C_note(void)
```

```
{
pr2 = 240;
ccpr1l = pr2/2;
delay_s(1);
}

void D_note(void)
{
pr2 = 212;
ccpr1l = pr2/2;
delay_s(1);
}

void E_note(void)
{
pr2 = 188;
ccpr1l = pr2/2;
delay_s(1);
}

void F_note(void)
{
pr2 = 180;
ccpr1l = pr2/2;
delay_s(1);
}

void G_note(void)
{
pr2 = 160;
ccpr1l = pr2/2;
delay_s(1);
}

void A_note(void)
{
pr2 = 142;
ccpr1l = pr2/2;
delay_s(1);
}

void B_note(void)
{
pr2 = 130;
ccpr1l = pr2/2;
delay_s(1);
}
```

In-Class Assignment:

1. Write a song you like and use buzzer to perfrom it.

Take-Home Assignment:

1. Write a program that making buzzer buzz in different tones, when push different buttons. For example, when push button 1, it buzz in C note. Button 2 in D note and so on.....

2. Bonus Assignment: Write a program that play Twelve Variations on "Ah vous dirai-je, Maman", K. 254/300e by Mozart.

Final Project - Real World Applications

We finished all the lab from this manual. It is the time for you to show what have you learned. Using Mt.SAC DEV board to create the applications you always dream on!

Materials:

MT. SAC DEV BBOARD

References:

PIC18F2525/2620/4525/4620 data sheet

BoostC C Compiler for PICmicro Reference Manual

MT.SAC DEV Board schematic diagram

Your Brain

Goals:

1. Use you imagination to create your dream application

Procedure:

1. Decide the project you want to make:

Example Project I.

Piano

Application II.

Simon says it

Application III

Digital Voltmeter

Applications:
Digital Etch-a-sketch
Digital Clock/Alarm Clock
DVM
Simon says

Sketch
Infrared/Remote control
Robot Application
Temperature Sensor
Function Generator
Piano/Organ/Music Box
Sampler

SPI -> LCD and DAC and Serial EERPOM
ADC -> Temp and Light and POTs
DAC & ADC -> Etch-a-sketch
PWM -> Buzzer
RTC

C Programing Topics

What is C Programming language?

Dennis Ritchie created C at the Bell Telephone Laboratories in 1972. C programming language is an general purpose and cross-platform computer programming language originally using with the Unix operating system.
It was designed to be compiled using a relative straightforward compiler because C is intended to map effectively to low level machine instructions. Because of its low-level capabilities, C is useful for many applications
written in assembly language previously. Since it match to machine instructions, C can be port to a wide variety of computers. a supercomputer such as "Deep Blue" to embedded microcontroller such as PIC.

Source Code -> Compiler -> Linker -> Object Code

When we start to write program in C. We have to write C source code first. Source code is a collection of statement or declarations written in
Human-readable computer programming language. C source code has to be compiled with a C compiler to create machine code translation in
order for machine to understand it. The machine code is not directly usable until linking is done by a program called Linker. The linker add library
codes to the main program in order to create a working program.

Variable + Data Types + Literals + Arrays

Definitions:

Variables - In computer programming a variable is a special element that act as a handle (also often called a reference) to a particular value.

Data Types - A data type in programming languages is an attribute of a data which tell the

computer something about the kind of data it is.

Literals - Any notation for representing a value within programming language.

Arrays - A data structure consisting of a group of elements that are accessed by index

Variables - Traditional C practice is to use lower case for variable names , and all upper case for symbolic constants (K & R P. 35)

Data Types - A sort of information stored in a particular way

Literals

Arrays

Operators
Loops
- for
- while
- do

Conditionals
- if
- if-else
- switch

Functions
- passing parameters
- returning values

Pointers
Interrupts
Includes
Preprocessor Commands
- #define
- #pragma
- #ifdef
- #else
- #endif

Structures + Unions

Electronics Topics

Memory
- Non-Volatile - NVM, a memory able to retain stored information even when not powered.
  - RAM - Static - or SRAM, Unlike DRAM, it does not need to be refreshed periodically.
    - RAM - Dynamic - or DRAM, it is volatile memory
- Volatile
  - ROM - a non-volatile memory, Read Only Memory, Data stord in ROM cannot be modified.
  - PROM - a non-volatile memory, Programmable ROM. Once PROM programmed, it programmed permenently. They are frequntly seen in video game consoles.
  - EPROM - a non-volatile memory, requiring a UV light to erase it.
  - EEPROM - It can also written as E^2PROM and prononunced e-e-prom, a non-volatile memory, can be erased and rewritten electronically.

Flash - a non-volatile memory, a type of EEPROM, but a term EEPROM is generally for non-flash type of EEPROM. Modern PIC is using flashing memory technology. Other information storage devices are such as Compact Flash, USB flash drive, Multi-media Card(MMC), Secure Digital(SD), Memory Stick(MS)...etc

Notes: Upcoming Memory type,

Volatile:

- Z-RAM
- TTRAM

Non-volatile:

- FeRAM
- MRAM
- CBRAM
- PRAM
- SONOS
- RRAM
- Racetrack memory
- NRAM

ADC

Temperature Sensing

Light Sensing

DAC

Apendix ? - ASCII Chart

Binary	Oct	Dec	Hex	Glyph
010 0000	040	32	20	
010 0001	041	33	21	!
010 0010	042	34	22	"
010 0011	043	35	23	#
010 0100	044	36	24	$
010 0101	045	37	25	%
010 0110	046	38	26	&
010 0111	047	39	27	'
010 1000	050	40	28	(
010 1001	051	41	29	)
010 1010	052	42	2A	*
010 1011	053	43	2B	+
010 1100	054	44	2C	,
010 1101	055	45	2D	-
010 1110	056	46	2E	.
010 1111	057	47	2F	/
011 0000	060	48	30	0

011 0001	061	49	31	1
011 0010	062	50	32	2
011 0011	063	51	33	3
011 0100	064	52	34	4
011 0101	065	53	35	5
011 0110	066	54	36	6
011 0111	067	55	37	7
011 1000	070	56	38	8
011 1001	071	57	39	9
011 1010	072	58	3A	:
011 1011	073	59	3B	;
011 1100	074	60	3C	<
011 1101	075	61	3D	=
011 1110	076	62	3E	>
011 1111	077	63	3F	?

Binary	Oct	Dec	Hex	Glyph
100 0000	100	64	40	@
100 0001	101	65	41	A
100 0010	102	66	42	B
100 0011	103	67	43	C
100 0100	104	68	44	D
100 0101	105	69	45	E
100 0110	106	70	46	F
100 0111	107	71	47	G
100 1000	110	72	48	H
100 1001	111	73	49	I
100 1010	112	74	4A	J
100 1011	113	75	4B	K
100 1100	114	76	4C	L
100 1101	115	77	4D	M
100 1110	116	78	4E	N
100 1111	117	79	4F	O
101 0000	120	80	50	P
101 0001	121	81	51	Q
101 0010	122	82	52	R
101 0011	123	83	53	S
101 0100	124	84	54	T
101 0101	125	85	55	U
101 0110	126	86	56	V
101 0111	127	87	57	W
101 1000	130	88	58	X
101 1001	131	89	59	Y
101 1010	132	90	5A	Z
101 1011	133	91	5B	[
101 1100	134	92	5C	\
101 1101	135	93	5D	]
101 1110	136	94	5E	^
101 1111	137	95	5F	_

Binary	Oct	Dec	Hex	Glyph
110 0000	140	96	60	`
110 0001	141	97	61	a

110 0010	142	98	62	b
110 0011	143	99	63	c
110 0100	144	100	64	d
110 0101	145	101	65	e
110 0110	146	102	66	f
110 0111	147	103	67	g
110 1000	150	104	68	h
110 1001	151	105	69	i
110 1010	152	106	6A	j
110 1011	153	107	6B	k
110 1100	154	108	6C	l
110 1101	155	109	6D	m
110 1110	156	110	6E	n
110 1111	157	111	6F	o
111 0000	160	112	70	p
111 0001	161	113	71	q
111 0010	162	114	72	r
111 0011	163	115	73	s
111 0100	164	116	74	t
111 0101	165	117	75	u
111 0110	166	118	76	v
111 0111	167	119	77	w
111 1000	170	120	78	x
111 1001	171	121	79	y
111 1010	172	122	7A	z
111 1011	173	123	7B	{
111 1100	174	124	7C	|
111 1101	175	125	7D	}
111 1110	176	126	7E	~

Apendix ? - Datasheets
Apendix ? - Schematic
Apendix ? - C Reference

The C Language Reference

Operators:

Arithmetic Operators (K&R 2.5)

In Brian W. Kernighan and Dennis M. Ritchie's The C "Origramming lannguage" chpater 2 section 5 mentioned " The binary arithmetic operators are +, -, *, / and the modulus operator %."

Here is the complete list of arithmetic operators and its syntax:

Operator Name	Syntax
Unary Plus	**+**a
Addition (Sum)	a **+** b
Assignment by Addition	a **+=** b
Unary Minus (Negation)	**-**a
Subtraction (Difference)	a **-**
Assignment by Subtraction	a **-=** b
Multiplication (Product)	a ***** b
Assignment by	a ***=** b

Multiplication
Division (Quotient) a **/** b
Assignment by Division a **/=** b
Modulus (Remainder) a **%** b
Assignment by Modulus a **%=** b

Relational and Logical Operators

The relational operators are (K&R 2.6 P.41)

> >= < <=

They all have the same precedence. Just below them in precedence are the equality operator:

== !=

Operator Name	Syntax
Less Than	a **<** b
Less Than or Equal To	a **<=** b
Greater Than	a **>** b
Greater Than or Equal To	a **>=** b
Not Equal To	a **!=** b
Equal To	a **==** b

...More interesting are the logical operators && and ||. (K&R 2.6 P.42)
...The unary negation operator ! converts a non-zero operand into 0, and a zero operand into 1. (K&R 2.6 P.42)

Operator Name	Syntax
Logical Negation	**!**a
Logical AND	a **&&** b
Logical OR	a **||** b

Increment and Decrement Operators

C provides two unusal operators for incrementing and decrementing variales. The increment operator ++ adds 1 to its operand, while the
decrement operator -- subtracts 1. We have frequently used ++ to increment variables.....(K&R 2.8)

Operator Name	Syntax
Prefix Increment	**++**a
Postfix Increment	a**++**
Prefix Decrement	**--**a
Postfix Drecrement	a**--**

Bitwise Operators

C provides six operator for bit manipulation; these may only be applied to integral operands,

that is, char, short, int, and long, whether signed or unsigned

&	bitwise AND
|	bitwise inclusive OR
^	bitwise exclusive OR
<<	left shift
>>	right shift
~	one's complement (unary)

(K & R 2.9 p.48)

Operator Name	Syntax
Bitwise Left Shift	a **<<** b
Assignment by Bitwise Left Shift	a **<<=** b
Bitwise Right Shift	a **>>** b
Assignment by Bitwise Right Shift	a **>>=** b
Bitwise One's Complement	**~**a
Bitwise AND	a **&** b
Assignment by Bitwise AND	a **&=** b
Bitwise OR	a **|** b
Assignment by Bitwise OR	a **|=** b
Bitwise XOR	a **^** b
Assignment by Bitwise XOR	a **^=** b

Other Operators

Operator Name	Syntax
Basic Assignment	a **=** b
Function, Call	a**()**
Array Subscript	a**[**b**]**
Indirection (Dereference)	***a**
Address-of(Reference)	**&**a
Member by Pointer	a**->**b
Member	a**.**b
Cast	(*type*) a
Comma	a **,** b
Ternary Conditional	a **?** b **:** c
Size-of	**sizeof** a **sizeof**(*type*)

Language extensions

Operator Name	Syntax
Label Address	**&&** label
Get Type	**typeof** a **typeof**(*expr*)

Reserved Words:

Data Type and Size (K&G 2.2)

A data type is a particular sort of information stored in a particular way

There are several way in represent a number 42 in the real life

Method used to represent 42	**Representation**
Decimal Number	42
Roman Numerals	XLII
English	forty-two
Hexadecimal Number	$2A_{16}$
Spanish	cuarenta y dos
French	quarante-deux
German	zweiundvierzig
Tagalog	apatnapu't dalawang
Chinese	四十二，肆拾贰
Japanese	四十二，四拾弐
Arabic	اثنين والأربعين
Czech	dvaačtyřicet
Greek	σαράντα δύο
Hebrew	ארבעים שני
Hindi	बयालीस
Russian	сорок два
Binary	101010_2
Octal	52_8
Duodecimal	36_{12}
The hitchhiker guide to the galaxy	Answer to life, the Universe and Everything

There are only a few basic data types in C: (K&R 2.2 P.36)

char a single byte, capable of holding on character in the local character set.

int an integer, typically reflecting the natural size of integers on the host machine.

float single-precision floating point

double double-precision floating point

In addition, there are a number of qualifiers that can be applied to these basic types. short and long apply to integers

```
short int sh;
long int counter;
```

.....

.....

The complete set of escape sequences is (K&R 2.3 P38)

\a	alert (bell) character
\b	backspace
\f	formfeed
\n	newline
\r	carriage return
\t	horizontal tab
\v	vertical tab
\\	backslash
\?	question mark
\'	single quote
\"	double quote

\ooo octal number

\xhh hexadecimal number

Apendix ? - Setup HyperTerminal

Getting Started with HyperTerminal

HyperTerminal is a multipurpose text terminal communications program available on various versions of Windows. Here you will learn how to use HyperTerminal to open a serial connection and communicate with a Pontech Board.

Start HyperTerminal.

Here is a quick way to start HyperTerminal.

You can also start HyperTerminal through the Start Menu following:
Start » All Programs » Accessories » Communication » HyperTerminal

Silly Dialogs

Every time you start HyperTerminal you will be bugged with these silly dialogs unless you supply an Area Code.

Name Connection

When you close this connection you will be asked to save it for later use. So give it a good name!

Configure Connection

Select the ComPort you believe your Pontech Board is connected to.

Adjust the COM Properties to communicate with your Pontech Board. Below is a list of appropriate setting per Pontech Product.

Product	Baudrate	Data Bits	Stop Bits	Parity	Flowcontrol
SV203x	9600 bps	8	1	None	None

STP10x	9600 bps	8	1	None	None
HBC101	115200 bps	8	1	None	None

The dialog below will function with either an SV203x or STP10x.

There are a few more things we need to configure in order to better display the data being exchanged in the terminal window.
Open "**File » Properties**" in the main HyperTerminal window.

Locate the "Settings" Tab and click on the "**ASCII Settings**" Button.
We now need to enable "**Send line ends with line feeds**", "**Echo typed charactors locally**", and "**Append line feeds to incoming lines ends**".

Communicate

At last we are able to communicate! Start by issuing the "**V?**" command. The "**V?**" command is recognized by all Pontech Controllers to report the Model & Firmware Version of the connected Product.

Saving Your Connection

After closing, you will be asked to save your Connection(COM Configuration).

Saving your Connection will add a convenient shortcut from which you can open a HyperTerminal window and begin communicating with one mouse click!
Saved Connections are made available at:
"**Start » All Programs » Accessories » Communications » HyperTerminal » *<Connection Name>***"

Appendix ?

Apendix ? - Nokia LCD

PIC in the real world:

4-bit MCUs mostly use for refreugartor, washing machine, cloth dryer, microwave oven and other house appliances. 8 bits and 16 bits MCU is using for general such as USB. 32 bits is usually for network operating

Besides PIC18F4620, there are also many other PICs from Microchip Inc. on the

market.

There are also many other companies produce MCU, such as Renesas, Atmel, Cypress, Holtek, Philips Semiconductors,and Zilog.

Interrupt - an interrupt is an asynchronous signal from hardware indicating the need for attention or a synchronous event in software indicating the need for a change in execution.
Capture -
Compare -
PWM - It generate Pulse-Wide Modulation signal
MSSP - The Master Synchronous Serial Port (MSSP) module is a serial interface, useful for communicating with other peripheral or microcontroller devices. These peripheral
devices may be serial EEPROMs, shift registers, display drivers, A/D converters, etc.
SPI - The Serial Peripheral Interface Bus or SPI bus is a synchronous serial data link standard named by Motorola that operates in full duplex mode. It has 4 pins SCLK — Serial Clock (output from master) MOSI/SIMO — Master Output, Slave Input (output from master) MISO/SOMI — Master Input, Slave Output (output from slave) SS — Slave Select (active low; output from master)
I^2C
USART
Watchdog Timer
Programmable Brown-out Reset

FAQ

Apendix ? - Interfacing serial port from computer side
Python
Perl
Visual Basic
C++
Appendix ? - **Frequent Ask Questions**